HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

MASTER THESIS

Continual Learning for Document Understanding: Diagnosis-Guided Lexically-Routed Latent Replay

HOANG DUC THANH
hoangducthanh283@gmail.com

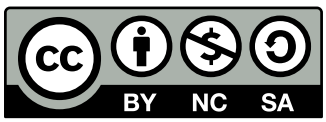
Major : Computer Science

Thesis advisor	:	Advisor Name _____ Signature of advisor
Department	:	Department of Computer Science
Institute	:	School of Information and Communication Technology

Hanoi, June-2026

© 2026 - Hoang Duc Thanh
All rights reserved.
Re-distributed by Hanoi University of Science and Technology under license with the author.

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 3.0 Unported" license.



MASTER THESIS ASSIGNMENT

1. Student's information :
   Name : Hoang Duc Thanh.
   Phone :
   Email: hoangducthanh283@gmail.com
   Class :
   Affiliation : Hanoi University of Science and Technology.
   Duration : 2026.

2. Thesis title : Continual Learning for Document Understanding: Diagnosis-Guided Lexically-Routed Latent Replay

3. Thesis statement :
   This thesis investigates where catastrophic forgetting occurs within a tri-modal document encoder (LayoutLMv3) under continual key-information extraction, and proposes a mechanism-targeted continual-learning method derived from that diagnosis. The objectives are: (i) to build a per-component forgetting diagnostic using representational drift (CKA) and per-group Fisher information; (ii) to characterise which components forget most across continual scenarios; and (iii) to design and evaluate a remedy that concentrates its mechanism on the dominant-forgetting component, benchmarked against ten baselines on the FCS-CL benchmark (FUNSD/CORD/SROIE).

4. Declarations/Disclosures :
   I declare that this thesis and the work presented in it are my own. I confirm that where I have consulted or quoted the published work of others, it is always clearly attributed, and that, with the exception of such quotations, this thesis is entirely my own work.

   Hanoi, date month year 2026
   Author

   Hoang Duc Thanh

5. Attestation of thesis advisor:
   . . . . . .
   . . . . . .

Hanoi, date month year 2026

Thesis Advisor

Advisor Name

Acknowledgments

I would like to thank my supervisor for guidance throughout this work, particularly at the method-decision point where the pilot findings were turned into a concrete design. I am grateful to the School of Information and Communication Technology at the Hanoi University of Science and Technology for the research environment, and to my family and colleagues for their support.

This research was conducted with modest compute resources; I thank the providers of the open document-understanding datasets (FUNSD, CORD, SROIE) and the authors of the open-source backbones (LayoutLMv3, LiLT, BROS) on which this work builds.

Continual Learning for Document Understanding: Diagnosis-Guided Lexically-Routed Latent Replay

Abstract

Deployed document-understanding systems must absorb new document types, schemas, and domains over time without forgetting what they already know, and they usually cannot keep the raw documents they were trained on. Multimodal document encoders such as LayoutLMv3 meet the demand for multimodal understanding by fusing text, layout, and visual pathways into a single network, yet it remains unknown which part of this heterogeneous network drives catastrophic forgetting. Prevailing continual-learning methods do not ask: they regularise the network uniformly or freeze layers by hand-tuned heuristics, and even parameter-efficient and modular adaptation presupposes knowing where to intervene. The document setting adds two difficulties of its own, since key-information extraction is dense token-level sequence labelling rather than the classification most continual-learning methods were built for, and document features are close to full rank, so past tasks cannot be compressed into a small summary.

This thesis studies a stream of key-information-extraction tasks, in which scanned documents arrive as page images, OCR tokens, and bounding boxes with BIO entity labels, under class-incremental, domain-incremental, and mixed regimes. A pre-trained tri-modal encoder learns them in sequence with a bounded memory and no access to past raw documents, and must end as a single model that extracts entities from every task seen so far. We first locate where in the encoder forgetting concentrates, then use that answer to decide what a memory-bounded remedy must retain and which of its stored evidence it should re-expose, arriving at a single method, **lexically-routed latent replay**, in which each of these choices follows from a measurement.

The diagnosis masks the input modalities of a single LayoutLMv3, holding the architecture fixed, and reads forgetting out through linear CKA drift, per-group Fisher information, and Fisher-weighted parameter displacement at every task boundary, with LiLT, BROS, and a text-only BERT as contrasts. It shows that forgetting is output-side: the classifier head has over ten times the Fisher importance of any other parameter group, and CKA drift rises monotonically with depth, from 1.00 at the input encoders to 0.25 at the late layers and 0.16 at the head. The same site recurs on every backbone, including the unimodal one, so it is not a multimodal artefact. Protecting it does not help, however: freezing the head and late layers merely moves the drift to whatever remains plastic, which suggests that a remedy should re-supply old-task evidence at this site rather than protect

its parameters.

A ten-method comparison over three seeds bears this out. Regularisation, distillation, and prompt- or LoRA-based methods fail for a common reason, as the head's output marginal snaps to the newest task while the representation beneath it drifts, and pre-registered eval-time and marginal-level corrections recover nothing. Replay alone closes the gap, reaching an average entity-F1 of about 88 on domain-incremental learning against the joint oracle's 88.7, but only when it replays whole documents: every summary that breaks the binding of feature, position, and label collapses to the naive level, even when built from real activations. Individual documents, unlike the pooled feature space, are nevertheless low-rank, so storing each past document as its own low-rank factors at an intermediate layer (CoLaR) preserves that binding and reaches 87.6 with a 60 MB memory, $2.7\times$ smaller than raw replay and without keeping any raw page.

The proposed method builds on this memory and changes only how it is read. Rather than sampling stored documents uniformly, it routes replay by an OCR-vocabulary signature that depends on the input alone and therefore cannot drift, the same signal that gates LexSlot's slot memory at the diagnosed site, so that each incoming batch is rehearsed against the stored documents most relevant to it. Together with the diagnostic protocol, the benchmark, and the tooling released with the thesis, the method offers a measurement-driven route to continual learning for multimodal document encoders.[1]

Keywords: continual learning, catastrophic forgetting, document understanding, key-information extraction, LayoutLMv3, latent replay.

[1]Source code: `https://github.com/hoangthanh283/doccl` (`doccl` branch); experiments tracked on Weights & Biases at `https://wandb.ai/thanh-workspace/doccl-aaai2027`.

Contents

Abstract **ii**

List of Figures **vii**

List of Tables **ix**

List of Acronyms **xii**

List of Notations **xiv**

1 Introduction **1**
- 1.1 Background and Context 1
- 1.2 Problem Statement 3
- 1.3 Objectives 4
- 1.4 Achievements 4
- 1.5 Overview of the Dissertation 6

2 Background Theory and Literature Review **8**
- 2.1 Document Understanding and Information Extraction 8
  - 2.1.1 Information-Extraction Tasks 8
  - 2.1.2 Multimodal Document Encoders 9
- 2.2 Catastrophic Forgetting and Continual Learning 9
  - 2.2.1 The Stability–Plasticity Dilemma 9
  - 2.2.2 Continual-Learning Scenarios 11
  - 2.2.3 The Five Method Families 12
  - 2.2.4 Measuring Forgetting Inside the Network 16
- 2.3 Continual Learning for Information Extraction 17
  - 2.3.1 Continual NER and the O-Label Problem 17
  - 2.3.2 Continual Relation Extraction 18
  - 2.3.3 Document-Level Continual Learning 18

2.4 Multimodal and Component-Level Forgetting . . . 19
2.4.1 Where Forgetting Lives: Text vs. Visual vs. Fusion . . . 19
2.4.2 Superficial versus Essential Forgetting . . . 20
2.4.3 Measuring Representational Drift: CKA . . . 21
2.5 Backbones for Continual Document IE . . . 21
2.6 Research Gaps and Opportunities . . . 22
2.6.1 Gap 1: No Per-Component Forgetting Diagnosis for Tri-Modal Encoders . . . 22
2.6.2 Gap 2: Sentence-Level Bias of Continual Information Extraction . 22
2.6.3 Gap 3: Heuristic Rather than Measurement-Driven Remedies . . 23
2.6.4 Gap 4: Fragmented Evaluation for Continual Document IE . . . 23
2.7 Positioning of the Current Research . . . 23
2.8 Chapter Summary . . . 24

3 Methodology **25**
3.1 Problem Formulation . . . 25
3.1.1 Defining a Component . . . 26
3.2 A Per-Component Forgetting Diagnostic . . . 27
3.2.1 Motivation . . . 27
3.2.2 Controlled Modality Ablation . . . 27
3.2.3 Forgetting Decomposition Metrics . . . 28
3.2.4 Diagnostic Protocol . . . 30
3.2.5 Hypotheses and Decision Rule . . . 31
3.3 Mechanism-Targeted Continual Learning . . . 34
3.3.1 Why Component-Agnostic Continual Learning Is Insufficient . . 34
3.3.2 Design Space: Where a Mechanism Can Attach in a Single-Stream Encoder . . . 34
3.3.3 Testable Component Hypotheses . . . 35
3.3.4 Selection Criterion . . . 35
3.3.5 Pre-Registered Fallback (Not Triggered) . . . 36
3.4 Lexical-Slot Head-Memory Continual Learning (LexSlot) . . . 36
3.4.1 Slot Memories at the Diagnosed Site . . . 37
3.4.2 Per-Task Isolation: Write-Once Slots . . . 38
3.4.3 The Lexical Gate and the Sharing Ablation . . . 39
3.4.4 Implementation Notes . . . 39
3.5 Lexically-Routed Latent Replay . . . 40
3.6 Summary . . . 40

4 Implementation and Integration **42**
4.1 Framework Selection . . . 42
4.1.1 Backbone Selection . . . 42
4.1.2 Software Stack: Selection Criteria and Justification . . . 42
4.1.3 Why a Bespoke Continual-Learning Harness . . . 44
4.2 Document-IE Pipeline Integration . . . 45
4.3 System Architecture Overview . . . 47
4.3.1 Continual-Learning Core . . . 47
4.3.2 Per-Component Instrumentation . . . 49
4.3.3 Datasets and Scenario Builders . . . 50
4.3.4 Compute and Deployment . . . 50
4.3.5 Experiment Tracking and Monitoring . . . 51
4.4 Reproducibility . . . 51
4.5 Summary . . . 52

5 Experimental Setup **53**
5.1 Datasets . . . 53
5.2 Continual-Learning Task Construction . . . 54
5.2.1 Class-Incremental (CIL-CORD) . . . 54
5.2.2 Domain-Incremental (DIL) . . . 56
5.2.3 Mixed (Compositional) . . . 57
5.2.4 Extended-Benchmark Scenarios (Language Shift and Scale) . . . 58
5.2.5 Task Ordering . . . 59
5.3 Model Architecture and Training Configuration . . . 59
5.3.1 Architecture Specification . . . 59
5.3.2 Training Objective . . . 60
5.3.3 Optimisation Protocol . . . 60
5.3.4 Checkpointing and Model Selection . . . 61
5.4 Baselines and Continual-Learning Strategies . . . 61
5.5 Evaluation Methodology . . . 62
5.5.1 Validation Protocol . . . 62
5.5.2 Metrics . . . 64
5.5.3 Few-Shot Considerations . . . 64
5.5.4 Repeated-Seed Protocol and Statistical Testing . . . 64
5.6 Summary . . . 65

6 Results and Discussion **66**
6.1 Where Forgetting Lives: The Per-Component Diagnosis . . . 66

6.1.1 Per-Component Forgetting Profile . . . 67
6.1.2 Per-Layer Breakdown . . . 69
6.1.3 Stability of the Finding . . . 72
6.2 Mechanism-Targeted Remedy versus Baselines . . . 77
6.2.1 Independent Per-Dataset Upper Bound . . . 77
6.2.2 Main Comparison . . . 78
6.2.3 Forgetting and Plasticity Breakdown . . . 81
6.2.4 Is the Site Causal? A Depth-Targeting Probe . . . 83
6.2.5 The Lexical Gate at the Diagnosed Site: LexSlot . . . 87
6.2.6 Generalisation Across Backbones . . . 89
6.3 Ablation Studies . . . 89
6.3.1 Component Ablation: Memory Alone, Gate Alone, and Both . . . 91
6.3.2 Where: Depth-Targeting Probe . . . 91
6.3.3 What: The Form of the Memory . . . 92
6.3.4 Which: The Routing Mode . . . 93
6.4 Computational Efficiency . . . 93
6.5 Extended Scenarios: Scale and Language Shift . . . 94
6.6 Why Other Remedies Fail, and Lexically-Routed Latent Replay . . . 96
6.7 Discussion . . . 100

7 Conclusions and Future Work **103**
7.1 Limitations . . . 103
7.2 Conclusions . . . 104
7.3 Future Work . . . 106

References **114**

Appendices **115**

A Supplementary Material **115**
A.1 Key Hyperparameters . . . 115
A.1.1 CIL-CORD Session Class Lists . . . 115
A.2 Per-Component Metric Definitions . . . 115
A.3 Extended Per-Task Result Tables . . . 117

List of Figures

2.1 LayoutLMv3 tri-modal architecture and its four components 10
2.2 The stability–plasticity trade-off 11
2.3 Taxonomy of continual-learning method families 16

3.1 The diagnostic conditions as active and zeroed pathways 29
3.2 Annotated structure of the forgetting matrix 30
3.3 The per-component diagnostic pipeline 32
3.4 Decision tree mapping diagnostic outcomes to the remedy 33
3.5 LexSlot architecture 37
3.6 Two consistent lexical isolation mechanisms 38

4.1 The five-step document preprocessing pipeline. Dataset-provided OCR tokens, word boxes, and the page image are tokenised, normalised, and aligned into the tri-modal inputs of the backbone; colour chips mark which input stream each step feeds. 46
4.2 System architecture 47

5.1 The CIL-CORD class-incremental scenario 56
5.2 The DIL domain-incremental scenario 57
5.3 The Mixed compositional scenario 58

6.1 Per-component forgetting localiser 69
6.2 Forgetting-matrix heatmaps across backbones 70
6.3 Cross-backbone forgetting signature 72
6.4 Per-layer representational drift (CKA depth gradient) 73
6.5 Continual-learning metrics by backbone 75
6.6 Where forgetting lives: synthesis 77
6.7 Average accuracy by method and scenario 82
6.8 LexSlot versus replay versus oracle on DIL 83
6.9 DIL close-up: LexSlot approaches the oracle 84

6.10 Backward transfer by method and scenario 84
6.11 Average forgetting by method and scenario 85
6.12 Forgetting trajectory under naive fine-tuning 85
6.13 Replay-dominant ordering across backbones 90
6.14 Extended scenarios: scale and language shift 96
6.15 The latent-replay elimination ladder 98
6.16 Replay memory versus retention 99
6.17 Why replay works 101

List of Tables

2.1 The five continual-learning method families 15
2.2 The continual-learning-for-IE landscape 19

3.1 The diagnostic conditions 28
3.2 Diagnostic outcomes and the remedy each selects 33

4.1 Backbone comparison 43
4.2 Software stack 44
4.3 Per-run artefacts 51

5.1 Dataset statistics 55
5.2 CIL-CORD session split 56
5.3 DIL unified-schema mapping 58
5.4 Architecture specification 60
5.5 Baseline suite 63

6.1 Forgetting matrices across backbones 68
6.2 Per-component Fisher importance 68
6.3 Per-layer representational drift (CKA) 72
6.4 Cross-architecture depth-gradient contrast 74
6.5 Forgetting by diagnostic condition 75
6.6 Independent per-dataset upper bounds 78
6.7 Main comparison: average accuracy across scenarios 81
6.8 Backward transfer across scenarios 86
6.9 LexSlot ablation: sharing and depth 88
6.10 Method comparison across backbones 90
6.11 Component ablation of the proposed method 92
6.12 Depth-targeting probe 93
6.13 Computational overhead 94
6.14 CoLaSlot capacity-gate verdicts 100

A.1 Default hyperparameters 116
A.2 CIL-CORD per-session class lists 117

List of Acronyms

A-GEM Averaged Gradient Episodic Memory.
AA Average Accuracy.
AF Average Forgetting.

BERT Bidirectional Encoder Representations from Transformers.
BIO Begin–Inside–Outside tagging scheme.
BWT Backward Transfer.

CF Catastrophic Forgetting.
CIL Class-Incremental Learning.
CKA Centered Kernel Alignment.
CL Continual Learning.
CNER Continual Named-Entity Recognition.
CRE Continual Relation Extraction.

DER Dark Experience Replay.
DIL Domain-Incremental Learning.

EE Event Extraction.
ER Experience Replay.
EWC Elastic Weight Consolidation.

FIM Fisher Information Matrix.
FWT Forward Transfer.

GDPR General Data Protection Regulation.
GEM Gradient Episodic Memory.
GPM Gradient Projection Memory.

iCaRL incremental Classifier and Representation Learning.
IE Information Extraction.

KBQA Knowledge-Based Question Answering.
KD Knowledge Distillation.
KGC Knowledge-Graph Completion.
KIE Key-Information Extraction.

L2P Learning to Prompt.
LoRA Low-Rank Adaptation.
LwF Learning without Forgetting.

MAS Memory-Aware Synapses.
MIM Masked Image Modelling.
MLLM Multimodal Large Language Model.
MLM Masked Language Modelling.
MoE Mixture of Experts.

NER Named-Entity Recognition.

O-LoRA Orthogonal Low-Rank Adaptation.
OCR Optical Character Recognition.
OGD Orthogonal Gradient Descent.
OIE Open Information Extraction.

PEFT Parameter-Efficient Fine-Tuning.

PII Personally Identifiable Information.

PTM Pre-Trained Model.

RE Relation Extraction.

SI Synaptic Intelligence.

SVD Singular Value Decomposition.

TIL Task-Incremental Learning.

ViT Vision Transformer.

VQA Visual Question Answering.

VRDU Visually-Rich Document Understanding.

WPA Word–Patch Alignment.

List of Notations

$D = (W, B, I)$ a document: OCR token sequence W, bounding boxes B, and page image I.

N number of tokens in a document.

T number of tasks in the continual sequence.

$\mathcal{T}_t$ the t-th task (data and, in CIL, its label space).

$\theta,\ \theta^{(t)}$ encoder parameters; parameters after training task t.

Y sequence of BIO labels $(y_1, \ldots, y_N)$.

$\mathcal{L}_{\textbf{task}}$ dataset-specific BIO tag set.

$R[i, j]$ accuracy matrix: entity-F1 on task j after training task i.

$\mathrm{CKA}(X, Y)$ linear Centered Kernel Alignment between activation matrices.

F_g empirical Fisher information of parameter group g.

$\phi(\textbf{boxes})$ layout signature: 4×4 grid histogram of box centres (16-dim).

λ regularisation strength (the global scale on a Fisher penalty, e.g. EWC or the depth-targeting probe).

M replay memory buffer of stored past exemplars (size $|M|$).

P parameter budget (bound on $\|\Delta\theta\|$).

d encoder hidden dimension (768 for the base model).

L number of transformer layers (12 for the base model).

r LoRA rank (low-rank adapter dimension).

$q(x)$ layout-aware prompt query for input x.

h_{CLS} the cls-token embedding.

$W_{\textbf{layout}}$ trainable projection of the layout signature $\phi(\text{boxes})$.

Chapter 1

Introduction

1.1 Background and Context

Modern organisations process a continuous stream of *visually-rich documents* (forms, invoices, receipts, contracts, and reports) whose meaning depends not only on the words they contain but on *where* those words sit on the page. Extracting structured information from such documents is the task of Key-Information Extraction (KIE), the cornerstone of document-understanding systems deployed in finance, logistics, healthcare, and public administration. The dominant approach casts KIE as token-level sequence labelling over a tri-modal input: text obtained by Optical Character Recognition (OCR), the two-dimensional layout encoded as bounding boxes, and the page image. Multimodal document encoders such as LayoutLMv3 (Huang et al., 2022), LiLT (Wang et al., 2022a), and BROS (Hong et al., 2022) fuse these signals and define the state of the art.

Deployed document systems are not static. New document types appear, annotation schemas are extended, and the distribution of incoming documents drifts as an organisation enters new markets or adopts new templates. A model must therefore *evolve*, learning new entity types and new domains over time, without re-training from scratch on all historical data, which is often impossible because earlier data is unavailable, expensive to store, or restricted for privacy reasons (Personally Identifiable Information (PII) under General Data Protection Regulation (GDPR)). This is the setting of Continual Learning (CL): learning a sequence of tasks while retaining performance on those seen earlier.

The central obstacle to CL is Catastrophic Forgetting (CF) (McCloskey and Cohen, 1989): when a neural network is fine-tuned on a new task, gradient updates overwrite the parameters that encoded earlier tasks, and performance on those tasks collapses. A large literature has developed remedies: regularising important weights (Kirkpatrick

et al., 2017), distilling from past models (Li and Hoiem, 2016), replaying stored exemplars (Buzzega et al., 2020, Rolnick et al., 2019), learning task-specific prompts (Smith et al., 2023, Wang et al., 2022c,d), or allocating low-rank adapters per task (Wang et al., 2023). Almost all of these methods, however, were developed and evaluated on *unimodal* image- or text-classification benchmarks, and they treat the network as a homogeneous block: regularisation is applied uniformly across parameters, or layers are frozen by hand-tuned heuristics.

For a *tri-modal* document encoder this uniform treatment is unsatisfying, because the network is manifestly heterogeneous: it contains a text stream, a visual/patch stream, layout (2D position) embeddings, shared self-attention and feed-forward blocks at increasing depth, and a task head. These parts are trained on different signals, carry different inductive biases, and, there is every reason to suspect, are unlikely to age at the same rate under a shifting task distribution. Yet the prevailing remedies are blind to this structure: elastic-weight penalties (Kirkpatrick et al., 2017) apply one importance-weighted quadratic term across all parameters, distillation (Li and Hoiem, 2016) matches whole-network outputs, and heuristic layer freezing draws an arbitrary depth-wise cut that ignores the modality a layer serves. Even the modern parameter-efficient and modular families (low-rank adapters (Wang et al., 2023) and prompt pools (Smith et al., 2023, Wang et al., 2022c,d)) only sharpen the question rather than answer it: deciding *which* weights to adapt or *where* to inject a prompt implicitly presupposes that one already knows which part of the encoder is at risk. Evidence from unimodal vision transformers shows that forgetting is far from uniform across depth and is concentrated in identifiable layers (Ramasesh et al., 2021), and recent probes of multimodal models report modality-dependent degradation (Zhai et al., 2024); but for a document encoder that jointly reasons over text, layout, and pixels, *no prior work has measured where: at which depth, and in which input stream or the head, forgetting concentrates*, nor whether that pattern differs from a unimodal encoder. That measurement is the missing prerequisite this thesis supplies.

Answering that question (which component forgets most, and whether the pattern is stable across backbones and continual scenarios) is the prerequisite for designing a CL method that spends its capacity where forgetting actually happens rather than spreading it thinly across the whole network. This thesis therefore takes a *measure-first, then-fix* stance: it first *diagnoses* where forgetting lives in a multimodal document encoder, then uses the diagnosis to rule remedies in or out, and only then proposes a remedy whose design follows from both.

1.2 Problem Statement

Input. A stream of document-understanding tasks $\mathcal{T}_1, \ldots, \mathcal{T}_T$ arriving one after another. Each task is a set of scanned documents, each given as a page image, its OCR tokens, and their two-dimensional bounding boxes, annotated with token-level Begin–Inside–Outside tagging scheme (BIO) entity labels for KIE. The stream follows one of three regimes: Class-Incremental Learning (CIL), where the label space grows from session to session (FUNSD, CORD, WildReceipt sessions); Domain-Incremental Learning (DIL), where the label space is fixed and the document type shifts (forms to receipts, or one XFUND language to the next); and a mixed regime. The learner starts from a pre-trained tri-modal encoder (LayoutLMv3; LiLT, BROS, and a text-only BERT as secondary backbones) with a linear token-classification head. Past raw documents are not retained, for privacy and storage reasons; at most a bounded memory may be kept.

Output. A single model that, after task $\mathcal{T}_T$, extracts entities from documents of *every* task seen so far. It is measured by average entity-F1 (AA), backward transfer (BWT), forgetting, and the size of its memory in bytes.

Research questions. The thesis asks two questions, in this order:

RQ1 (diagnosis). Where in a heterogeneous multimodal encoder (the text, layout, and visual streams, the shared layers by depth, or the classifier head) does catastrophic forgetting concentrate under sequential KIE, and is that site specific to multimodal fusion or general to the architecture?

RQ2 (remedy). Given the site, what must a memory-bounded remedy supply there to recover joint-training accuracy, and *which* stored evidence should it re-expose for a given incoming document?

Motivation and challenges. Five difficulties make these questions both necessary and hard:

1. *Deployment pressure.* Deployed systems must absorb new schemas and domains, and re-training from scratch on all historical data is usually infeasible.
2. *Structure-blind methods.* Existing CL methods regularise uniformly or freeze layers heuristically, and parameter-efficient or modular methods presuppose knowing where to intervene; none tells us where that is for a document encoder.
3. *Heterogeneous encoders.* A tri-modal encoder has several input streams, shared layers, and a head; one modality could carry all of the forgetting, or none of it.

4. *Dense token labelling.* KIE labels every token of a long document, with a dominant background class and rare entity classes; most CL methods were built for image or sentence classification and do not transfer directly.

5. *Memory that cannot be summarised cheaply.* Document features are close to full rank (a rank-16 projection keeps only 43% of the variance of the head-input features), so a small, faithful summary of past tasks does not exist, while raw replay is costly in storage and in privacy.

1.3 Objectives

The thesis pursues four objectives, one per step of the approach:

1. **Diagnose where forgetting lives (RQ1).** Build a per-component diagnostic protocol: controlled masking of the input modalities of a single LayoutLMv3 (holding the architecture fixed), read out through linear Centered Kernel Alignment (CKA) drift, per-group Fisher information, and old-task-Fisher-weighted parameter displacement at every task boundary, and repeated on LiLT, BROS, and a text-only BERT (Chapter 3, §3.2).

2. **Rule out the remedies the diagnosis rules out (RQ2).** Test regularisation, distillation, prompt and LoRA methods, eval-time and marginal-level corrections, and summarised replay against the diagnosed site, and identify the single reason they fail (Chapter 6, §§6.2–6.6).

3. **Replay at the site, within a bounded memory (RQ2).** Establish what replay must preserve to work, and show that a compressed latent memory respecting it (CoLaR, per-document SVD factors at layer k) keeps replay-level accuracy at a fraction of the storage (§6.6).

4. **Route the replay lexically (RQ2).** Use LexSlot's drift-immune OCR-vocabulary signature, designed as the gate of a slot memory at the diagnosed site, to decide which stored document latents are replayed for the incoming document (§3.4).

All results are reported over three seeds on a released benchmark (FUNSD, CORD, SROIE, extended with WildReceipt and XFUND) under one configuration system (Chapters 4 and 5).

1.4 Achievements

Contributions at a glance. The thesis makes four contributions:

1. a **forgetting-localisation diagnosis** of tri-modal document encoders, showing that forgetting is output-side, architecture-general, and conserved under constraint;
2. an **explanation of why buffer-free remedies fail** with one identified cause, the readout-marginal snap on a drifting representation, and the law that replay must preserve *whole-document consistency*;
3. **lexically-routed latent replay**, a remedy derived from both: a compressed, per-document latent memory (CoLaR) at the diagnosed site, with LexSlot's lexical signature as the gate that selects what is replayed;
4. the **FCS-CL benchmark** and reproducible diagnostic tooling.

The remainder of this section expands each contribution.

Where forgetting lives (RQ1). We introduce, to our knowledge, the first protocol that localises CF in a multimodal document encoder along network *depth* and across its *separable* parts: the three input streams (text, layout, visual), the shared self-attention and feed-forward blocks at each depth, and the classifier head. We are deliberate about what is separable: LayoutLMv3 is a *single-stream* encoder, so after the input embeddings text and visual tokens share the same attention projections and no "fusion" *parameter* group is identifiable; we therefore measure the modalities' contributions *behaviourally*, by masking the input streams of a single LayoutLMv3, and localise *parameter* forgetting by depth, stream, and head. Three findings follow. Forgetting is *output-side*: under naive sequential fine-tuning an 88-F1 task collapses to $\sim$2 F1 after one further task, the head carries more than an order of magnitude more Fisher importance than any other group, and per-token CKA drift rises monotonically with depth (from 1.00 at the input encoders to 0.25 at the late layers and 0.16 at the head), while the input encoders barely move. It is *architecture-general*: the same head-concentrated pattern holds on LiLT, BROS, and a text-only BERT, so the site is the general output side, not a multimodal artefact, and we do *not* claim a fusion-specific effect. And it is *conserved*: freezing everything after the first task forgets nothing but learns nothing (AA 41.1), while freezing only the diagnosed head and late layers leaves naive-level forgetting (BWT -76.8) and the early and mid layers, stable under naive training, now drift (CKA $\approx 1.0 \rightarrow 0.19$). The drift is moved, not removed; a fixed consolidation budget recovers $\sim$85 AA when placed on the head/late site but collapses to 42.6 AA when spread uniformly across depth.

Why buffer-free remedies fail (RQ2). The benchmark ordering follows the diagnosis: replay $\gg$ regularisation $\gg$ distillation. Replay, which re-exposes old-class targets at the head, reaches the joint upper bound on domain-incremental learning (AA ≈ 88 vs the oracle's 88.7, backward transfer ≈ 0); EWC curbs forgetting under a domain shift; LwF

and the prompt/LoRA families track the naive bound. A per-boundary root-cause analysis names the single mechanism: the head's output marginal on old tasks snaps to the just-trained task's label marginal (cosine 0.997), and classes that are not re-exercised are extinguished to 0.0 F1 under every modality mask. Pre-registered eval-time and marginal-level corrections all recover nothing, because the old-class probability mass has already been destroyed. What replay must preserve is then isolated by an elimination ladder at a fixed layer: replaying real whole documents retains AA 87.3, while a rank-16 summary (41.9), per-class Gaussians (39.4), and even real k-means centroids (36.7) collapse. The binding of feature, position, and label within one document is the necessary ingredient.

Lexically-routed latent replay (RQ2). Individual documents, unlike the pooled feature space, *are* low-rank. CoLaR stores each replayed document as rank-r SVD factors of its layer-k activations, which keeps whole-document binding and reaches AA 87.6 with a 60 MB memory, $2.7\times$ smaller than raw replay and within 1.4 points of the joint oracle, with no raw document retained. LexSlot, an isolated slot memory at the head and late layers gated by a drift-immune OCR-vocabulary signature, reaches AA 87.3 ± 1.0 with backward transfer -2.3 ± 1.2 with a compact 200-exemplar buffer, forgetting less than plain replay (-2.8/-3.4). The thesis joins the two: LexSlot's signature becomes the *gate* and CoLaR the *memory*, so that for each incoming document the replay draws the stored latents whose vocabulary is nearest (or farthest, or from the nearest task), rather than sampling the store uniformly. The diagnosis decides *where* the memory attaches; the lexical signature, immune to the representation drift that defeats every buffer-free method, decides *what* is replayed.

The FCS-CL benchmark and reproducible tooling. We provide a unified CL benchmark over five document datasets (FUNSD, CORD, SROIE, WildReceipt, XFUND) with class-incremental, domain-incremental, mixed, large-scale, and cross-lingual scenarios, a baseline suite spanning the major method families, and per-component instrumentation, all under one configuration system and a tracked experiment log, enabling the like-for-like comparison that the fragmented prior literature has lacked.

1.5 Overview of the Dissertation

Chapter 2 reviews document understanding and Information Extraction (IE), the CL problem and its method families, the nascent literature on CL for IE, evidence on where forgetting lives in multimodal models, and the backbones relevant to this work; it closes by identifying the research gaps the thesis addresses. **Chapter 3** presents the methodological contributions: the per-component forgetting diagnostic (RQ1), the remedy design space it

opens, LexSlot's lexically-gated slot memory, and the lexically-routed latent replay that joins that gate to a compressed per-document memory (RQ2). **Chapter 4** describes the framework, the document-IE pipeline, and the system architecture, including the instrumentation hooks and experiment tracking. **Chapter 5** details the datasets, the continual task construction, the model and training configuration, the baselines, and the evaluation methodology. **Chapter 6** answers RQ1 with the diagnosis and its stability across seeds, task orders, and backbones, then answers RQ2: the benchmark ordering, the root cause of the buffer-free failures, the whole-document consistency law, CoLaR, and lexically-routed replay, with ablations, a computational-cost analysis, and threats to validity. **Chapter 7** states limitations, conclusions, and future work.

Chapter 2

Background Theory and Literature Review

This chapter establishes the foundations on which the thesis builds. It proceeds from the application domain (document understanding and IE), through the CL problem and its method families, to the specific and still-nascent literature on CL for IE. It then reviews what is known about *where* forgetting occurs in multimodal models, surveys the backbones relevant to continual document IE, and closes by isolating the research gaps that motivate the contributions of Chapter 3.

2.1 Document Understanding and Information Extraction

2.1.1 Information-Extraction Tasks

IE converts unstructured content into structured records. Its canonical sub-tasks are Named-Entity Recognition (NER), which labels spans with semantic types; Relation Extraction (RE), which links entity pairs; Event Extraction (EE), which identifies event triggers and their argument roles; and KIE, which populates schema fields from forms, invoices, and receipts. Standard benchmarks differ markedly in label-space size: NER corpora range from the four types of CoNLL-2003 to the eighteen of OntoNotes, RE from the forty-one relations of TACRED to the ninety-six, cross-sentence relations of DocRED, which is itself a source of difficulty when such tasks are learned in sequence.

In the document setting these tasks differ from their sentence-level counterparts in three ways that matter for CL. First, documents are *spatially structured*: the role of a token (a form's question versus its answer, a receipt's label versus its total) is often signalled by position and typography rather than by surrounding words. Second, documents are *multimodal*: text, layout, and image jointly determine meaning. Third, document schemas

are *diverse and evolving*: a forms dataset and a receipts dataset share little vocabulary and almost no label space, so a system that learns them sequentially faces large distribution and output-space shifts. A further document-specific difficulty is *cross-sentence context* (coreference and multi-hop reasoning), which sentence-level methods ignore entirely.

This thesis adopts the dominant framing of KIE as token-level sequence labelling with the BIO tagging scheme, where each OCR token is assigned a Begin/Inside/Outside tag for its entity type and the model is trained with token-level cross-entropy. This framing aligns with the mainstream Visually-Rich Document Understanding (VRDU) literature, yields a unified output format across datasets, and is directly compatible with the encoder-only backbones used here.

2.1.2 Multimodal Document Encoders

A multimodal document encoder ingests three streams (OCR tokens, their bounding boxes (2D position embeddings), and image patches) and fuses them through transformer self-attention. LayoutLMv3 is the most widely adopted encoder-only backbone in the VRDU literature (Huang et al., 2022): it dispenses with a convolutional vision front-end, instead consuming raw 16×16 image patches in the manner of a Vision Transformer (ViT), and is pre-trained with three objectives: Masked Language Modelling (MLM) over text, Masked Image Modelling (MIM) over image patches, and Word–Patch Alignment (WPA), an explicit cross-modal supervision that aligns words with the patches that contain them. It reports strong single-task results (entity-level F1 of roughly 92 on FUNSD and 97 on CORD). LiLT (Wang et al., 2022a) decouples layout from text so that a language-agnostic layout transformer can be paired with different text encoders; BROS (Hong et al., 2022) emphasises relative spatial encoding and an area-masking objective; and DocFormer (Appalaraju et al., 2021), an earlier design, couples modalities through a shared spatial attention. A key structural feature shared by these models, and central to this thesis, is that they contain *distinguishable components* (a text stream, a visual stream, layout embeddings, and cross-modal fusion) whose individual behaviour under CL has not previously been measured. Figure 2.1 annotates these four components on the LayoutLMv3 architecture.

2.2 Catastrophic Forgetting and Continual Learning

2.2.1 The Stability--Plasticity Dilemma

CL trains a model on a sequence of tasks while retaining performance on those seen earlier. Its central obstacle is CF (McCloskey and Cohen, 1989): gradient-based optimisation

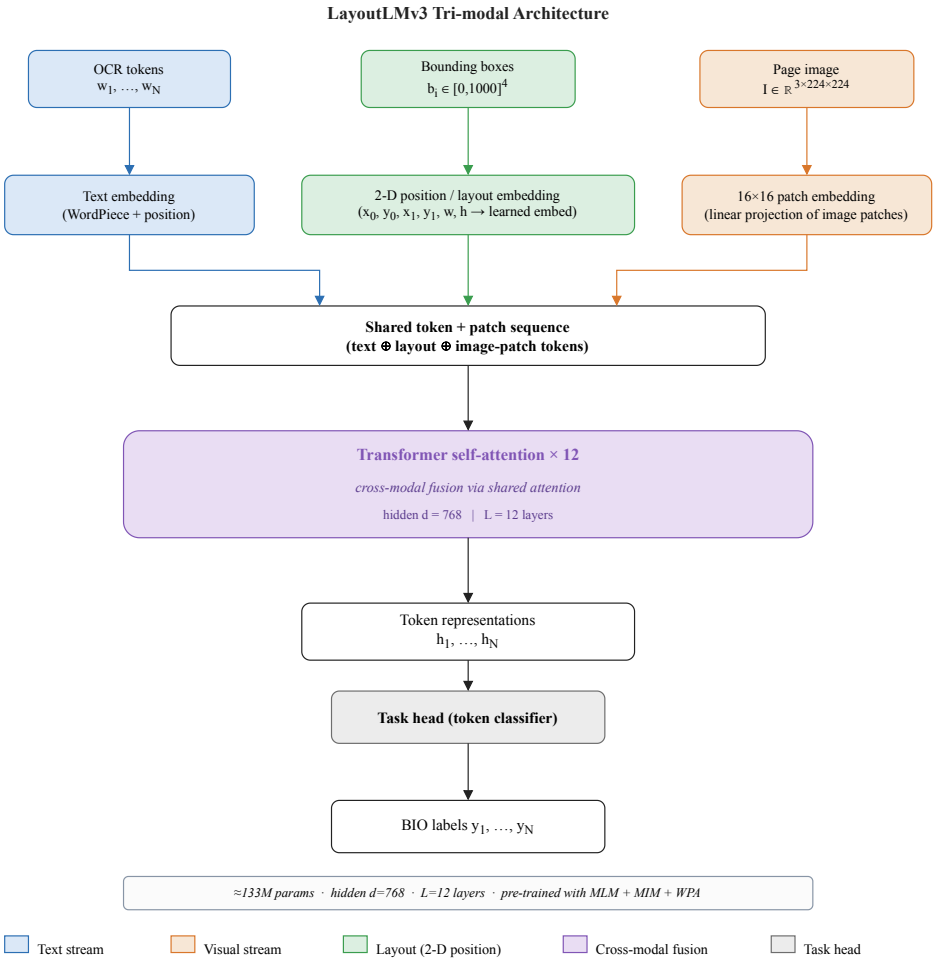


Figure 2.1: The LayoutLMv3 tri-modal architecture. OCR tokens, 2D bounding-box position embeddings, and image patches are projected into a shared sequence and fused by transformer self-attention. The four components diagnosed in this thesis (text stream, visual stream, layout embeddings, and cross-modal fusion) are highlighted; their individual susceptibility to forgetting is the object of Chapter 3.

on a new task overwrites the weight configuration that encoded earlier tasks, and prior performance collapses. CF is governed by the *stability–plasticity dilemma*: a model must remain *plastic* enough to acquire new tasks yet *stable* enough to retain old ones. Pure fine-tuning maximises plasticity and suffers CF; freezing the network maximises stability but prevents learning. Every CL method is, in effect, a particular operating point on this trade-off (Figure 2.2), and a recurring theme of this thesis is that the right operating point may differ *per component* of a multimodal encoder. Forgetting also has more than one face: it can be *superficial* (the output format drifts while knowledge is intact) or *essential* (the underlying knowledge is lost), a distinction made precise in §2.4.2.

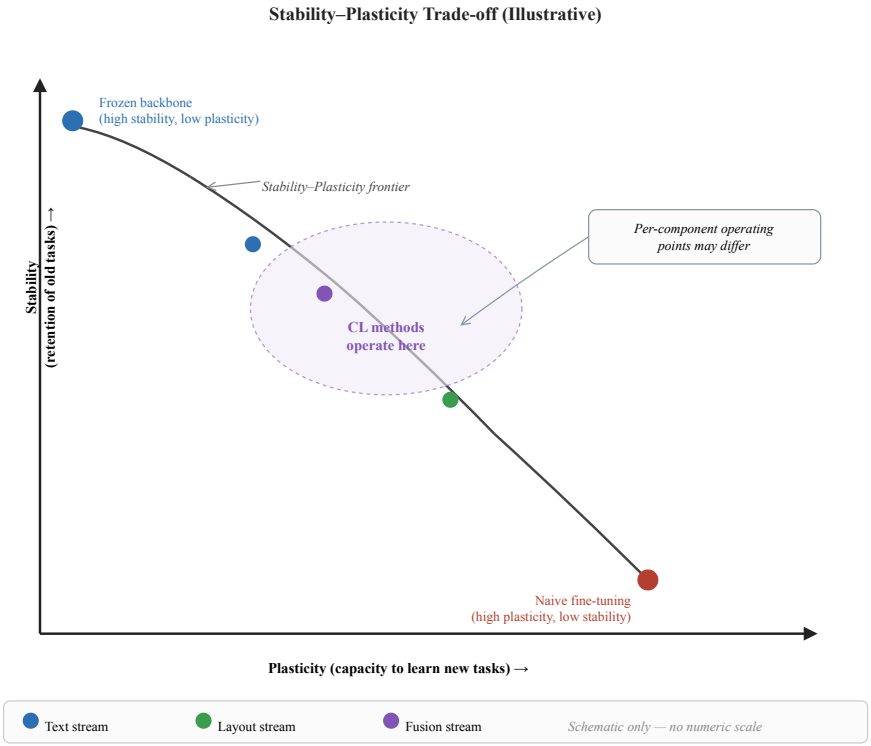


Figure 2.2: Illustrative schematic of the stability–plasticity trade-off (no measured data). Pure fine-tuning sits at the high-plasticity, low-stability extreme and suffers catastrophic forgetting; freezing the backbone sits at the opposite extreme and cannot acquire new tasks. Continual-learning methods occupy intermediate operating points. A central hypothesis of this thesis is that the optimal operating point is not a single global setting but may differ across the components of a multimodal encoder.

2.2.2 Continual-Learning Scenarios

The literature distinguishes three canonical scenarios by what changes across the task sequence and what is known at test time. In Task-Incremental Learning (TIL) the task identity is available at inference, so a separate head can be selected: the easiest case. In DIL the input distribution shifts but the label space is shared and task identity is unknown. In CIL new classes are introduced over time and the model must discriminate among all classes seen so far without task identity; CIL is generally the hardest because the output space itself grows.

For document KIE cast as BIO sequence labelling, CIL additionally provokes the *semantic shift of the "O" tag*, a form of structural interference that has no counterpart in image-classification CL. Let $\mathcal{L}_t$ denote the foreground (entity) label set introduced up to session t. Under class incremental learning these sets grow monotonically,

$$\mathcal{L}_1 \subset \mathcal{L}_2 \subset \cdots \subset \mathcal{L}_T, \qquad O_t = \mathcal{Y} \setminus \mathcal{L}_t, \tag{2.1}$$

where $\mathcal{Y}$ is the universe of foreground types and O_t is the background ("O") class, defined as the *shrinking complement* of $\mathcal{L}_t$. Because $\mathcal{L}_s \subset \mathcal{L}_t$ for $s < t$, a token labelled O in session s may carry a genuine entity label in a later session $t > s$; the same surface token is therefore a negative example in one session and a positive example in another. Consequently the conditional $p(y = O \mid x)$ is *non-stationary* across the session sequence, and the supervision the model received for old spans becomes inconsistent with what later sessions demand. This moving decision boundary, rather than mere distribution shift, is the structural difficulty unique to continual sequence labelling, and it is amplified in documents where a single entity type recurs across many tokens. This thesis studies CIL, DIL, and a mixed scenario, and explicitly leaves TIL to future work.

2.2.3 The Five Method Families

Following the widely-used taxonomy of Wang et al. (2024), CL methods fall into five families summarised in Table 2.1 and Figure 2.3.

Regularisation-based. These penalise changes to parameters deemed important for past tasks. Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017) weights a quadratic penalty by the diagonal of the Fisher Information Matrix (FIM), optimising

$$\mathcal{L} \;=\; \mathcal{L}_{\text{new}} \;+\; \lambda \sum_i F_i \left(\theta_i - \theta_i^*\right)^2, \tag{2.2}$$

where θ^* are the parameters at the end of the previous task, F_i is the Fisher diagonal estimating the importance of parameter θ_i, and λ sets the stability–plasticity balance. Synaptic Intelligence (SI) (Zenke et al., 2017) accumulates importance online from the path integral of parameter change, and Memory-Aware Synapses (MAS) (Aljundi et al., 2018) estimates importance from output sensitivity without labels; both can be read as alternative estimators of the same per-parameter weights F_i. A known failure mode, noted by later analyses rather than the original work, is that this importance accumulates across many tasks until the penalty effectively freezes the network (Schwarz et al., 2018). A related *function*-space regulariser, Learning without Forgetting (LwF) (Li and Hoiem, 2016), instead distils the previous model's outputs on new-task inputs, so we group it here as output-side regularisation rather than as a parameter-importance penalty. *Implication for document IE.* The per-parameter importance F_i aggregates naturally into the named component groups of a tri-modal encoder, making regularisation a natural vehicle for a *component-targeted* remedy; and because token-level supervision provides a label at every position of a 512-token page, the Fisher estimate is populated from few documents.

Replay-based. These store and re-present a memory of past exemplars. Gradient Episodic Memory (GEM) (Lopez-Paz and Ranzato, 2017) projects the new-task gradient g so that it does not increase the loss on the stored memory, enforcing the inequality constraint

$$\langle g,\, g_{\text{ref}} \rangle \;\geq\; 0, \tag{2.3}$$

where g_{ref} is the gradient evaluated on the memory of a past task; an update that violates this is projected onto the nearest gradient that satisfies it. GEM also introduced the now-standard backward- and forward-transfer metrics. Averaged Gradient Episodic Memory (A-GEM) (Chaudhry et al., 2019) relaxes this to a single averaged constraint; Experience Replay (ER) (Rolnick et al., 2019) simply interleaves replayed examples; DER++ (Buzzega et al., 2020) adds a logit-distillation term; and iCaRL (Rebuffi et al., 2017) combines herding-based exemplar selection with nearest-mean classification. Replay is empirically among the strongest families (Masana et al., 2023, Wang et al., 2024) but stores raw data. *Implication for document IE.* Documents carry PII under GDPR, so buffering raw scans is problematic in deployment; and with document training sets of only 149–800 pages (Chapter 5), a fixed-size buffer can cover a large fraction of, or even exceed, a past task, blurring the line between replay and joint training (§5.5.3).

Optimisation-based. These constrain the geometry of updates. Gradient Projection Memory (GPM) (Saha et al., 2021) computes, by Singular Value Decomposition (SVD) of layer activations, the subspace spanned by past tasks and projects new gradients orthogonally to it. Concretely, after each task GPM collects the layer-ℓ activations into a matrix R^{ℓ}, computes its SVD $R^{\ell} = U\Sigma V^{\top}$, and retains the leading left-singular vectors whose retained energy exceeds a threshold ϵ_{th}; these accumulate across tasks into a column-orthonormal basis M_{ℓ}. Subsequent updates are projected onto the orthogonal complement of this basis,

$$\nabla_{W_{\ell}} \mathcal{L} \;\leftarrow\; \nabla_{W_{\ell}} \mathcal{L} - \big(\nabla_{W_{\ell}} \mathcal{L}\big) M_{\ell} M_{\ell}^{\top}, \tag{2.4}$$

where M_{ℓ} spans the core gradient space of past tasks at layer ℓ and ϵ_{th} controls how much of the past subspace is protected (Saha et al., 2021). Orthogonal Gradient Descent (OGD) (Farajtabar et al., 2020) follows the same orthogonality principle directly in gradient space. Such methods are stable but the protected subspace grows until its capacity is exhausted. A complementary recent line targets the *geometry* of the minimum rather than the gradient subspace: C-Flat (Bian et al., 2024) adds a sharpness-aware, zeroth-order flatness penalty on top of any CL method, steering optimisation toward flat minima that empirically forget less, a plug-and-play optimiser-side remedy we adopt as the fam-

ily's 2025 representative. *Implication for document IE.* Projection methods are inherently per-layer, implicit evidence that forgetting is not uniform across depth, but the SVD of 512-token tri-modal activations is costly, and the protected subspace is defined per layer, not per modality; measuring which component needs protection is the cheaper, more interpretable first step.

Representation-based. The dominant 2024–2026 paradigm freezes a large Pre-Trained Model (PTM) backbone and learns small task-specific modules. RanPAC (McDonnell et al., 2023) lifts frozen features by a random projection and fits class prototypes in closed form; SAFE (Zhao et al., 2024b) couples a slow and a fast learner. Prompt-based methods learn a pool of soft prompts and a set of learnable keys $\{k_i\}$, retrieving for an input x the subset of prompts whose keys are most similar to a query $q(x)$ extracted from the frozen backbone,

$$K_x \;=\; \underset{i}{\text{top-K}}\ \mathrm{sim}\big(q(x),\, k_i\big), \tag{2.5}$$

so that knowledge is partitioned across the pool rather than overwritten (Wang et al., 2022d). DualPrompt (Wang et al., 2022c) separates task-shared from task-specific prompts; CODA-Prompt (Smith et al., 2023) replaces this hard $\mathrm{top\text{-}K}$ selection with a differentiable decomposed attention weighting; and S-Prompt (Wang et al., 2022b) learns independent prompts per domain for DIL. Low-Rank Adaptation (LoRA)-based methods instead inject a trainable low-rank update into frozen weight matrices,

$$W \;=\; W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times d},\ r \ll d, \tag{2.6}$$

so each task adds only $2dr$ parameters while W_0 stays fixed (Hu et al., 2022). Orthogonal Low-Rank Adaptation (O-LoRA) (Wang et al., 2023) keeps the per-task subspaces BA orthogonal across tasks, and InfLoRA (Liang and Li, 2024) constrains new adapters to be orthogonal to the input subspaces of old tasks. Its 2025 successor CL-LoRA (He et al., 2025) pairs a *task-shared* adapter that retains cross-task knowledge by distillation with an orthogonality-constrained *task-specific* adapter, the dual-adapter design we adopt as the family's current member. That such adapters can be allocated *per layer* rather than uniformly is exploited by D-MoLE (Ge et al., 2025), which dynamically routes a budget of low-rank experts to the layers where adaptation is most needed: direct evidence that the capacity a continual learner should spend is not distributed uniformly across a network. *Implication for document IE.* Frozen-PTM methods presume that the pre-trained features suffice for all future tasks, plausible for ImageNet-scale ViTs, less obviously so for document encoders whose layout and visual streams are pre-trained on a narrower corpus; and D-MoLE's per-layer budget routing supports measuring first and allocating second.

Architecture-based. These allocate separate parameters per task by masking (PackNet (Mallya and Lazebnik, 2018)) or growth (Dynamically Expandable Networks (Yoon et al., 2018)), trading parameter count for isolation. *Implication for document IE.* Per-task parameter growth is unattractive for deployed KIE parsers that accrue dozens of schema versions, and hard parameter isolation sits awkwardly with the shared, non-stationary O tag of Equation (2.1).

Table 2.1: The five continual-learning method families (Wang et al., 2024), their core mechanism, and representative methods. Methods in bold are used as baselines in this thesis (Chapter 5).

Family	**Core mechanism**
Regularisation	Parameter-importance penalties (EWC/SI/MAS) or function/output distillation (LwF
Replay	Store and replay past exemplars
Optimisation	Gradient projection/constraints
Representation	Frozen PTM + prompts/adapters
Architecture	Separate parameters per task

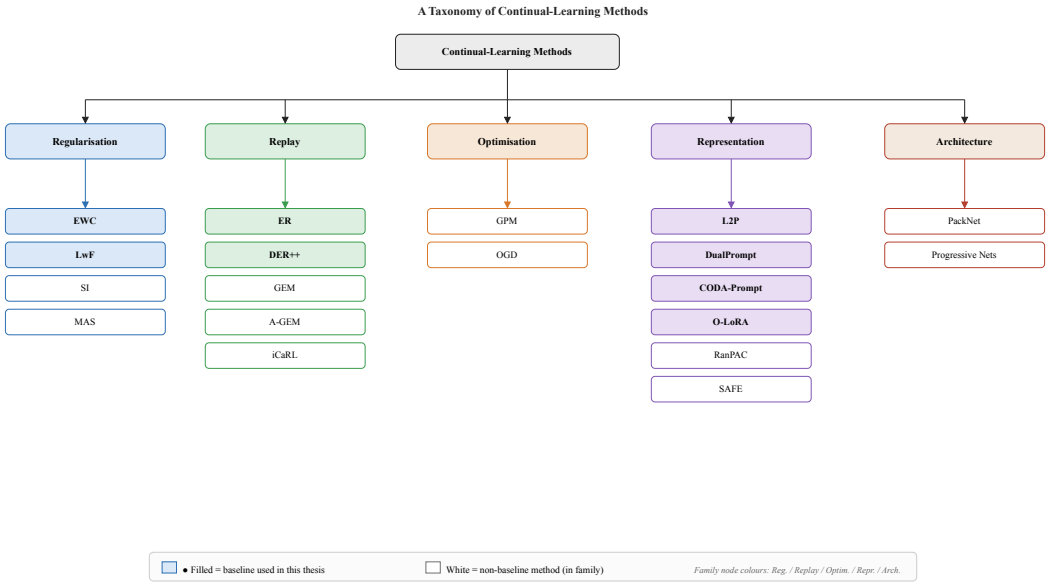


Figure 2.3: Taxonomy of the five continual-learning method families and their representative methods. Methods set in bold are used as baselines in this thesis. The families are not mutually exclusive: several state-of-the-art methods combine mechanisms (for example, DER++ pairs replay with logit distillation, and O-LoRA adds an orthogonality constraint to low-rank adaptation), but the grouping organises the design space by the primary mechanism each method uses to resist forgetting.

2.2.4 Measuring Forgetting Inside the Network

Three measurement tools recur across the families above and underpin the diagnostic of Chapter 3. The first is the FIM, defined together with its diagonal as

$$\begin{aligned} F(\theta) &= \mathbb{E}_{x\sim\mathcal{D}}\,\mathbb{E}_{y\sim p_\theta(\cdot|x)}\left[\nabla_\theta \log p_\theta(y \mid x)\,\nabla_\theta \log p_\theta(y \mid x)^\top\right], \\ F_i &= \mathbb{E}\left[\left(\partial \log p_\theta(y \mid x)\,/\,\partial\theta_i\right)^2\right], \end{aligned} \tag{2.7}$$

where $\mathcal{D}$ is the task data distribution and p_θ the model's predictive distribution. The FIM measures the local curvature of the Kullback–Leibler divergence around θ, how sharply the predictive distribution changes when θ_i moves, and its diagonal estimates per-parameter importance, the basis of EWC (Equation (2.2)) (Amari, 1998, Kirkpatrick et al., 2017). The *empirical* Fisher replaces the samples $y \sim p_\theta$ with the ground-truth labels; this is the variant used by practical EWC implementations and by the per-group estimator of Chapter 3 and Appendix A.2. SVD of layer activations identifies the task-critical subspace and underlies GPM (Equation (2.4)). Finally, CKA (Kornblith et al., 2019) measures the similarity of two *representations* rather than two weight vectors, enabling detection of representational drift that weight-space measures miss (§2.4.3). Together they make it possible to ask not merely *how much* a model forgets but *where*.

2.3 Continual Learning for Information Extraction

CL for IE is real but nascent, fragmented, and almost entirely sentence-level. Table 2.2 maps the landscape.

The dominance of the sentence-level setting is not incidental: it reflects what the surveyed Continual Named-Entity Recognition (CNER) and Continual Relation Extraction (CRE) methods were designed for, and it is precisely why they transfer poorly to documents. Five structural reasons explain the failure when the unit of extraction grows from a sentence to a page:

1. **Cross-sentence entity and coreference resolution.** A field and its value, or repeated mentions of the same entity, routinely span several lines or text blocks; a sentence-windowed model cannot link them.
2. **Multi-hop relations.** Document-level relations (the DocRED setting) chain evidence across sentences, so the reasoning a continual learner must preserve is not captured by any single local context.
3. **Layout and visual context loss.** Sentence-level methods discard bounding boxes and image patches, yet in forms and receipts the role of a token is signalled by position and typography rather than neighbouring words.
4. **O-tag shift amplified across many mentions.** The non-stationary background class of Equation (2.1) (§2.2.2) is far more damaging at document scale, where one entity type recurs across dozens of tokens, so a single session's inconsistent supervision corrupts many spans at once.
5. **Unknown task identity at document scale.** A real document stream does not announce which session or schema a page belongs to, ruling out the TIL assumption on which several sentence-level methods (for example, the prompt-pool CRE approaches) implicitly rely.

These five reasons motivate the document-level, multimodal framing adopted throughout this thesis.

2.3.1 Continual NER and the O-Label Problem

The *semantic shift of the "O" tag* is the defining difficulty of CNER: spans labelled as entities in a future session appear as background in the current session's data, creating conflicting supervision. Methods address it in two ways. *Distillation/pseudo-labelling* approaches keep the schema discriminative: CPFD (Zhang et al., 2023a) combines

pooled-feature distillation with confidence-based pseudo-labelling of old entities, reporting roughly a 6.3-point Micro-F1 gain over prior CNER methods; CFNER (Zheng et al., 2022) distils the causal effect of the Other class; ICE (Liu and Huang, 2023) freezes the feature extractor and adds an individual classifier per session; and SpanKL (Zheng et al., 2023) reformulates the task at the span level with multi-label Knowledge Distillation (KD) to sidestep the token-level O conflict. *Generative* approaches eliminate the O label by construction: GenCNER (Yang et al., 2025) reformulates NER as entity-span generation, so old types are simply not emitted. A separate, practically-motivated line, ProtoNER (Kumar et al., 2024), studies few-shot class-incremental NER with prototypical networks *directly on LayoutLM-family encoders*, underscoring how costly adding entity types to deployed KIE models is. The shared limitation of all of these is that they operate at sentence level and ignore layout and vision.

2.3.2 Continual Relation Extraction

The dominant CRE paradigm combines episodic memory with relation prototypes: a small set of exemplars is stored per relation, and a prototype distilled from those exemplars anchors the relation's representation as new relations arrive. EMAR (Han et al., 2020) instantiates this with a two-step procedure of memory activation and reconsolidation, replaying stored exemplars and then re-anchoring each old relation to its prototype so that its decision region survives the new-task update. RP-CRE (Cui et al., 2021) refines sample embeddings with prototype attention, conditioning each replayed example's representation on the stored relation prototypes so that replayed relations resist drift. CRL (Zhao et al., 2022) adds contrastive replay with distillation, a consistency objective that keeps old relation embeddings stable while the encoder adapts. A second, rehearsal-free line uses prompts: WAVE-CRE (Le et al., 2025b) tackles CRE with task-specific prompt pools in a Mixture of Experts (MoE) style under a TIL setting on TACRED and FewRel, achieving rehearsal-free state-of-the-art. The few-shot regime is addressed by shaping discriminative representations within clusters of similar relations (Le et al., 2025a). All of these methods operate on sentence-level benchmarks (TACRED, FewRel) with gold entity pairs; none confronts cross-sentence evidence (the DocRED setting) or layout signals, the second structural failure mode identified in the introduction to this section (§2.3).

2.3.3 Document-Level Continual Learning

Document-level CL is the least explored and most relevant to this thesis. GDI-Bench, together with the LW-AFT method (Li et al., 2025), provides the first document-intelligence CL benchmark (2.3k images, nineteen tasks, with visual and reasoning difficulty decou-

pled) and shows that supervised fine-tuning of a large Multimodal Large Language Model (MLLM) (InternVL3-8B) causes measurable forgetting, worst on the harder reasoning tasks; LW-AFT mitigates this by *heuristically* freezing layers. Separately, continual table detection (Minouei et al., 2022) demonstrates CL for a document detection task with simple ER, recovering roughly 15% of the lost performance. The crucial gap is that these works either use a heuristic remedy or address detection rather than IE, and none diagnoses *which component* of the backbone forgets: the question this thesis answers.

Table 2.2: The continual-learning-for-IE landscape and the gap each line leaves open. Sentence-level, text-only, and heuristic approaches dominate; no prior work provides a per-component forgetting diagnosis for a tri-modal document encoder.

Setting	**Method**	**Gap remaining**
Continual NER	CPFD / CFNER / ICE / SpanKL	Class-IL only; text-only; sentence-level
Continual NER	GenCNER	Small backbone; no document/layout context
Continual NER	ProtoNER	Few-shot only; no full-grid CL evaluation
Continual RE	EMAR / RP-CRE / CRL	Replay + prototypes; sentence-level; closed-world
Continual RE	WAVE-CRE	Task-IL; closed-world; sentence-level
Continual RE	Similar-cluster FS-CRE	Few-shot; no document-level relations
Document CL	GDI-Bench + LW-AFT	Heuristic freezing; no IE tasks; no diagnosis
Document CL	CL Table Detection	Detection only; simple replay; no IE
Multimodal CL	CL-CrossVQA / CLiMB / CLOVE	VQA/VL only; no document IE
Multimodal CL	SEFE	Instruction tuning; not IE; no per-component view

2.4 Multimodal and Component-Level Forgetting

2.4.1 Where Forgetting Lives: Text vs. Visual vs. Fusion

Evidence from multimodal CL benchmarks suggests that forgetting is not uniform, but the picture is incomplete. CL-CrossVQA (Zhang et al., 2025) finds across five Visual Question Answering (VQA) datasets that dual-stream models forget less than single-stream

ones and that deeper layers forget more, hinting that architectural structure shapes forgetting. CLiMB (Srinivasan et al., 2022) observes that CL can prevent forgetting on vision-and-language tasks yet fails to enable positive transfer. CLOVE (Lei et al., 2023) and the Knowledge-Based Question Answering (KBQA) method DAHR (Zhao et al., 2024a) converge on a sharper point: continual *reasoning* is not continual *classification*: preserving a reasoning mechanism requires different machinery than preserving a feature representation, which matters for the structured outputs of EE and multi-hop RE. These findings concern VQA/KBQA rather than document IE and, crucially, do not localise forgetting to specific *components* of a single encoder. The methodological inspiration for this thesis comes instead from analyses of forgetting in unimodal networks: the "anatomy of catastrophic forgetting" (Ramasesh et al., 2021) shows that forgetting concentrates in higher layers and can be probed layer-by-layer, and the study of forgetting in MLLMs (Zhai et al., 2024) adapts such diagnostics to the multimodal setting.

Their methodologies are worth stating precisely, since this thesis builds on both. Ramasesh et al. (Ramasesh et al., 2021) study sequential pairs of split image-classification tasks and localise forgetting by resetting or freezing individual blocks of layers to their pre-second-task values and measuring how much first-task performance returns, complemented by layer-wise probes; they conclude that forgetting is driven predominantly by the deeper layers nearest the output, while early representations remain largely reusable. Zhai et al. (Zhai et al., 2024) evaluate multimodal large language models as image classifiers, fine-tuning on one dataset while tracking accuracy on held-out datasets, and show that the unimodal forgetting picture carries over to multimodal models and worsens with longer fine-tuning. This thesis borrows the checkpoint-pair, per-layer probing stance of both works but extends the unit of analysis from depth to modality-specific components (text, layout, visual, and cross-modal fusion) within a single tri-modal encoder, a question neither work addresses.

2.4.2 Superficial versus Essential Forgetting

SEFE (Chen et al., 2025) formalises a distinction between *superficial* forgetting (loss of the expected answer format) and *essential* forgetting (loss of the underlying knowledge), and addresses them with answer-style diversification and a regularised LoRA. The distinction matters for KIE, where a model may retain the ability to localise entities yet lose the correct label mapping (or vice versa). It motivates measuring forgetting with multiple, complementary instruments rather than a single accuracy number, a principle adopted in Chapter 3.

2.4.3 Measuring Representational Drift: CKA

To attribute forgetting to components, this thesis measures how internal representations change between checkpoints. CKA (Kornblith et al., 2019) compares two activation matrices X and Y (samples $\times$ neurons) and returns a similarity $\mathrm{CKA}(X, Y) \in [0, 1]$, the precise estimator of which is given in Chapter 3. Two invariance properties make it the appropriate instrument here. First, linear CKA is invariant to *orthogonal transformations* of the representation: for any orthogonal Q, $\mathrm{CKA}(XQ, Y) = \mathrm{CKA}(X, Y)$, so a mere rotation or permutation of neurons, which leaves the function computed by a layer unchanged, is correctly scored as no drift. Second, it is invariant to *isotropic scaling*, $\mathrm{CKA}(\alpha X, Y) = \mathrm{CKA}(X, Y)$ for $\alpha \neq 0$, so a uniform change in activation magnitude is likewise ignored. These are exactly the symmetries a representation-level measure should respect, and they are why CKA is preferable to a naive cosine similarity between weight vectors: cosine similarity in weight space is *not* invariant to neuron permutation or rotation, so two functionally identical networks can register as maximally dissimilar simply because their hidden units are reordered. A CKA of one indicates unchanged representations; a low value indicates genuine reorganisation. Per-layer CKA between the model before and after a task thus localises the "forgetting hotspots," and is one of the two primary diagnostic metrics used here, alongside per-group FIM.

2.5 Backbones for Continual Document IE

The choice of backbone determines both the cost and the interpretability of the diagnosis.

Selection criteria. Five criteria govern the choice. **(i) Component separability:** distinguishable text, layout, visual, and fusion pathways, the prerequisite for a per-component diagnosis. **(ii) Document-specific pre-training:** layout and image objectives rather than text-only pre-training. **(iii) Scale within compute budget:** the full grid of methods $\times$ scenarios $\times$ seeds must run on a single GPU. **(iv) Ecosystem maturity:** a maintained reference implementation and processor, so the diagnosis runs on the canonical model rather than a re-implementation. **(v) Single-task strength:** strong FUNSD/CORD F1 so that forgetting is measured from a competent base (cf. §2.1.2).

Justification. LayoutLMv3-base ($\approx$ 133M parameters, encoder-only) satisfies all five. It exposes clearly-separable text, visual, layout, and fusion components (i); it is pre-trained with document-specific objectives (MLM, MIM, and WPA; §2.1.2) rather than text alone (ii); it is small enough to run the full grid of methods $\times$ scenarios $\times$ seeds on a single GPU, where multi-billion-parameter MLLMs would not be (iii); it has a maintained refer-

ence implementation and processor (iv); and it reports strong single-task F1 on FUNSD and CORD (v). The instantiation-level comparison of candidate backbones is given in Table 4.1 (Chapter 4).

Secondary backbones and excluded alternatives. To test that the diagnosis generalises beyond a single architecture, LiLT-base and BROS-base are used as secondary backbones on a reduced method subset. Large MLLMs such as InternVL3-8B are used in document CL benchmarks (GDI-Bench) but their scale and decoder-based design make a controlled per-component study expensive and harder to interpret; they are noted as a comparison target rather than a primary subject.

2.6 Research Gaps and Opportunities

The survey above exposes four gaps the thesis addresses; each is developed in turn below.

2.6.1 Gap 1: No Per-Component Forgetting Diagnosis for Tri-Modal Encoders

Prior diagnostic work measures forgetting at the level of whole tasks or, at most, layers. The anatomy of catastrophic forgetting (Ramasesh et al., 2021) localises forgetting by depth, but in unimodal image-classification networks. The MLLM study of Zhai et al. (2024) measures forgetting at the whole-model level only. CL-CrossVQA (Zhang et al., 2025) compares forgetting *across* architectures (dual-stream versus single-stream) rather than *within* one encoder. D-MoLE (Ge et al., 2025) implicitly assumes non-uniform capacity needs across layers but never measures forgetting per component. *What is missing:* an attribution of forgetting to the text, layout, visual, and fusion components of a single tri-modal document encoder.

2.6.2 Gap 2: Sentence-Level Bias of Continual Information Extraction

Continual NER methods (distillation and pseudo-labelling approaches (Liu and Huang, 2023, Zhang et al., 2023a, Zheng et al., 2022, 2023) and generative reformulations (Yang et al., 2025)) operate on sentence-level, text-only corpora, as do the continual RE lines (Le et al., 2025a,b). ProtoNER (Kumar et al., 2024) reaches LayoutLM-family encoders but only in a few-shot regime, without a full continual evaluation. None of these confronts the structural failure modes that arise when the unit of extraction grows from a sentence to a page (§2.3). *What is missing:* continual IE methods designed for, and evaluated on, document-level multimodal inputs.

2.6.3 Gap 3: Heuristic Rather than Measurement-Driven Remedies

Where remedies do exist, their targeting is heuristic. LW-AFT freezes layers by a heuristic schedule rather than by a measurement of where forgetting occurs (Li et al., 2025). GPM protects per-layer subspaces but never attributes them to semantic components of the network (Saha et al., 2021). The open question is whether a remedy *targeted by measurement* beats uniform or heuristic treatment. *What is missing:* a remedy whose targeting is derived from a per-component forgetting diagnosis rather than from a predetermined schedule.

2.6.4 Gap 4: Fragmented Evaluation for Continual Document IE

Existing results are scattered across incompatible protocols. Continual NER and RE use disjoint, text-only benchmarks; document CL uses GDI-Bench (Li et al., 2025) or detection tasks (Minouei et al., 2022); and the image-CL prompt methods (Smith et al., 2023, Wang et al., 2022c,d) have never been compared against IE-specific methods on document KIE under one protocol. *What is missing:* a shared continual benchmark for document IE that compares image-CL baselines and IE-specific methods on equal footing.

2.7 Positioning of the Current Research

This thesis is a *diagnostic-plus-remedy* study in the spirit of Ramasesh et al. (2021) and Zhai et al. (2024): it first characterises how forgetting distributes across the components of a tri-modal document encoder, then proposes one method, lexically-routed latent replay, whose memory attaches at the dominant-forgetting component. The measurement contribution stands on its own; it is informative even if forgetting turns out to be uniform, while the remedy is explicitly derived from, and validated against, the measurement. The four gaps of §2.6 map onto the contributions of this thesis as follows.

Gap 1 → Contribution 1 (the per-component forgetting diagnostic). The missing per-component diagnosis (§2.6.1) is supplied by controlled modality ablation of a single backbone, read out by per-layer CKA, per-group FIM, and standard forgetting metrics (Chapter 3).

Gap 2 → the benchmark's document-level framing. The sentence-level bias (§2.6.2) is countered by casting every task with tri-modal inputs and BIO KIE at page scale, with the O-tag shift retained rather than engineered away (Chapter 5).

Gap 3 → Contributions 2–3 (why remedies fail, and lexically-routed latent replay). The reliance on heuristic remedies (§2.6.3) is answered by a remedy whose every part is derived from measurement: the diagnosis fixes *where* it attaches (the head/late site), the tests of the failing remedies fix *what* its memory must keep (whole documents, stored as compressed per-document latents, CoLaR), and LexSlot's drift-immune lexical signature decides *which* stored documents are replayed (Chapter 3).

Gap 4 → Contribution 4 (the FCS-CL benchmark). The fragmented evaluation (§2.6.4) is resolved by unified scenarios, ten baselines spanning the method families plus image-CL prompt methods, a shared interface, three seeds, and statistical testing (Chapters 4 and 5).

The work is deliberately scoped to encoder-only document KIE on three public datasets, with larger backbones, additional scenarios, and multilingual extensions left as future directions (Chapter 7).

2.8 Chapter Summary

Document IE is a multimodal, spatially-structured, schema-diverse problem solved by trimodal encoders such as LayoutLMv3. CL of such tasks is obstructed by CF, for which five method families offer remedies, but almost all treat the network uniformly and were validated on unimodal benchmarks. CL for IE exists only in fragmented, sentence-level, mostly text-only form, and the one document-level remedy is heuristic. Crucially, no prior work measures *which component* of a multimodal document encoder forgets. Four gaps follow: component attribution, sentence-level bias, heuristic remedies, and fragmented evaluation (§2.6). The next chapter develops a diagnostic to answer that question and a mechanism-targeted remedy derived from the answer.

Chapter 3

Methodology

This chapter presents the methods that answer the two research questions of §1.2. For RQ1 it builds a *per-component forgetting diagnostic* for tri-modal document encoders (§3.2). For RQ2 it derives, from that diagnosis, where a remedy may attach (§3.3), designs LexSlot, a lexically-gated slot memory at the diagnosed site (§3.4), and joins LexSlot's gate to a compressed per-document latent memory in *lexically-routed latent replay* (§3.5). The ordering is deliberate: the diagnosis comes first and constrains every remedy that follows.

3.1 Problem Formulation

We study continual KIE as token-level sequence labelling. It is useful to separate three layers of "input/output" that are easily conflated: at the *user* level the input is a document scan and the output is extracted information; at the *machine-learning* level the input is a stream of datasets $(\mathcal{D}_1, \mathcal{D}_2, \dots)$ and the output is the trained parameters; at the *technical* level, the one we formalise, the input is a document tuple and the output is a label sequence. (All symbols introduced here are collected in the List of Notations.)

A document is a triple $D = (W, B, I)$ where $W = (w_1, \dots, w_N)$ is a sequence of N OCR tokens, $B = (b_1, \dots, b_N)$ with $b_i \in [0, 1000]^4$ are normalised bounding boxes, and $I \in \mathbb{R}^{3\times224\times224}$ is the page image. The model f_θ produces BIO labels $Y = (y_1, \dots, y_N)$ with $y_i \in \mathcal{L}_{\text{task}}$, trained by token-level cross-entropy ℓ with ignored positions masked out. Performance is measured by entity-level F1 with BIO-aware span matching.

In the continual setting the model sees a sequence of tasks $\mathcal{T}_1, \dots, \mathcal{T}_T$, each with its own data and (in CIL) its own growing label space. Let θ denote the encoder parameters and $\theta^{(t)}$ the parameters after training task t; the learner may access only $\mathcal{D}_t$ and (if any) a

memory M at task t. The continual objective minimises the summed risk over all tasks,

$$\theta_T = \arg\min_{\theta} \sum_{t=1}^{T} \mathbb{E}_{(x,y)\sim\mathcal{D}_t}\big[\ell(f_\theta(x), y)\big] \quad \text{s.t.} \quad |M| \le M_{\max}, \;\; \|\Delta\theta\| \le P,$$

under a bounded memory M (at most $M_{\max}$ exemplars) and a parameter budget P (both relevant because the document setting favours memory-light, privacy-preserving methods). The object of study is how performance on earlier tasks degrades as later tasks are learned, and, uniquely here, how that degradation is distributed across the *components* of θ.

The stream follows one of three regimes. In CIL the label space grows, $\mathcal{L}_1 \subset \mathcal{L}_2 \subset \cdots$, and the head is expanded at each session; in DIL the label space is fixed and only the document distribution shifts, so forgetting is pure representation drift; the mixed regime combines both. The memory M never holds the full past data: in this thesis it holds either a small set of exemplars or, for latent replay, the layer-k activations of past documents, stored in compressed form, so no raw page is retained. The two research questions then read, in this notation: **RQ1** asks how the loss on $\mathcal{T}_{<t}$ after training on $\mathcal{T}_t$ is distributed over the components of θ defined below; **RQ2** asks what M must contain at the diagnosed component for the loss on $\mathcal{T}_{<t}$ to stay near its joint-training value, and how the entries of M should be chosen for replay given the current input x.

3.1.1 Defining a Component

We partition the encoder parameters into groups that are *genuinely separable* in the backbone. A caveat governs this partition and is central to the honesty of the analysis: LayoutLMv3 is a *single-stream* encoder. Only the *input* embeddings are modality-specific; from the first encoder layer onward, text and visual patch tokens are concatenated and share the *same* query/key/value projections. There is therefore no identifiable "text-attention" versus "visual-attention" versus "cross-modal fusion" *parameter* group: those projections are one shared set. We do not pretend otherwise. We group instead by what the architecture actually separates:

- **Text word embeddings**: the token embedding table (`text_word_embed`).
- **Layout embeddings**: the 2D position embeddings encoding bounding-box geometry (`layout_2d_pos_embed`).
- **Visual patch embedding**: the image-patch projection (`image_patch_embed`).
- **Shared self-attention**: the (single-stream) query/key/value projections (`attn_qkv`)

and the attention output projection (`attn_out`), reported separately rather than folded into the feed-forward block.

- **Feed-forward and normalisation**: the position-wise feed-forward blocks (`ffn`) and layer norms (`layernorm`).
- **Task head**: the token-classification layer (`classifier`), tracked separately as it is expanded for new labels.

These groups are exhaustive and every one is populated; a residual catch-all is verified empty for the stock backbone. The modalities' *behavioural* contributions are recovered separately, by the input-masking conditions of §3.2.2, and forgetting is additionally localised by *depth* (input / early / mid / late encoder layers / head), the axis along which a single-stream encoder is most naturally heterogeneous. For CKA tracking we probe six depth-matched points (embeddings, patch embedding, early/mid/late encoder layers, and the classifier) so the gradient can be compared across backbones, including the unimodal BERT baseline.

3.2 A Per-Component Forgetting Diagnostic

3.2.1 Motivation

If forgetting concentrates in a particular component, a continual-learning method should spend its stability budget there; if it is uniform, uniform methods are justified. Either way the answer is informative. To obtain it we need (i) a way to isolate each modality's contribution without confounds, and (ii) metrics that attribute forgetting to components.

3.2.2 Controlled Modality Ablation

We use two complementary controls. First, to isolate the modalities *behaviourally* without confounding architecture, we hold the architecture fixed and mask the input streams of a *single* LayoutLMv3, defining four apples-to-apples conditions C1–C4 (Table 3.1): text only (C1), image+layout (C2), text+layout (C3), and the full tri-modal input (C4). Because only the inputs differ, any difference in forgetting is attributable to the modalities in play rather than to the network. Critically, the text-only condition (C1) keeps the real text tokens and zeroes *only* the image and layout streams: it is a genuine text-only LayoutLMv3, not a null input.

Second, because a same-architecture mask cannot tell us whether a multimodal encoder forgets *differently* from a unimodal one, we add an *external* unimodal baseline

Cb: a true BERT-base text encoder, tokenised with its own WordPiece vocabulary on the same documents and run through the same instrument (per-token CKA, per-group importance, displacement, forgetting metrics). The within-architecture conditions answer "which stream/depth forgets"; the Cb contrast answers "is this multimodal-specific?".

Table 3.1: The diagnostic conditions. C1–C4 are the *same* LayoutLMv3 architecture under different input-stream masks (only the active inputs differ); Cb is an external true-unimodal baseline. C1 keeps real text (image and layout zeroed); it is not a null-input condition.

ID	Model / active inputs	Purpose
Cb	BERT-base, text only	external true-unimodal contrast
C1	LayoutLMv3, text only	real text-only (image + layout zeroed)
C2	LayoutLMv3, image + layout	isolate visual+layout-only forgetting
C3	LayoutLMv3, text + layout	isolate text+layout-only forgetting
C4	LayoutLMv3, full tri-modal	main subject of analysis

Figure 3.1 depicts the LayoutLMv3 conditions as the same network with different input pathways switched on or off.

3.2.3 Forgetting Decomposition Metrics

We measure forgetting with three complementary instruments, following the multiple-instrument principle motivated in §2.4.2.

Representational drift (per-token linear CKA). For each tracked layer we compute the linear CKA (Kornblith et al., 2019) between activations of the model after task $t-1$ and after task t on a fixed probe set. Given two centred activation matrices $X \in \mathbb{R}^{n\times p}$ and $Y \in \mathbb{R}^{n\times q}$,

$$\mathrm{CKA}(X,Y) = \frac{\|Y^\top X\|_F^2}{\|X^\top X\|_F \, \|Y^\top Y\|_F}.$$

A drop of CKA below 1 at a layer marks it as a drift hotspot. Two design choices matter for validity. First, token classification lives in the *per-token* representation, so we compute CKA over *individual valid tokens* (the classifier-relevant first-subword positions), not over a document-mean vector that would average $\sim$500 token vectors per page and could read "no drift" while the token geometry has reorganised. Second, the probe collects up to $n = 2000$ distinct tokens (well above the hidden width $p = 768$), so the estimate is not in the high-variance $n < p$ regime; activations are captured *under the condition's modality mask* so each condition is probed on the inputs it was trained on.

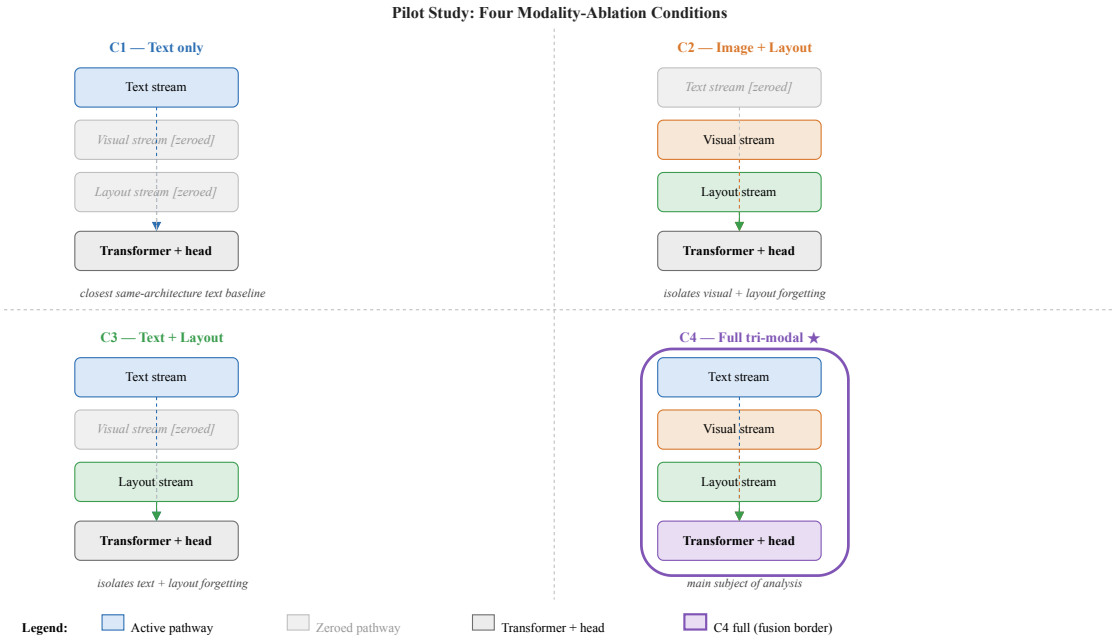


Figure 3.1: The LayoutLMv3 conditions C1–C4 realised on a single fixed architecture. Each panel shows the same LayoutLMv3 with a different subset of input pathways active (text, image, layout) and the remainder zeroed at the input: real text only (C1), image and layout (C2), text and layout (C3), and the full tri-modal input (C4). The external unimodal baseline Cb (BERT-base) is a separate model, not a mask. Because only the inputs differ across C1–C4, any difference in forgetting is attributable to the modalities in play rather than to the network.

Where forgetting lives: importance vs. displacement. We separate two distinct quantities that are easily conflated. The empirical Fisher *importance* of a group g on a task's own data,

$$F_g = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \sum_{\phi\in g} \left(\frac{\partial \log p_\theta(y \mid x)}{\partial \phi} \right)^2,$$

measures how *sensitive* the loss is to g *for that task*. It is *not* a forgetting signal: the classifier head carries the largest such gradients for *any* token classifier (BERT included) by the geometry of the cross-entropy output layer, regardless of whether those parameters were *moved*. To localise *forgetting* we use the old-task-Fisher-weighted *displacement* between the checkpoint before and after a task,

$$D_g = \sum_{\phi\in g} F_\phi^{(t-1)} \left(\theta_\phi^{(t)} - \theta_\phi^{(t-1)} \right)^2,$$

which is large precisely where parameters that mattered for the *previous* task ($F^{(t-1)}$) have been overwritten. We report D_g per component group and per depth bucket (input/early/mid/late/head) as the primary forgetting localiser, with Fisher *importance* re-

ported alongside but never labelled forgetting. Fisher is computed under each condition's modality mask.

Forgetting matrix. We record the standard accuracy matrix $R[i, j]$, the entity-F1 on task j after training task i, from which we derive average accuracy, backward transfer, and average forgetting (§5.5.2). Its structure is annotated in Figure 3.2: the diagonal holds just-learned accuracy, the lower triangle holds retained accuracy on earlier tasks (the source of backward transfer and forgetting), and the upper triangle, unused here as tasks arrive sequentially, would hold zero-shot forward transfer. This is the conventional, component-agnostic view that the per-component metrics refine.

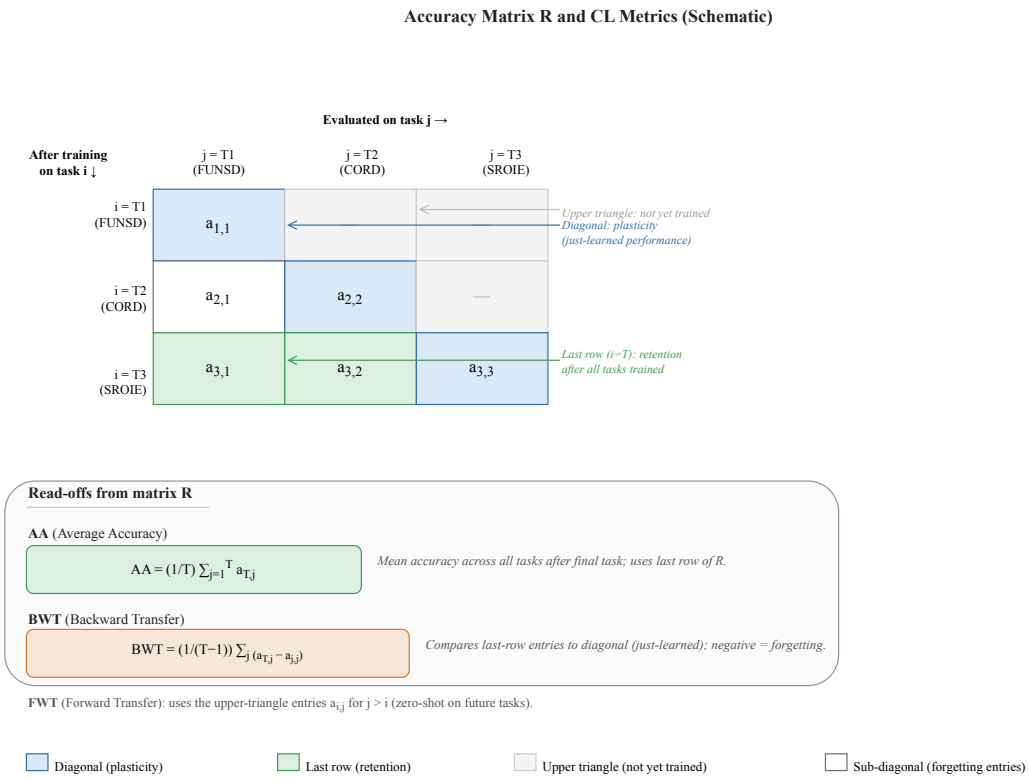


Figure 3.2: Schematic structure of the accuracy matrix $R[i, j]$ (entity-F1 on task j after training task i); symbolic labels only, with no measured values. The diagonal $R[i, i]$ records just-learned accuracy; the lower triangle $R[i, j]$ for $j < i$ records retained accuracy on earlier tasks, from which average accuracy, backward transfer, and average forgetting are derived; the upper triangle is unevaluated under sequential arrival.

3.2.4 Diagnostic Protocol

For each condition C1–C4 and each seed we train naively and sequentially on FUNSD $\rightarrow$ CORD $\rightarrow$ SROIE with no continual-learning strategy, expanding the

Algorithm 1 Per-component forgetting decomposition

Require: condition $c \in \{C1, C2, C3, C4\}$, task sequence $\mathcal{T}_{1:T}$, probe set $\mathcal{P}$, layer set $\mathcal{G}_{\text{CKA}}$, parameter groups $\mathcal{G}_{\text{F}}$

initialise $\theta^{(0)}$ from the pre-trained backbone
for $t = 1$ **to** T **do**
    expand classifier head to $\mathcal{L}_{\mathcal{T}_t}$
    $A_{t-1} \leftarrow \text{Activations}(\theta^{(t-1)}, \mathcal{P})$
    $\theta^{(t)} \leftarrow \text{FineTune}(\theta^{(t-1)}, \mathcal{T}_t)$ ▷ naive, modality mask c
    $A_t \leftarrow \text{Activations}(\theta^{(t)}, \mathcal{P})$
    for all $\ell \in \mathcal{G}_{\text{CKA}}$ **do**
        $\text{CKA}_{t,\ell} \leftarrow \text{LinearCKA}(A_{t-1}[\ell], A_t[\ell])$
    end for
    for all $g \in \mathcal{G}_{\text{F}}$ **do**
        $F_{t,g} \leftarrow \text{Fisher}(\theta^{(t)}, g, \mathcal{T}_t)$
    end for
    for all $j \leq t$ **do**
        $R[t, j] \leftarrow \text{EvalF1}(\theta^{(t)}, \mathcal{T}_j)$
    end for
end for
return $\{\text{CKA}_{t,\ell}\}, \{F_{t,g}\}, R$

classifier head at each task. At every task boundary we capture per-layer CKA, per-group Fisher, and the forgetting matrix. Algorithm 1 summarises the procedure and Figure 3.3 shows it as a data-flow diagram.

3.2.5 Hypotheses and Decision Rule

We test a null hypothesis H_0 that forgetting is uniform across the network against H_1 that it concentrates along depth (in particular at the classifier head and late encoder layers). The test uses the per-group displacement D_g and per-depth displacement of §3.2.3, aggregated *per seed first* (the per-seed statistic is the mean over that seed's boundaries) so the unit of analysis is the seed. For the dominant (largest mean) bucket we compare its seed-level displacements against the pooled remaining buckets with a one-sided Mann–Whitney U test, rejecting H_0 for a bucket when $p_g < 0.05/(K-1)$ for K buckets.

Statistical power (a hard constraint on the design). A rank test on small samples has a *minimum achievable* two-sided p-value of $2/\binom{n_1+n_2}{n_2}$ even under perfect separation. With three seeds per group this floor is $2/\binom{6}{3} = 0.10$, far above any Bonferroni threshold: such a test *cannot* reject H_0 regardless of the data, so a "failure to reject" would be an artefact of sample size, not evidence. We therefore run *at least five seeds* per condition (and pool the

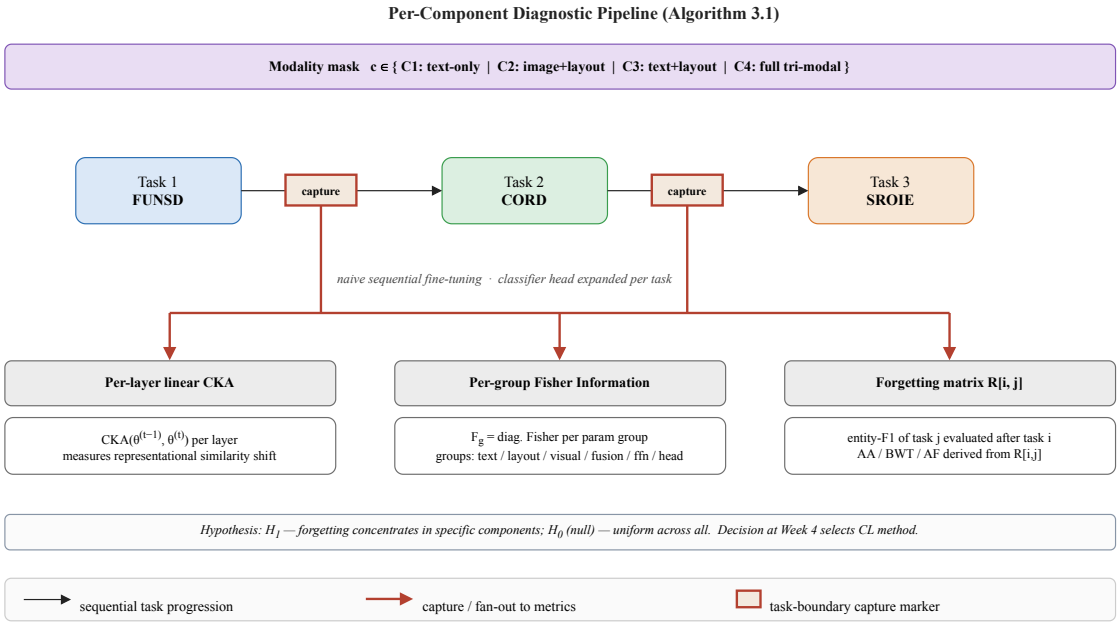


Figure 3.3: Data flow of the per-component diagnostic. The model is fine-tuned sequentially on FUNSD → CORD → SROIE under a fixed modality mask, and at each task boundary three instruments are captured: per-layer linear CKA between the pre- and post-task checkpoints on a fixed probe set, per-group empirical Fisher importance, and a row of the accuracy matrix $R[i, j]$ over all seen tasks. The procedure is repeated for every condition C1–C4 and every seed.

alternate task order), which lowers the floor for the cross-condition contrast to $2/\binom{10}{5} = 0.0079$, below the corrected threshold. Every reported non-rejection is accompanied by its power floor, and any comparison whose floor exceeds α is labelled *inconclusive by construction* rather than evidence for H_0.

Location, not just magnitude. Asking whether a multimodal encoder forgets *differently* is a question about the *distribution* of forgetting across components, which a scalar test on total backward-transfer magnitude cannot answer. We therefore add a distribution-targeting test: for each condition we form the per-depth displacement *profile* (an L_1-normalised vector over buckets) and compare the profile of C4 against each unimodal reference (C1 and the external Cb) with a permutation test on the cosine distance between mean profiles. This directly tests "do they forget in a different *place*?".

The diagnostic outcome maps to the remedy through the measurable branches of Table 3.2 and Figure 3.4. We deliberately restrict the decision to signals the instrument can actually produce: because LayoutLMv3 is single-stream, "fusion-dominant" or "visual-attention-dominant" branches are *not realisable* and are excluded; the admissible outcomes are a head/late-layer concentration, a uniform-across-depth pattern, or no concen-

tration.

Table 3.2: Measurable diagnostic outcomes and the remedy each selects. Only outcomes the single-stream instrument can realise are admitted; the decision is a function of the pilot (§3.3.4).

Pattern	**Signature**	**Triggers**
H_a head/late-dominant	Displacement concentrates at the head and late layers	LexSlot: isolated slots at head/late (§3.4)
H_b uniform-across-depth	Displacement broadly even across depth buckets	Uniform consolidation
H_0 no concentration	No bucket separates after correction (with adequate power)	Characterisation-only (§3.3.5)

The mapping from diagnostic outcome to remedy is made explicit in Figure 3.4: the decision is a function of the pilot, so the mechanism is not committed to in advance, and the no-concentration branch leads to the characterisation-only fallback rather than to a method.

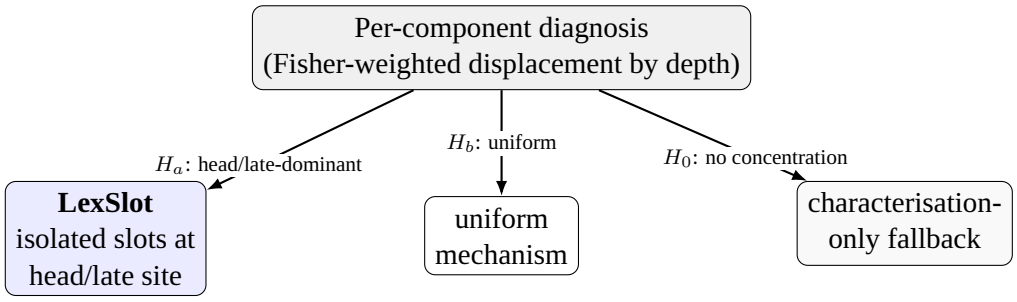


Figure 3.4: The decision rule as a tree over *measurable* outcomes. A head/late-layer concentration (H_a, the outcome the diagnosis returns) selects a head/late-targeted remedy, here **LexSlot**, an isolated slot memory at that site; a uniform-across-depth pattern (H_b) would instead select a uniform mechanism; no significant concentration (at adequate power, H_0) selects the characterisation-only fallback. Unrealisable branches (fusion- or visual-attention-dominant) are excluded because a single-stream encoder cannot produce those signals.

3.3 Mechanism-Targeted Continual Learning

The diagnosis localises forgetting to a specific region of the network, the classifier head and late encoder layers (Chapter 6), without prescribing how to protect it. This thesis derives one remedy from that measurement: **LexSlot** (§3.4), a parameter-efficient, isolated *slot memory* placed at the diagnosed head/late site, paired in its headline configuration with a compact 200-exemplar replay buffer. The motivation, design-space analysis, and selection criterion below (§§3.3.1–3.3.4) fix *where* a mechanism should attach; LexSlot is the mechanism we put there.

3.3.1 Why Component-Agnostic Continual Learning Is Insufficient

Generic continual-learning methods treat the encoder uniformly. EWC penalises all important parameters with a single Fisher-weighted term regardless of which component they belong to; LW-AFT freezes layers by a fixed heuristic. If forgetting concentrates in one component, both spend stability budget where it is not needed and may under-protect where it is. The hypothesis this thesis tests is that *concentrating* the continual-learning mechanism (here, a learnable slot memory placed only where forgetting lives) on the dominant-forgetting component improves the stability–plasticity trade-off at negligible extra cost.

3.3.2 Design Space: Where a Mechanism Can Attach in a Single-Stream Encoder

The three continual-learning families of §2.2.3, regularisation, replay, and distillation, each leave one design choice open for a document encoder: *where* in the network the mechanism concentrates its effort. This is precisely the choice that existing methods make blindly. EWC weights every parameter by its Fisher importance but applies the *same* quadratic form everywhere; replay re-exposes whole inputs without regard to which layers most need grounding; distillation matches the full output without asking which representations have drifted. The contribution of this thesis is not a new family but a principled, *measured* answer to the "where" question, and the honest constraint on that answer is the single-stream caveat of §3.1.1.

That caveat rules out an entire class of otherwise-appealing designs. One might wish to place per-modality low-rank adapters on a "text", "visual", and "fusion" bank, or to route per-modality prompt sub-pools by a layout-aware query, both natural ideas, and both presupposing separable text-, visual-, and fusion-attention parameter groups. LayoutLMv3

has none: from the first encoder layer the streams share one set of query/key/value projections (§3.1.1). We therefore do not pursue modality-partitioned adapters or routers here; they are reserved for a genuinely two-stream backbone where the streams physically exist (Chapter 7). What the architecture *does* separate is *depth*: input embeddings, early/mid/late encoder blocks, and the task head, and depth is the axis along which the diagnosis (§3.2) can actually localise forgetting. The remedy is therefore built on the one axis the instrument can measure.

Within that constraint, a mechanism can attach to the diagnosed depth in different ways: by spending a Fisher penalty there (a depth-scaled version of regularisation), by re-exposing old classes at the drifting head (a targeted version of replay), or by inserting dedicated learnable capacity there and freezing the rest (a structural, parameter-efficient option). This thesis takes the last route: **LexSlot** places small, isolated slot adapters at the head and late layers over a frozen backbone (§3.4). The components it uses, low-rank adapters and a frozen backbone, are established; what is novel is that *where* they attach is fixed by measurement, not by a hand-tuned heuristic or a uniform default, and that each task's slots are write-once isolated so old knowledge is structurally protected.

3.3.3 Testable Component Hypotheses

Two method-level hypotheses are pre-registered and map one-to-one to the component-targeting ablation in Chapter 6:

- $\mathbf{H_{target}}$**.** Placing the slot memory at the diagnosed site (the classifier head and late encoder layers, `slot_depth=head_late`) outperforms spreading slots uniformly across every depth (the *uniform* ablation) on the domain-incremental scenario.
- $\mathbf{H_{head}}$**.** Head-only slot placement recovers most of the gain of the full head/late configuration; adding late-layer slots provides at most a small further improvement (the *head-only* vs. *head/late* ablation).

A rejected hypothesis remains informative and is reported as such.

3.3.4 Selection Criterion

The remedy follows directly from the measurable diagnostic outcome (Table 3.2): a head/late-layer concentration selects a mechanism that concentrates its effort there: LexSlot with head/late slot placement (§3.4); a uniform-across-depth pattern would instead select a uniform mechanism; no concentration (at adequate power) selects the characterisation-only fallback (§3.3.5). Beyond alignment with the diagnosis, the method

must be implementable within the compute budget, defensible under reviewer scrutiny, and parameter-efficient enough to add no fragile new infrastructure.

3.3.5 Pre-Registered Fallback (Not Triggered)

For completeness we record the contingency registered before the diagnosis was run: had forgetting *not* concentrated (H_0 not rejected at adequate power), the diagnostic of §3.2 and the FCS-CL benchmark would have become the headline contributions and the remedy would have been withheld. The diagnosis did concentrate forgetting at the head and late layers (Chapter 6), so this branch was *not* taken: a head/late-targeted remedy is justified, and §§3.4–3.5 build the one this thesis proposes. We state the fallback explicitly because pre-registering it, rather than choosing the form after seeing which design won, is what licenses the claim that the remedy's *target* was *derived from* the measurement; and because the diagnostic's value stands independently of the remedy's margin regardless.

3.4 Lexical-Slot Head-Memory Continual Learning (LexSlot)

Role in the proposed method. The thesis proposes one method, lexically-routed latent replay (§3.5), with three parts: *where* it acts (the diagnosed head/late site), *what* it stores (compressed per-document latents, CoLaR), and *which* stored documents it replays (a lexical gate). This section develops the gate. It does so inside LexSlot, a slot memory at the diagnosed site that was designed and evaluated first; LexSlot's results (Chapter 6) are the evidence that the lexical signature is a usable, drift-immune routing signal at the head. Its slot memory is then replaced by CoLaR's latent memory, and its signature is kept as the gate.

The diagnosis fixes *where* a remedy should act, the classifier head and late encoder layers (§§3.3.1–3.3.4), and LexSlot is the mechanism this thesis puts there. The guiding question is whether the diagnosed site can be protected by a *parameter-efficient structured memory that stores no past inputs at all*, rather than by a consolidation budget that keeps a frozen teacher in memory and rehearses stored inputs. LexSlot answers it with an *isolated, write-once slot memory*: small additive slot adapters placed at the head and late layers over a partially-frozen backbone (lower encoder frozen, head and late layers trainable), where each task writes its own slots and a drift-immune lexical signature gates both updates and activations so earlier tasks are protected structurally: the slot mechanism itself needs no rehearsal buffer and no consolidation penalty to tune (Figure 3.5); its headline configuration adds a compact 200-exemplar buffer (Chapter 6). The lexical signature that gates the slots is the part of LexSlot that carries over to replay (§3.5).

3.4.1 Slot Memories at the Diagnosed Site

LexSlot freezes the lower encoder and inserts small additive *slot* modules exactly where the diagnosis places forgetting, leaving the late blocks and the classifier head trainable. Two kinds of slot are used. **Head logit-slots** attach to the classifier: a slot bank of n_{head} learnable value vectors produces a per-token logit shift $\Delta\ell = \big(fV^\top\big)P$ from the per-token features $f \in \mathbb{R}^d$ entering the head (value matrix $V \in \mathbb{R}^{n_{\text{head}} \times d}$, so $fV^\top \in \mathbb{R}^{n_{\text{head}}}$ are the slot activations; projection $P \in \mathbb{R}^{n_{\text{head}} \times |\mathcal{L}|}$ maps them to a shift over the $|\mathcal{L}|$ label logits), added to the head's output and zero-initialised through P. **Late-layer representation-slots** attach to each encoder block in the diagnosed *late* third: a rank-r adapter adds $\Delta h = (h\,D)\,U$ to the block's hidden state ($D \in \mathbb{R}^{d \times r}$ down, $U \in \mathbb{R}^{r \times d}$ up). All slot up-projections are *zero-initialised* (LoRA-style), so an untrained slot is an exact no-op and adding slots never perturbs the network until they are trained. Placement is governed by a `slot_depth` control (*head_only*, *head_late*, *uniform*); the canonical setting is *head_late* (head + late third), matching the diagnosed site, and *uniform* (slots on every layer) is the targeting ablation. Crucially the freeze boundary coincides with the slot boundary: the layers that carry slots are exactly the layers that stay trainable, so stable pretrained features flow into the slots without drifting while the slot-bearing site remains plastic.

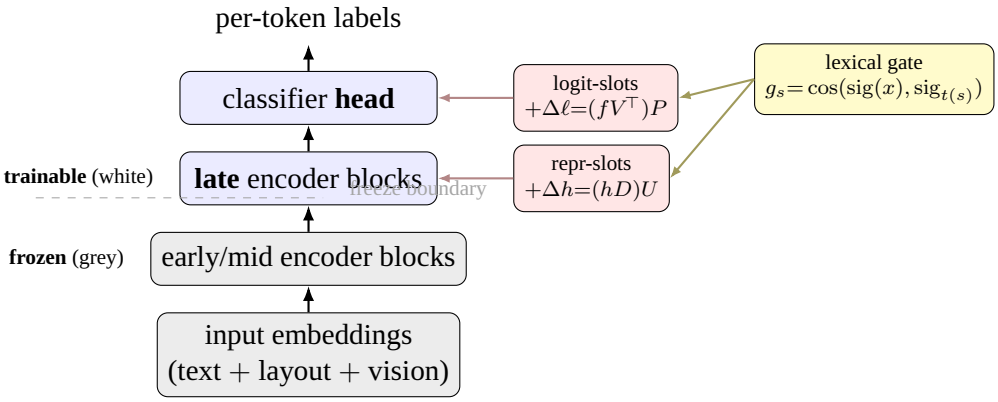
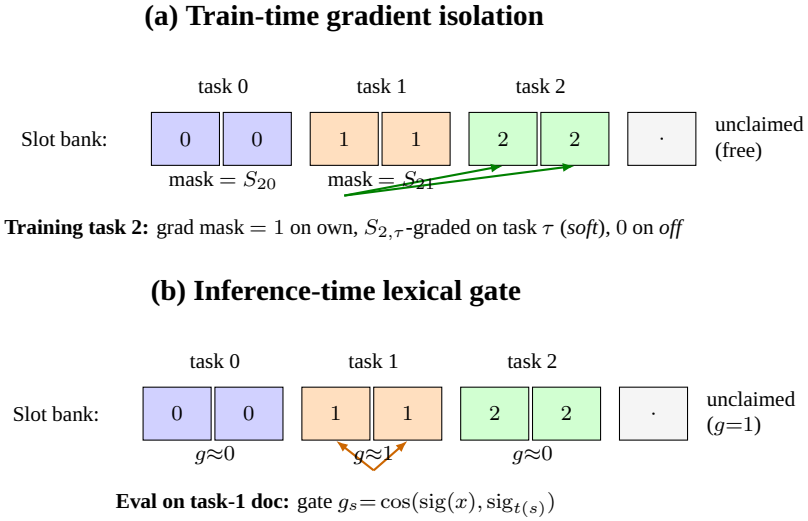


Figure 3.5: LexSlot places small additive slot adapters at the diagnosed forgetting site over a *partially-frozen* backbone. The lower $2/3$ encoder (grey) is frozen so stable pretrained features flow into the slots without drifting; the late blocks and the classifier head (white) stay trainable and carry the slots, the trainable set is exactly the slot-bearing set, and the dashed line marks the freeze boundary. **Logit-slots** add a per-token bias $\Delta\ell$ at the head; **repr-slots** add a low-rank shift Δh in the late blocks. Up-projections are zero-initialised, so an untrained slot is an exact no-op. The *inference lexical gate* (yellow) scales each slot's activation at forward time by the cosine between the document's signature and the slot's owner-task signature, routing the right memory to the right document at prediction time (active in both training and inference; §3.4.3).

3.4.2 Per-Task Isolation: Write-Once Slots

Each task *owns* a fixed, disjoint budget of n_{slots}/T slots, claimed from the unclaimed pool when the task begins (T the expected number of tasks). In the canonical configuration each task updates its own slots fully and co-trains a lexically similar prior task's slots in proportion to their signature similarity: a per-slot gradient mask scales every slot's update by that similarity, so a dissimilar new task can never meaningfully overwrite the parameters a previous task wrote. Because the lower encoder is frozen and low-similarity slots are masked, earlier tasks are *structurally* protected: there is no consolidation penalty to tune and no replay buffer to store. Protection is reinforced at *forward* time by the inference lexical gate (§3.4.3), which routes each document to the slots of its signature-matching task. This is the sense in which LexSlot is *write-once*: knowledge for a task is written into its own slots once and, gated by a drift-immune signature, never disturbed (Figure 3.6).



Both mechanisms are driven by the same drift-immune lexical signature S: the gradient mask protects parameters *at train time; the gate routes* activations *at forward time.*

Figure 3.6: LexSlot isolates tasks through two mechanisms driven by the same drift-immune lexical signature. (a) At *train* time a per-slot gradient mask scales each slot's update: a task trains its own slots fully and, under the canonical *soft* setting, co-trains a prior task τ's slots in proportion to $S_{t,\tau}$ (*off* zeroes them, the isolated ablation). (b) At *forward* time (train and inference) a per-document lexical gate $g_s = \cos(\text{sig}(x), \text{sig}_{t(s)})$ scales each slot's activation, so on a task-τ document the task-τ slots fire strongly and foreign slots are suppressed, routing the right memory to the right document, which the gradient mask alone cannot do. The two are consistent: the signature that gates updates also gates activations.

3.4.3 The Lexical Gate and the Sharing Ablation

Isolation is one extreme of a more general gate. LexSlot computes, for each task, a sparse *lexical signature* (a TF-IDF bag-of-tokens over the task's OCR input tokens) and the cosine similarity S_{ij} between task signatures. This signature is purely input-derived and therefore *drift-immune*: it does not depend on any learned, forgettable parameter. The signature drives *two* consistent mechanisms (Figure 3.6). **At train time** the `slot_sharing` control sets the per-slot gradient mask on a prior task's slots when training a new one: *soft* (the canonical setting) makes them trainable in proportion to S_{ij}, so lexically similar tasks co-train shared slots; *hard* unlocks them above a similarity threshold; *off* zeroes them (the isolated ablation). **At forward time**, in both training and inference, an *inference lexical gate* $g_s = \cos(\mathrm{sig}(x), \mathrm{sig}_{t(s)})$ scales each slot's activation by the cosine between the document's signature and the slot's owner-task signature, so on a task-τ document the task-τ slots fire strongly and foreign slots are suppressed. The gradient mask protects *parameters*; the gate routes *activations*; both use the same signature, so the train-time and forward-time decisions are consistent. The motivation for graded sharing is that on this benchmark the lexical overlap is itself label-bearing: the receipt tasks SROIE and CORD share the `total`/`tax`/`cash` vocabulary ($S \approx 0.31$) while the form task FUNSD is an outlier ($S \approx 0.10$), so high-S tasks might *help* each other by refining a shared lexical-to-label rule, and the inference gate ensures a document only ever activates the slots whose owner task is lexically close. Whether that putative transfer outweighs the interference of letting two tasks write the same slots is an empirical question, and the `slot_sharing` axis is the controlled ablation that answers it (Chapter 6). We present *soft* with the inference gate as the method (it lets lexically related tasks share what is shared and isolates what is not); *off* and *hard* are reported as the ablation that justifies the choice.

3.4.4 Implementation Notes

The slot memory adds no fragile new infrastructure: the head logit-slots and the low-rank late-layer adapters are standard modules attached by forward hooks, the per-slot gradient mask is a single element-wise multiply at the backward pass, the inference lexical gate is one cosine per slot installed by a forward pre-hook, and the lexical signature is one sparse pass over input tokens. In the canonical configuration the protection of old tasks comes entirely from the lexical gradient mask and inference gate: no Fisher penalty, no distillation teacher, and no rehearsal buffer, so the method's standing cost is just the small slot parameters ($\sim$1–2% over the frozen lower encoder, plus the trainable late blocks and head). Slot counts, ranks, and the per-task budget are listed in Appendix A.1.

3.5 Lexically-Routed Latent Replay

The diagnosis says *where* old-task evidence must be re-supplied (the head and late layers); the negative results of Chapter 6 say *what form* it must take (whole documents, whose feature, position, and label stay bound together). Lexically-routed latent replay adds the remaining decision: *which* stored documents to replay for the document currently being learned. It has two parts.

The memory: compressed per-document latents (CoLaR). The encoder is split at layer k; layers below k are frozen after the first task. For each of up to d documents per task, the layer-k activation matrix $H \in \mathbb{R}^{n \times h}$ (n positions, hidden size h) is stored as truncated SVD factors $H \approx U_r \Sigma_r V_r^\top$, together with the document's bounding boxes, attention mask, and labels. Because a single document is low-rank even though the pooled feature space is not, rank r keeps the whole-document binding at a fraction of the raw size. During training on task t, each step adds a replay loss: stored latents are reconstructed and injected at layer k, run through the upper layers and the head, and scored against their stored labels.

The gate: lexical routing. Each stored document j keeps a sparse bag-of-token-ids signature σ_j computed from its OCR tokens at capture time, the same input-derived, drift-immune signal that gates LexSlot's slots. For the current batch, with mean signature $\sigma(x)$, the similarity to each stored document is $s_j = \sigma(x)^\top \sigma_j$, and replay indices are drawn from

$$p_j \;=\; \mathrm{softmax}_j\big(\pm\, s_j/\tau\big),$$

where the sign selects *near* routing ($+$, replay the stored documents whose vocabulary is closest to the current one, where interference is most likely) or *far* routing ($-$, replay the least similar ones, where the current task gives no support), and τ is a temperature. A third mode, *task* routing, first averages s_j within each past task, applies the softmax over tasks, and then samples uniformly within the chosen task. Uniform sampling of the store ($p_j = 1/|M|$) is the CoLaR baseline that every routing mode is compared against. Routing changes only which latents are replayed: the memory, its size, and the replay loss are identical, so any difference is attributable to the gate alone.

3.6 Summary

We have defined the genuinely separable components of a single-stream tri-modal document encoder, a controlled modality-ablation protocol with an external unimodal contrast, and complementary metrics (per-token CKA, per-group importance, an old-task-Fisher-

weighted displacement (the forgetting localiser), and the forgetting matrix) that together localise forgetting by depth, stream, and head. From that measurement we derive where a remedy may attach, and two remedies that attach there: **LexSlot**, an isolated slot memory at the diagnosed site, gated by a drift-immune lexical signature and paired with a compact 200-exemplar replay buffer in its headline configuration; and **lexically-routed latent replay**, which keeps LexSlot's signature as the gate and replaces its memory with compressed per-document latents (CoLaR) at layer k, so the gate decides which stored documents are replayed. The next chapter describes how this methodology is realised in software.

Chapter 4

Implementation and Integration

This chapter describes how the methodology of Chapter 3 is realised as a research framework: the backbone and software stack (§4.1), the document-IE data pipeline (§4.2), the system architecture, including the per-component instrumentation and experiment tracking (§4.3), and the reproducibility provisions that make every reported number regenerable from logs (§4.4).

4.1 Framework Selection

4.1.1 Backbone Selection

The primary backbone is LayoutLMv3-base (`microsoft/layoutlmv3-base`, $\approx$ 133M parameters, 12 layers, hidden size 768). It was chosen because it is document-specific, encoder-only, and small enough to run the full experimental grid (ten baselines and the selected method across three scenarios and three seeds) within the compute budget, while exposing the four separable components central to the diagnosis. LiLT-base and BROS-base serve as secondary backbones for a generalisation study on a reduced method subset. Table 4.1 compares the candidates. This choice directly instantiates the backbone selection criteria laid out in §2.5.

4.1.2 Software Stack: Selection Criteria and Justification

Selection criteria. Five requirements drove the choice of software stack. **(i) Reference implementations:** the diagnosis must run on the canonical LayoutLMv3 weights and preprocessing, not a re-implementation whose deviations could masquerade as forgetting

Table 4.1: Backbones considered for the study. LayoutLMv3-base is the primary subject; LiLT and BROS test generalisation of the diagnosis; large multimodal LLMs are noted as comparison targets but are out of scope for the controlled per-component study.

Backbone	**Type**	**Scale**	**Role**
LayoutLMv3-base	encoder-only, tri-modal	$\approx$ 133M	primary
LiLT-base	encoder-only, decoupled layout	$\approx$ 130M	secondary
BROS-base	encoder-only, spatial	$\approx$ 110M	secondary
InternVL3-8B	multimodal LLM	8B	comparison target (out of scope)

signal. **(ii) Composable configuration at grid scale:** the full grid of methods $\times$ scenarios $\times$ seeds must be launchable from a single entry point with no per-run code edits. **(iii) BIO-correct evaluation:** the metric must be entity-level, span-aware F1 rather than token accuracy. **(iv) Queryable experiment tracking:** the complete grid must be reconstructable from logged runs alone. **(v) Determinism support:** the CKA and Fisher patterns reported in Chapter 6 must reflect signal, not run-to-run noise, so the framework must expose deterministic execution modes.

Justification. PyTorch ($\geq$2.3) satisfies (i) and (v): it is the framework in which the canonical LayoutLMv3 is released, it provides the forward-hook API on which the CKA activation probes are built, and it exposes the cuDNN determinism flags required for reproducible kernels. TensorFlow and JAX were rejected because neither has a maintained LayoutLMv3 reference implementation. HuggingFace Transformers ($\geq$4.41) supplies the canonical LayoutLMv3 model together with its `LayoutLMv3Processor`, and Datasets ($\geq$4.0) loads the benchmark corpora from the HuggingFace Hub, jointly covering (i); reimplementing the backbone or hand-rolling loaders would forfeit the canonical-weights guarantee. PEFT ($\geq$0.11) provides audited LoRA injection for the LoRA-family baselines, avoiding hand-rolled adapter code. For (ii), Hydra ($\geq$1.3.2) with OmegaConf composes declarative configuration groups (method, scenario, model, training) into a fully specified run; `argparse` plus hand-rolled YAML was rejected because it cannot express the sweep grid declaratively. For (iv), Weights & Biases ($\geq$0.17) records every run with structured, queryable tags; TensorBoard was rejected because it offers no structured cross-run querying. Finally, for (iii), `seqeval` ($\geq$1.2.2) computes entity-level BIO-aware F1; plain token accuracy was rejected because it is inflated by the dominant O class and would mask entity-level forgetting. Experiments are launched through a single entry point and configured by composition, e.g.

```
python scripts/train.py method=ewc scenario=cil_cord seed=42
```

```
method.lambda_=1000
```

so that every run is fully specified by its config and reproducible. Table 4.2 summarises the stack.

Table 4.2: Software stack with minimum versions, the role each component plays in this study, and the alternatives considered and rejected against criteria (i)–(v).

Component	**Package (min. version)**	**Role in this study**	**Alternative considered**
Language	Python $\geq$3.10	runtime	N/A
DL framework	torch $\geq$2.3	training; forward hooks (CKA); determinism flags	TensorFlow, JAX
Models	transformers $\geq$4.41	LayoutLMv3 backbone + processor	re-implementation
Data	datasets $\geq$4.0	HuggingFace Hub loading	bespoke loaders
Adapters	peft $\geq$0.11	LoRA injection	hand-rolled adapters
Configuration	hydra-core $\geq$1.3.2	composable configs; sweep launch	argparse + YAML
Tracking	wandb $\geq$0.17	run logging; tag queries	TensorBoard, MLflow
Evaluation	seqeval $\geq$1.2.2	BIO-aware entity F1	token accuracy

4.1.3 Why a Bespoke Continual-Learning Harness

The stack above leaves one decision open: whether to build the continual loop on an established CL framework, Avalanche (Lomonaco et al., 2021), Mammoth (Boschini et al., 2022), or PyCIL (Zhou et al., 2023), or to write a bespoke harness. A bespoke harness was chosen, for three reasons rooted in the task rather than in preference.

Task-shape mismatch. All three frameworks are engineered for image classification: a sample is a single tensor with one integer label, and the metric is top-1 accuracy. Their training abstractions encode this directly: an Avalanche `SupervisedTemplate` reads its input as `mbatch[0]` and scores a sample-level criterion; a Mammoth model implements `observe(inputs, labels, …)` over a single input tensor with integer-target task masking; PyCIL slices an image dataset by integer class sets and herds exemplars on pooled image features. The unit of work here is instead a tri-modal document (WordPiece ids, normalised boxes, and a page image) carrying a *per-token* BIO label sequence and scored with entity-level `seqeval` F1, a metric absent from all three. Adopting any of them would

require replacing their data, model, loss, and evaluation layers at once, which amounts to reproducing the loop described in this chapter inside a contract built for a different task.

Class-incremental at the token level. The head here is a single linear layer that *grows* across sessions over a token label space, preserving trained rows and initialising new ones from $\mathcal{N}(0, 0.02)$. The frameworks either assume a fixed output dimension or grow a head while still routing one input tensor and one label per sample; class-incremental *token* labelling is not a supported mode.

Method coverage. The harness already implements the classical baselines and the 2024–2025 line, flatness-aware replay, LoRA-based and prompt-based methods, used in Chapter 6. The newest strategy shipped by Avalanche or Mammoth predates several of these, so adopting a framework would subtract, not add, capability. These libraries remain the reference for the classical baselines, whose published algorithm definitions the implementations follow; control is simply not inverted to them, for the reasons above.

4.2 Document-IE Pipeline Integration

Every document passes through the same five-step preprocessing pipeline, regardless of dataset or continual scenario:

1. **Load.** Dataset-provided OCR tokens, word boxes, and the page image are loaded; no OCR engine is ever run.
2. **Tokenise.** The `LayoutLMv3Processor` (`apply_ocr=False`) WordPiece-tokenises the words, truncating and padding to a maximum sequence length of 512.
3. **Normalise layout.** Word boxes are scaled to the integer range $[0, 1000]^4$ and degenerate or out-of-range boxes are clamped.
4. **Encode image.** The page image is resized to 224×224 and consumed by the backbone as 16×16 patches.
5. **Align labels.** Each sub-token inherits its word's box, only the first sub-token carries the BIO label, and trailing sub-tokens are set to the ignore index -100.

Figure 4.1 summarises the pipeline. The same preprocessing is shared across all datasets so that the only thing that changes between continual tasks is the data and label space, not the pipeline. For the diagnostic conditions (C1–C4) a modality mask is applied at the input layer, zeroing image pixels, replacing text with padding tokens, or zeroing lay-

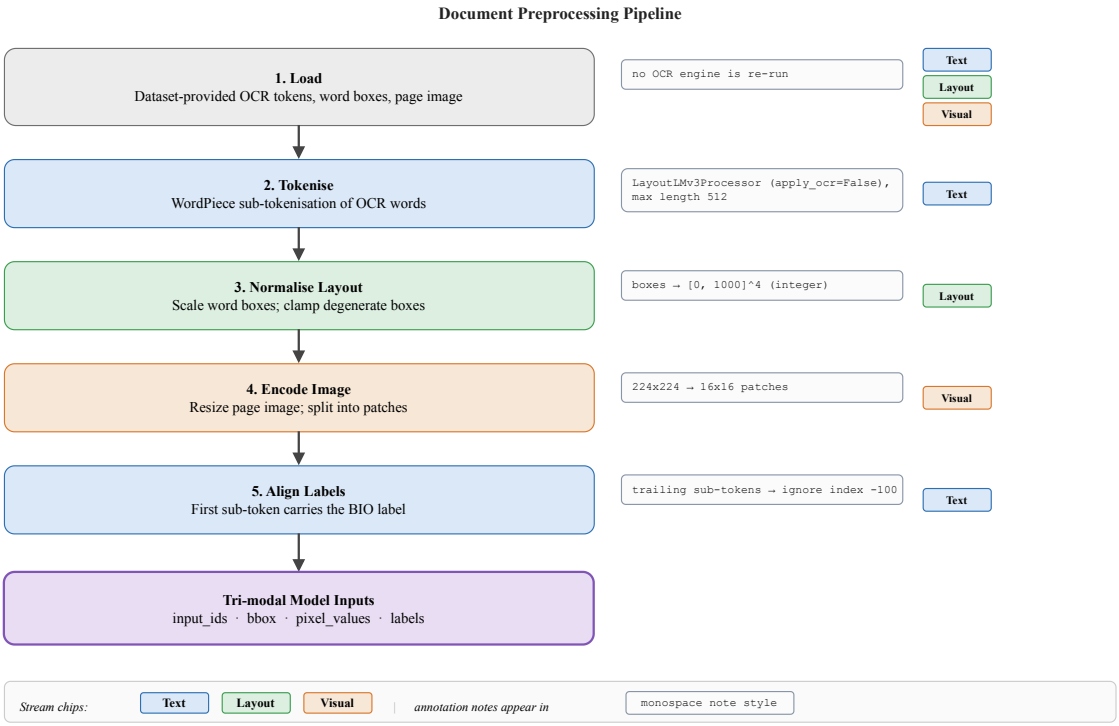


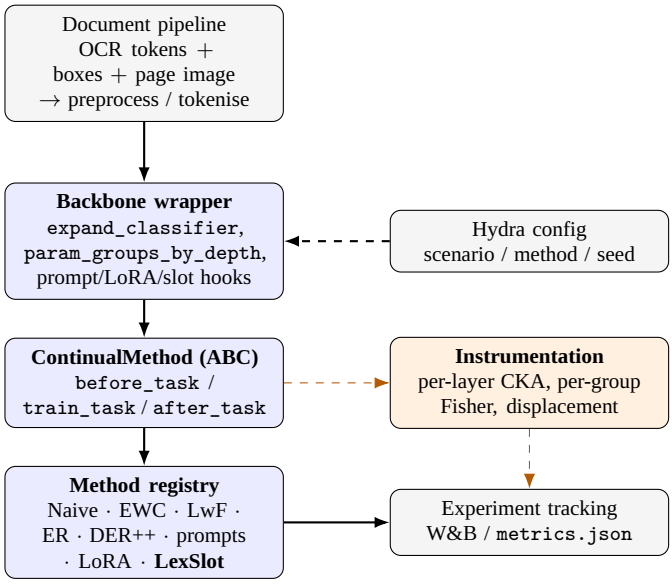
Figure 4.1: The five-step document preprocessing pipeline. Dataset-provided OCR tokens, word boxes, and the page image are tokenised, normalised, and aligned into the tri-modal inputs of the backbone; colour chips mark which input stream each step feeds.

out positions, through the backbone wrapper's `modality_mask` forward argument rather than a separate ablated model, so the four conditions differ only in which input pathways are active.

Data integrity and validation. Three preprocessing invariants are enforced and checked so that the forgetting signal is not confounded by data artefacts. First, WordPiece token–box alignment is validated at load time: every sub-token is tied to exactly one word box, the first sub-token of a word carries that word's BIO label, and the trailing sub-tokens are set to the -100 ignore index so that the loss and the entity-F1 metric see one prediction per word. Second, bounding boxes are normalised to the $[0, 1000]^4$ integer range expected by the backbone's 2D position embeddings, with clamping to guard against out-of-range or degenerate (zero-area) boxes. Third, and most importantly for a controlled diagnosis, the pipeline consumes the dataset-provided OCR transcriptions and boxes directly and never re-runs an OCR engine; this removes OCR quality as a confounder, so that any measured drift is attributable to continual training of the encoder rather than to changing recognition behaviour across tasks.

Chapter 5 specifies the datasets this pipeline consumes.

4.3 System Architecture Overview



Solid: data/control flow. Dashed/orange: config and instrumentation taps (do not alter the training path).

Figure 4.2: System architecture. The continual-learning core exposes a uniform method interface; the backbone wrapper exposes extension points for classifier expansion, prompts, and LoRA banks; the instrumentation layer records per-layer CKA and per-group Fisher; all runs report to the experiment tracker. Data flows from the shared document pipeline through the wrapped backbone to the method-specific heads, while the instrumentation layer taps the backbone at task boundaries without altering the training path.

Figure 4.2 gives the overall picture: a thin continual-learning core orchestrates the per-task loop, the backbone wrapper hides architecture-specific details behind a stable set of extension points, the document pipeline feeds both, and a decoupled instrumentation layer observes the backbone without perturbing it. The remainder of this section describes each block in turn.

4.3.1 Continual-Learning Core

All methods implement a common `ContinualMethod` interface with four hooks: `before_task` (e.g. expand the classifier head or allocate a LoRA bank), `train_task` (the training loop), `after_task` (update the replay buffer, compute Fisher, freeze weights), and `evaluate` (evaluate on all tasks seen so far). This interface lets regularisation, replay, prompt-based, and LoRA-based methods share the same training and evaluation harness, which is essential for like-for-like comparison. Algorithm 2 shows how the core drives these hooks per task.

Algorithm 2 The continual-learning core loop

Require: task sequence $\mathcal{T}_{1:T}$, method m, backbone $\theta^{(0)}$

for $t = 1$ **to** T **do**
  m.`before_task`$(\mathcal{T}_t)$ ▷ expand head (Eq. 4.1); allocate prompts, adapters, penalties
  m.`train_task`$(\mathcal{T}_t)$ ▷ AdamW, constant LR, val-F1 early stopping; replay/distillation internal to m
  m.`after_task`$(\mathcal{T}_t)$ ▷ update buffers, estimate Fisher, freeze parameters as m requires
  for all $j \leq t$ **do**
    $R[t, j] \leftarrow$ EntityF1$(m$.`evaluate`$, \mathcal{T}_j)$
  end for
  Snapshot(per-layer CKA, per-group Fisher) ▷ decoupled instrumentation hooks
end for
return R and derived AA/BWT/AF/FWT

Joint upper bound. The Joint method bypasses the loop entirely: the classifier is expanded to the union label space upfront and a single training pass runs over the concatenation of all sessions' data. It is the only method that sees all data simultaneously and serves as the oracle reference of Chapter 5.

Convergence-based training. The shared `train_task` loop trains each task to convergence rather than for a fixed number of epochs. A common `EarlyStopper` (in `doccl/methods/base.py`) evaluates the task's validation entity-F1 after each epoch, keeps a CPU snapshot of the best-scoring weights, and stops once the score fails to improve for two consecutive epochs; `restore_best` reloads that snapshot before the task boundary. Because the stopper lives in the base class, every method, regularisation, replay, distillation, and the proposed method alike, inherits the identical stopping rule for free, which is what makes the comparison fair under the protocol of §5.3.4.

The proposed method (lexically-routed latent replay). The method is implemented in `doccl/methods/colar_pp.py` on top of the latent-replay base class (`doccl/methods/latent_replay.py`). The memory is CoLaR's: a forward hook captures each stored document's layer-k activations, which are kept as truncated SVD factors and re-injected at layer k during replay. The gate is a `replay_route` key (*none*/*near*/*far*/*task*) with a temperature `route_temp`: when it is set, each stored document keeps a sparse bag-of-token-ids signature computed at capture time, and the replay sampler draws indices from the softmax of the current batch's similarity to those signatures. With `replay_route=none` the method reduces exactly to uniform CoLaR, which is the control.

The lexical gate's origin (LexSlot). LexSlot is implemented in `doccl/methods/lexslot.py` (with the slot modules in `doccl/methods/lexslot_memory.py` and the lexical gate in `doccl/methods/lexslot_mask.py`) over the same frozen-backbone harness as the baselines, so it introduces no new training subsystem. The slot adapters attach by forward hooks at the classifier head and the late encoder blocks located through the `param_groups_by_depth` grouping of the backbone; a `slot_depth` configuration key selects the *head_only*, *head_late*, or *uniform* placement and a `slot_sharing` key the *off*/*soft*/*hard* gate, which together drive the ablations of Chapter 6. A per-task gradient mask, derived from the lexical-signature similarity, freezes other tasks' slots during the backward pass. The method is registered under the same Hydra factory as the baselines, so it is selected by name (`method=lexslot`) on the identical harness.

4.3.2 Per-Component Instrumentation

The diagnostic of §3.2 is implemented as a set of hooks that, at each task boundary, capture activations at the tracked layers (for CKA) and accumulate squared gradients per parameter group (for Fisher), writing per-run JSON dumps that are later aggregated into a long-format dataframe for analysis and plotting. The instrumentation is decoupled from the methods so that it can be attached to any run, including the naive-sequential pilot runs. Four mechanisms make up the layer:

- **Representational drift (CKA).** Forward hooks capture per-layer activations on a *fixed* probe set: the same held-out documents are used at every task boundary so that representations are compared on identical inputs. Linear CKA between the activations of two checkpoints scores how much each layer's representation has changed; because it is invariant to orthogonal transforms and isotropic scaling, a drop signals genuine representational reorganisation rather than a trivial rotation. The probe activations are gathered through a hook helper and scored layer by layer.
- **Parameter importance (per-group Fisher).** Over a task's data the diagonal empirical Fisher accumulates squared gradients of the log-likelihood with respect to each parameter, summed within the eight named parameter groups (text word embeddings, 2D layout-position embeddings, image patch embedding, text attention, visual attention, cross-modal fusion, feed-forward, and classifier). Comparing the per-group Fisher before and after a task gives a per-component importance-drop profile that complements the CKA view.
- **Classifier-head expansion.** When a new task introduces new BIO labels, the classification head is grown to cover them while the rows corresponding to previously seen labels are preserved, so that old-label logits are not reset by the arrival of new

classes. Formally,

$$W^{(t)} = \begin{bmatrix} W^{(t-1)} \\ W_{\text{new}} \end{bmatrix}, \qquad W_{\text{new}} \sim \mathcal{N}(0,\, 0.02^2), \qquad b_{\text{new}} = 0, \tag{4.1}$$

where the rows of $W^{(t-1)}$, the previously seen BIO labels, are copied unchanged so that old-label logits are exactly preserved, and $W^{(t)} \in \mathbb{R}^{|\mathcal{Y}_t| \times 768}$ for the label set $\mathcal{Y}_t$ after session t. Expansion is performed once by the training loop before each task, and the label↔index maps are updated in lockstep.

- **Modality ablation.** The pilot conditions are realised by zeroing input pathways *in place* rather than by instantiating separate models: the text, visual, and layout streams are masked individually through a single forward argument, so C1–C4 share identical weights and differ only in which modalities reach the encoder. This is what makes the per-condition comparison apples-to-apples.

Code pointers. For reproducibility, the components above map directly onto the source: linear CKA and the activation hooks live in `doccl/eval/cka.py`; the diagonal and per-group Fisher in `doccl/eval/fisher.py`; classifier expansion, the eight-group `param_groups` mapping, and the `modality_mask` forward argument in `doccl/models/layoutlm_wrapper.py`; and the method lifecycle hooks in `doccl/methods/base.py`. The single Hydra entry point `scripts/train.py` wires these together and resolves each method from its registry.

4.3.3 Datasets and Scenario Builders

Dataset loaders expose FUNSD, CORD, and SROIE in the shared format, and scenario builders assemble them into continual sequences (class-incremental, domain-incremental, mixed) through a registry, so that a scenario is selected by name from the config.

4.3.4 Compute and Deployment

Hardware tiers. Experiments run on a single RTX 4090 (cloud), with a local RTX 2060 reserved for debugging only; paper-result experiments are never run on the debug machine. Determinism is enforced (fixed seeds, deterministic cuDNN) so that CKA and Fisher patterns reflect signal rather than run-to-run noise.

Memory management. Two documented low-VRAM knobs allow the debug tier to run the same code path: reducing the batch size from 8 to 2 and enabling gradient checkpoint-

ing. Production runs use full fp32 precision (an fp16 flag exists but is disabled by default), so no precision-related nondeterminism enters the measurements.

Resource accounting. Every run records wall-clock time per task, parameter counts, and peak GPU memory in its `metrics.json` (§4.3.5), enabling the computational-cost comparison of Chapter 6.

4.3.5 Experiment Tracking and Monitoring

All runs report to Weights & Biases under a single public project (`https://wandb.ai/thanh-workspace/CL4IE`), with a structured tag schema (`{phase}/{scenario}/{method}/seed{seed}`; pilot runs use `pilot/{condition}/seed{seed}`), so that the full grid is queryable and the result tables of Chapter 6 can be regenerated from the tracked logs. Each run is named `{scenario}_{method}_seed{seed}`, and its per-run `results/<run>/metrics.json` holds the final Average Accuracy (AA), Backward Transfer (BWT), Average Forgetting (AF), and Forward Transfer (FWT), the full performance matrix R, wall-time per task, parameter counts, and peak GPU memory, with the raw matrix additionally saved as `matrix.npy`. Downstream, `scripts/analyze_results.py` aggregates the runs into the LaTeX result tables and `scripts/ingest_to_thesis.py` copies them into the thesis source, so every reported number is regenerable from logs. Table 4.3 lists the artefacts each run produces and who consumes them.

Table 4.3: Artefacts produced by every run and their consumers in the offline results pipeline.

Artefact	**Contents**	**Consumer**
`results/<run>/metrics.json`	final AA, BWT, AF, FWT; full matrix R; wall-time; parameter counts; peak GPU memory	`analyze_results.py`
`results/<run>/matrix.npy`	raw $R[i, j]$	aggregation and plotting
W&B run	per-step losses, config snapshot, tags	grid-level queries

4.4 Reproducibility

Repository and environment. The complete implementation is available at `https://github.com/hoangthanh283/doccl` (branch `doccl`). The environment is

Python $\geq$3.10 with the minimum versions of Table 4.2 pinned in `pyproject.toml`.

Seeds and determinism. All main results use three seeds (42, 123, 7). cuDNN runs in deterministic mode with benchmarking disabled (`torch.backends.cudnn.deterministic=True`, `benchmark=False`), and every run is fully specified by its Hydra configuration.

One-command reproduction. The diagnostic study, four conditions $\times$ three seeds over FUNSD $\rightarrow$ CORD $\rightarrow$ SROIE, reproduces with `bash scripts/run_pilot.sh`; the main grid with `bash scripts/run_grid.sh`, where the `PHASE` variable selects the core, advanced, selected-method, or ablation slices; and a single run via the Hydra entry point of §4.1.2. SROIE is prepared once by `python scripts/prepare_sroie.py` from a public HuggingFace mirror (`mp-02/sroie`) or the ICDAR-2019 originals.

Tracked artefacts. The public Weights & Biases project of §4.3.5, together with the per-run artefacts of Table 4.3, allows the result tables to be regenerated end-to-end from logs.

4.5 Summary

The framework pairs a small, component-separable backbone with a uniform continual-method interface, a shared document pipeline, decoupled per-component instrumentation, and structured experiment tracking, and it is reproducible end-to-end through a public repository, a pinned environment, and result tables regenerated directly from tracked logs. This design makes the diagnosis attachable to any run and the baseline comparison strictly like-for-like, which the next chapter relies upon.

Chapter 5

Experimental Setup

This chapter specifies the datasets (§5.1), the continual task construction (§5.2), the model and training configuration (§5.3), the baselines (§5.4), and the evaluation methodology (§5.5). Together these define the FCS-CL benchmark used in Chapter 6.

5.1 Datasets

We use five public VRDU datasets (Table 5.1). Three English datasets (FUNSD, CORD, SROIE) form the core benchmark and span forms and receipts across a wide range of label-space sizes; two further datasets extend the benchmark onto axes the core trio cannot probe. **XFUND** adds a *language-shift* axis (the multilingual sibling of FUNSD, seven non-English languages) and **WildReceipt** adds a *scale and schema-richness* axis (a larger receipt corpus with a 24-class key/value schema). All datasets are used with their *dataset-provided* OCR and bounding boxes; we do not re-run OCR, which removes a confound and keeps the pipeline fixed across the whole study.

FUNSD. Loaded from the HuggingFace release `nielsr/funsd-layoutlmv3`, FUNSD provides 149 training and 50 test documents and ships no validation split. Its three entity types (HEADER, QUESTION, ANSWER) plus background give 7 BIO tags. The documents are noisy, full-page scanned forms: small in number but the clearest test of the KEY/VALUE relation.

CORD. Loaded from `naver-clova-ix/cord-v2`, CORD provides 800 training, 100 validation, and 100 test receipts. Its 30 fine-grained classes, grouped into five super-classes (menu, sub_total, total, void_menu, sub), give 61 BIO tags at fine granularity and 11 at super-class granularity. The fine-grained label space makes CORD the natural

host for the class-incremental scenario of §5.2.1, while the super-class view is used in the domain-incremental remapping.

SROIE. Prepared once by `scripts/prepare_sroie.py` from the HuggingFace mirror `mp-02/sroie` (or, equivalently, the ICDAR-2019 originals), SROIE provides 626 training and 347 test scanned receipts and, like FUNSD, ships no validation split. Its four fields (company, address, date, total) give 9 BIO tags, and it contributes the largest test split.

XFUND (cross-lingual extension). Loaded from `nnul/xfund-multilingual`, XFUND is the multilingual counterpart of FUNSD, covering seven non-English languages (German, Spanish, French, Italian, Japanese, Portuguese, Chinese). It shares FUNSD's exact entity schema (HEADER, QUESTION, ANSWER $\rightarrow$ 7 BIO tags), which lets it reuse the unified domain-incremental mapping unchanged. XFUND supports the *cross-lingual* domain-incremental scenario (§5.2.2), in which the domain shift between tasks is the *language* while the label space is held fixed, isolating representation drift under language shift from any schema confound. The default language order de $\rightarrow$ es $\rightarrow$ fr $\rightarrow$ it $\rightarrow$ zh spans four Latin-script languages plus Chinese, so the sequence stresses both vocabulary drift and a script change.

WildReceipt (scale extension). Loaded from `kaydee/wildreceipt`, WildReceipt is a larger English receipt corpus ($\sim$1.7k images) with a richer 24-class key/value schema (paired `*_key`/`*_value` fields; the upstream `Ignore`/`Others` classes map to background), giving 49 BIO tags after run-encoding the upstream flat span labels. It hosts a second class-incremental scenario (§5.2.1) at greater scale and label-space breadth than CORD.

All datasets are consumed by the shared five-step preprocessing pipeline of §4.2; OCR transcriptions and word-level bounding boxes are dataset-provided throughout, so no OCR engine enters the experimental loop. XFUND and WildReceipt ship page images with their HuggingFace releases, so no separate image-preparation step is required.

5.2 Continual-Learning Task Construction

Three scenarios are constructed over these datasets.

5.2.1 Class-Incremental (CIL-CORD)

CORD's 30 fine-grained classes are split into five sessions of six classes each, ordered as a curriculum from core menu fields to auxiliary void/sub categories (Table 5.2, Figure 5.1).

Table 5.1: Statistics of the five datasets (three core + two extension). Tag counts are BIO-aware and include the background "O" tag; split sizes follow the standard releases used here. OCR is dataset-provided throughout. FUNSD and SROIE ship no validation split; validation usage is specified in §5.3.4. XFUND counts are per language.

Dataset	**Train**	**Val**	**Test**	**Types**	**BIO tags**	**Domain (avg. tokens)**
FUNSD	149	–	50	4	7	scanned forms (∼700)
CORD	800	100	100	30	61	receipts, fine-grained (∼80)
SROIE	626	–	347	4	9	scanned receipts (∼150)
Extension datasets (language-shift and scale axes):						
XFUND	∼149	–	∼50	4	7	multilingual forms, 7 languages
WildReceipt	∼1.4k	–	∼0.3k	24	49	receipts, key/value schema

Formally, the per-session label spaces grow monotonically,

$$\mathcal{L}_1 \subset \mathcal{L}_2 \subset \cdots \subset \mathcal{L}_5, \qquad |\mathcal{L}_t| - |\mathcal{L}_{t-1}| = 6, \tag{5.1}$$

with six fine-grained classes added at each session over a single shared input domain $P(X)$ (receipt images): only the label space changes from task to task. Because new classes are absent from earlier sessions, their tokens are folded into the background "O" tag until they are introduced, so the meaning of "O" shifts as the sequence advances. Fine-grained (not super-class) growth is deliberate: real KIE pipelines face exactly this: a receipt parser that begins with a handful of fields and accrues dozens over its deployment lifetime, whereas a five-super-class split would be trivially easy. The monotone growth of Equation (5.1) is precisely what induces the CIL *semantic shift of the "O" tag* (§2.2.2) noted above, which each session inherits and which we handle with the KD/pseudo-label protocol established in the CNER literature.

The 30 classes are partitioned into the five sessions by a hand-curated easy-to-hard ordering rather than at random, so that the curriculum reflects the order in which fields are realistically added to a deployed parser; the exact six classes introduced at each session are tabulated in Appendix A.1.1. A direct consequence is that class frequencies are markedly imbalanced across sessions: core menu fields dominate early sessions while void and auxiliary categories are sparse and arrive late. We deliberately leave this imbalance *unmodified* (no re-sampling or re-weighting) and treat it as a controlled factor of the benchmark, since artificially re-balancing the sessions would misrepresent the long-tailed schema growth that motivates the scenario. Concretely, applying the split to CORD's 800 training receipts (keeping every document that contains at least one in-session entity and relabelling out-of-session tokens to "O") yields per-session training pools of 800, 382, 549, 723, and 780 documents for sessions 0–4 respectively, reflecting how many receipts carry

each session's fields.

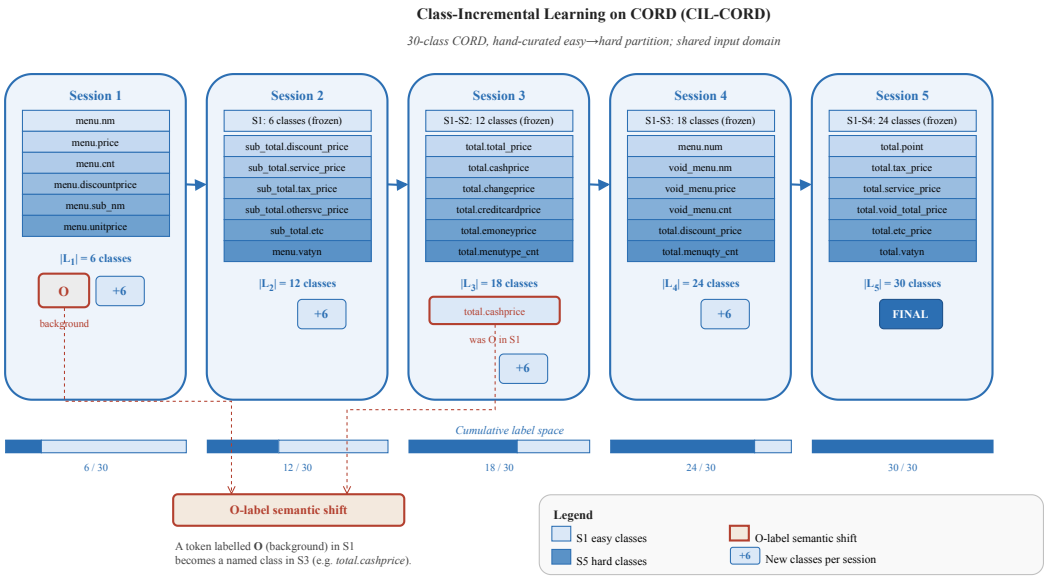


Figure 5.1: The class-incremental CIL-CORD scenario. CORD's 30 fine-grained classes are introduced six at a time across five sessions over a single fixed input domain, so the label space grows monotonically while the receipt-image distribution is held constant. Tokens of not-yet-introduced classes are absorbed into the background "O" tag, which therefore changes meaning as the sequence advances.

Table 5.2: The class-incremental split of CORD into five sessions of six classes, ordered easy-to-hard. Class names follow CORD's native `category.field` naming; the full per-session class lists are given in Appendix A.1.1.

Session	Super-class focus	Difficulty
0	core menu fields	easiest
1	extended menu + sub	moderate
2	sub_total fields	moderate
3	total / payment methods	hard
4	residual total + void/auxiliary	hardest

5.2.2 Domain-Incremental (DIL)

FUNSD, SROIE, and CORD are remapped onto a unified four-entity schema and presented in order of increasing distance from that canonical schema: FUNSD $\rightarrow$ SROIE $\rightarrow$ CORD (Figure 5.2). Formally, a single label space (9 BIO tags) is held fixed across the sequence,

$$\mathcal{L} = \{\text{header}, \text{key}, \text{value}, \text{other}\}, \tag{5.2}$$

while the input distribution shifts from task to task, $P_t(X) \neq P_{t'}(X)$ for $t \neq t'$, with each step moving further from the canonical form schema (full-page forms → receipts → fine-grained receipts). The mapping (Table 5.3) is the part of the benchmark most exposed to judgement: FUNSD aligns naturally (QUESTION/ANSWER = KEY/VALUE), SROIE has no explicit key/value split (COMPANY is treated as KEY-like, the rest as VALUE), and CORD has no KEY at all (productive content maps to VALUE, void/sub to OTHER). The asymmetric SROIE mapping is chosen so that every tag receives discriminative gradient signal. Two consequences of this construction are made explicit rather than hidden, because they bear directly on how the DIL results must be read. First, KEY originates *only* from FUNSD (QUESTION) and SROIE (COMPANY); CORD contributes none, so the unified schema is *VALUE-dominant*. Second, the SROIE BIO tags are themselves run-encoded from a single-token-per-field (S-scheme) mirror (first token of a run → `B-`, the rest → `I-`). We therefore *report per-class F1 and per-task label frequencies* for every DIL result (§5.5.4, Chapter 6): a high aggregate average accuracy on a VALUE-dominant schema is only interpreted as genuine cross-domain retention if the sparse classes (KEY in particular) are themselves retained, and is otherwise flagged as a label-degeneracy artefact. The per-task frequencies are produced model-free by `scripts/dil_label_report.py`.

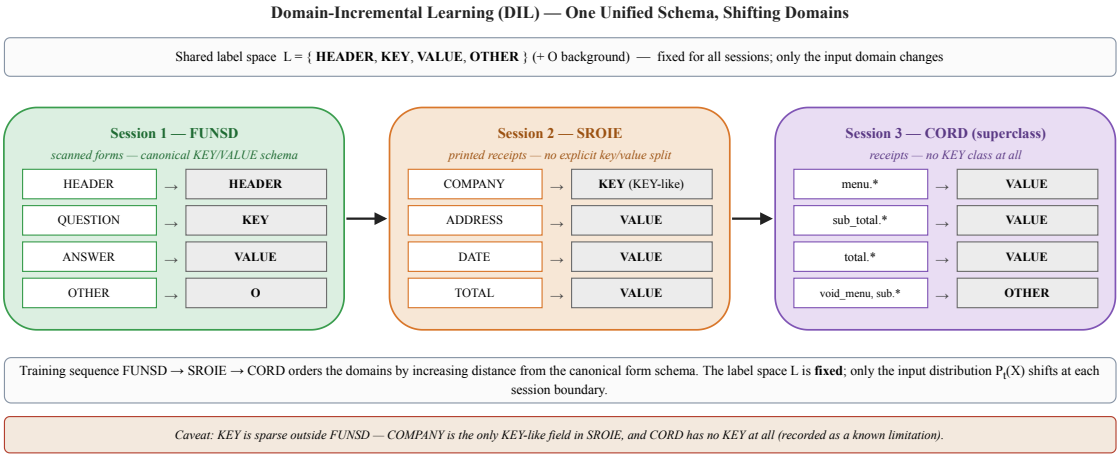


Figure 5.2: The domain-incremental DIL scenario. A single unified four-entity schema (HEADER/KEY/VALUE/OTHER) is held fixed while the input distribution shifts along the sequence FUNSD → SROIE → CORD, each step moving further from the canonical form schema. The label space is constant; only the document distribution changes.

5.2.3 Mixed (Compositional)

A six-session scenario interleaves class-incremental growth within CORD with domain shifts across datasets, e.g. CORD-menu → FUNSD → CORD-menu-extended → SROIE

Table 5.3: Mapping of each dataset's native classes to the unified DIL schema (HEADER/KEY/VALUE/OTHER). The reverse order (CORD → SROIE → FUNSD) is used as a robustness check.

Dataset	**Native → unified**
FUNSD	HEADER→HEADER; QUESTION→KEY; ANSWER→VALUE; OTHER→O
SROIE	COMPANY→KEY; ADDRESS/DATE/TOTAL→VALUE
CORD	menu/sub_total/total→VALUE; void_menu/sub→OTHER

→ CORD-sub_total → FUNSD-revisit, the final session serving as an explicit retention check (Figure 5.3). Formally, the scenario is a length-six task sequence $(\mathcal{T}_1, \ldots, \mathcal{T}_6)$ in which each boundary $\mathcal{T}_t \to \mathcal{T}_{t+1}$ is typed as either a *new-class* transition (label growth at a fixed domain, as in §5.2.1) or a *new-domain* transition (a shift in $P_t(X)$ at a fixed schema, as in §5.2.2). By alternating the two boundary types within one sequence, the scenario stresses plasticity and stability simultaneously rather than in isolation.

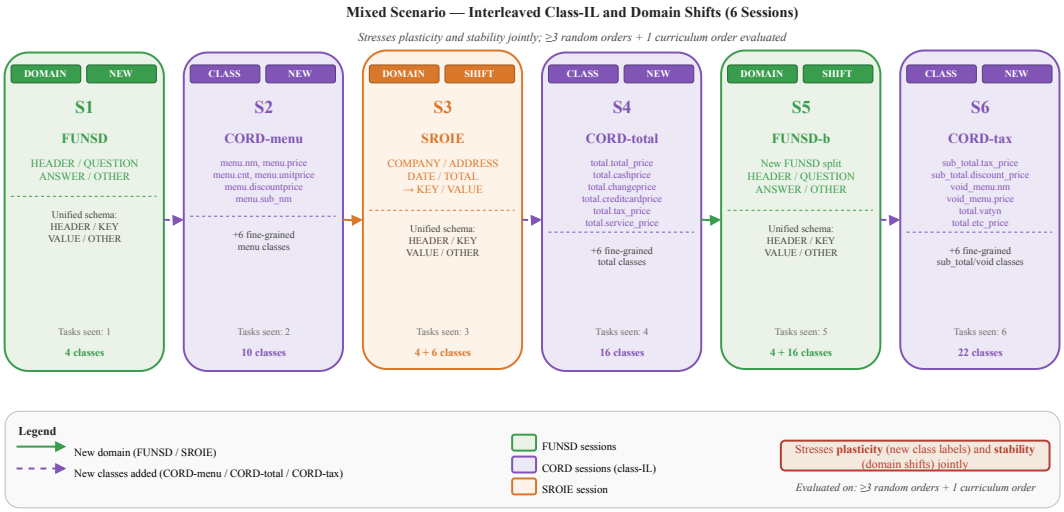


Figure 5.3: The Mixed compositional scenario: a length-six task sequence whose boundaries alternate between new-class transitions (label growth within CORD at a fixed domain) and new-domain transitions (a shift in input distribution at a fixed schema). The final FUNSD-revisit session is an explicit retention check, so the sequence probes plasticity and stability jointly.

5.2.4 Extended-Benchmark Scenarios (Language Shift and Scale)

Two further scenarios extend the benchmark beyond the three core English datasets, each probing an axis the core trio cannot. They reuse the same methods, metrics, seeds, and training protocol, so they compose directly with the core results.

Cross-lingual domain-incremental (`dil_xlingual`). A domain-incremental sequence over XFUND in which the domain shift between tasks is the *language*: de $\rightarrow$ es $\rightarrow$ fr $\rightarrow$ it $\rightarrow$ zh. Because every XFUND language shares FUNSD's schema, the unified label space (Table 5.3, row "FUNSD") is held *fixed* across all five tasks and the classifier head never grows. Forgetting in this scenario is therefore pure representation drift under language shift, with no label-space confound, making it a direct test of whether the output-side forgetting finding of Chapter 6 generalises beyond English. The order interposes four Latin-script languages and a final logographic one (Chinese), so the sequence stresses both vocabulary drift and a script change.

Larger-scale class-incremental (`cil_wildreceipt`). A second class-incremental scenario splits WildReceipt's 24 key/value entity classes into four sessions with a head that grows session by session, exactly as in §5.2.1 but at greater scale ($\sim$1.7k receipts) and label-space breadth (49 BIO tags). It tests whether the class-incremental findings hold when the host dataset is larger and its schema richer than CORD.

These two scenarios are part of the benchmark and its tooling; their empirical results are obtained on the same grid as the core scenarios and reported alongside them (Chapter 6).

5.2.5 Task Ordering

For each scenario we report one fixed order for the main results and additionally run at least two alternate (random) orders, reporting mean $\pm$ standard deviation, so that conclusions do not hinge on a single sequence. The diagnosis of Chapter 3 is likewise repeated under an alternate order.

5.3 Model Architecture and Training Configuration

5.3.1 Architecture Specification

The backbone is LayoutLMv3-base (§4.1), used identically across every method, scenario, and seed. Table 5.4 records the full specification so that the experimental conditions are reproducible from the thesis alone. The token-classification head is expanded as new labels appear, following the logit-preserving expansion of Equation (4.1) in Chapter 4: old-class rows are carried over unchanged and only the newly added rows are freshly initialised.

Table 5.4: Architecture specification used for all experiments on the primary backbone.

Item	Specification
Backbone	`microsoft/layoutlmv3-base`: encoder-only, 12 layers, hidden size 768, ≈133M parameters
Text input	WordPiece, maximum sequence length 512
Layout input	word boxes normalised to $[0, 1000]^4$
Image input	224×224 page image as 16×16 patches
Classification head	linear over the current BIO label set; expanded per session (Eq. 4.1)
Label convention	first sub-token labelled; trailing sub-tokens ignored (-100)

5.3.2 Training Objective

Training minimises the plain token-level cross-entropy

$$\mathcal{L}_{\mathrm{CE}}(\theta) = -\frac{1}{|\mathcal{I}_B|} \sum_{i \in \mathcal{I}_B} \log p_\theta(y_i \mid x)\,, \qquad \mathcal{I}_B = \{\, i : \tilde{y}_i \neq -100 \,\}, \tag{5.3}$$

where $\mathcal{I}_B$ indexes the first sub-token of each word; padding and trailing sub-tokens carry the ignore label -100, and no class weights are applied. The loss is deliberately unweighted (no re-weighting, focal terms, or re-sampling), so that the native long-tailed label distribution is preserved as a controlled benchmark factor, matching the treatment of session imbalance in §5.2.1. The -100 masking also makes the loss and the entity-level F1 metric agree on one prediction per word (step 5 of the pipeline, §4.2).

5.3.3 Optimisation Protocol

Unless otherwise stated, training uses AdamW with a learning rate of 5×10^{-5} held constant throughout (no scheduler, no warmup), weight decay 0.01, batch size 8 without gradient accumulation, gradient clipping at max-norm 1.0, all in full fp32 precision (an fp16 flag exists in the framework but is disabled for all reported runs). Each task is trained *to convergence* rather than for a fixed number of epochs: training stops when the task's validation entity-F1 fails to improve by at least 0.1 points for two consecutive epochs (patience 2), subject to a safety ceiling of 100 epochs that in practice never binds: tasks converge in roughly four to eight epochs. The same constant learning rate and the same stopping *rule* (identical patience and criterion) are applied to every method and every session, so the comparison remains apples-to-apples while each task is allowed to reach its own convergence point; this replaces an earlier fixed-budget protocol that systematically under-trained the slower-converging methods (notably EWC, whose Fisher penalty delays

fitting). Hyperparameter sensitivity is treated as a separate ablation in the main grid rather than tuned per method, to keep the comparison fair. Determinism is enforced via fixed seeds and deterministic cuDNN. Full hyperparameters, including the per-method sweeps, are listed in Appendix A.1.

5.3.4 Checkpointing and Model Selection

The model carried into session $t{+}1$, and evaluated on all tasks seen so far, is the *best-validation-F1* checkpoint of session t: at each epoch the task's validation entity-F1 is measured, and the weights at the highest-scoring epoch are restored before the session boundary. This pairs with the convergence-based stopping of §5.3.3 so that what is carried forward is a properly-fitted model rather than a possibly under- or over-trained final epoch; the same rule is applied identically to every method, so it does not advantage any one of them.

Validation signal and its honest limitation. Of the three datasets only CORD provides a native validation split; FUNSD and SROIE do not (Table 5.1). For those two the early-stopping signal is therefore read from the task's evaluation set, the same set on which the reported metrics are later computed. This introduces a mild optimistic selection bias: the best-epoch checkpoint is chosen using information from the set it is then scored on. We bound rather than eliminate this bias, and we are explicit about it. The bias is small because the stopping rule has short patience (2) and selects only among a handful of late epochs rather than searching a large checkpoint pool; it is shared identically by every method, so it is unlikely to substantially reorder the comparison (though, because methods have different epoch-wise evaluation trajectories, we cannot rule out small reorderings near ties); and we retain the earlier fixed-budget (no-early-stopping) runs as a reference point against which the convergence-trained numbers can be checked (§6.7). A held-out validation split carved from each training set would remove the issue entirely and is noted as a refinement for the final camera-ready (Chapter 7); given FUNSD's small training set ($\sim$150 documents) the practical split is constrained, so we prefer explicit disclosure here.

5.4 Baselines and Continual-Learning Strategies

Every comparison in this thesis uses one frozen reference set of twelve rows: two bounds and ten published baselines, one per family and era, all under the shared interface of §4.3.1 (Table 5.5). The ten are ER (Rolnick et al., 2019) and DER++ (Buzzega et al., 2020) for replay, LwF (Li and Hoiem, 2016) for distillation, CODA-Prompt (Smith et al., 2023) for prompts, CL-LoRA (He et al., 2025) for parameter-efficient adaptation, LCA (Tran et al.,

2026) for model merging, and two *modality-paired* slots that hold the strongest head-side and representation-side method of each modality: SLCA (Zhang et al., 2023b) and RanPAC (McDonnell et al., 2023) on image class-incremental learning, CPFD (Zhang et al., 2023a) and IS3 (Chen et al., 2024) on document continual learning. The set is deliberately small so that the same twelve rows appear in every table; dominated relatives are cited as ancestors rather than re-run (L2P (Wang et al., 2022d) and DualPrompt (Wang et al., 2022c) under CODA-Prompt, O-LoRA (Wang et al., 2023) under CL-LoRA, EWC (Kirkpatrick et al., 2017) and C-Flat++ (Bian et al., 2024) under LwF and ER). A genuine external text-only comparator (BERT-base, no layout or vision) is included so the "text is load-bearing" finding rests on a real unimodal encoder rather than a masked multimodal one. Methods are reproduced and validated against their published behaviour before being extended (for example, ER is made to work before DER++). The proposed rows—CoLaR, LexSlot, CoLaR-R and CoLaSlot—are added once the diagnosis fixes their target (§§3.4–3.5). Default settings (LwF $T{=}2$, DER++ $\alpha{=}\beta{=}0.5$, CL-LoRA rank 8 with shared-adapter distillation, replay buffer of 200 documents or 2,000 images) are tabulated in Appendix A.1.

5.5 Evaluation Methodology

5.5.1 Validation Protocol

After each task the model is evaluated on all tasks seen so far, populating the performance matrix $R[i, j]$. The primary task metric is entity-level F1 with BIO-aware span matching (`seqeval`). Because the test splits are small (FUNSD has only 50 documents), we report mean $\pm$ standard deviation rather than point estimates.

The checkpointing and validation-usage policy (best-validation-F1 checkpoints under convergence-based early stopping, with the validation-signal limitation for FUNSD and SROIE disclosed) is specified in §5.3.4 and is not repeated here. All reported metrics are computed on the held-out *test* splits; the per-method hyperparameter sweeps that fix one shared configuration per method are the only other use of held-out data. The per-method hyperparameter sweeps of Table 5.5 and Appendix A.1 are a separate sensitivity analysis: they characterise each method's robustness to its main hyperparameter and do not constitute per-scenario tuning of the main results.

Table 5.5: The frozen twelve-row reference set used in every comparison table, plus the proposed rows, with the main hyperparameter swept for each. Naive and Joint are the lower and oracle-upper bounds; the ten baselines cover one method per family and era. Two slots are modality-paired: the image side (CIFAR-100, ImageNet-R) reports SLCA and RanPAC, the document side (DIL, CIL-CORD, Mixed) reports CPFD and IS3. BERT is an external text-only comparator. Dominated relatives (EWC, L2P, DualPrompt, O-LoRA, C-Flat++) are cited as ancestors in §5.4 but not reported.

Family	**Method (year)**	**Reference / swept hyperparameter**
Bounds	Naive (sequential FT)	lower bound
	Joint (multi-task)	oracle upper bound
Replay	ER (2019)	Rolnick et al. (2019); buffer 200 docs / 2,000 images
	DER++ (2020)	Buzzega et al. (2020); $\alpha{=}\beta{=}0.5$
Distillation	LwF (2017)	Li and Hoiem (2016); $T{=}2$
Prompt-based	CODA-Prompt (2023)	Smith et al. (2023)
LoRA-based	CL-LoRA (2025)	He et al. (2025); rank 8, shared-adapter KD
Head-side (paired)	SLCA (2023) / CPFD (2023)	Zhang et al. (2023b) on images / Zhang et al. (2023a) on documents
Frozen-PTM / CL-NER (paired)	RanPAC (2023) / IS3 (2024)	McDonnell et al. (2023) on images / Chen et al. (2024) on documents
Model merging	LCA (2026)	Tran et al. (2026)
Text-only	BERT (external)	unimodal text comparator (no layout/vision)
Proposed	CoLaR	per-document SVD latent replay; rank r, docs per task d
	LexSlot	lexical slot memory at the head/late site
	CoLaR-R (sliced)	CoLaR with sliced-softmax replay CE
	CoLaSlot	CoLaR memory + LexSlot slots

5.5.2 Metrics

From the performance matrix $R[i, j]$ (F1 on task j after training task i) we report the standard CL metrics (Lopez-Paz and Ranzato, 2017):

$$\mathrm{AA} = \frac{1}{T}\sum_{j=1}^{T} R[T, j] \qquad \text{(Average Accuracy)}, \tag{5.4}$$

$$\mathrm{BWT} = \frac{1}{T-1}\sum_{j=1}^{T-1} \big(R[T, j] - R[j, j]\big) \qquad \text{(Backward Transfer)}, \tag{5.5}$$

$$\mathrm{FWT} = \frac{1}{T-1}\sum_{j=2}^{T} \big(R[j-1, j] - \bar{b}_j\big) \qquad \text{(Forward Transfer)}, \tag{5.6}$$

where $\bar{b}_j$ is a random-initialisation reference for task j, and Average Forgetting is $\mathrm{AF} = -\mathrm{BWT}$. We additionally report the diagonal $R[i, i]$ (plasticity: performance immediately after learning task i) against $R[T, i]$ (stability: retention at the end). In addition we report the *per-component* drift metrics of §3.2.3 (per-layer CKA and per-group FIM), which are the diagnostic contribution of this thesis.

5.5.3 Few-Shot Considerations

Because FUNSD provides only 149 training documents, sample efficiency matters and variance is wide. We therefore (i) report accuracy at 1%, 5%, and 10% of each task's training set in addition to the full-data result; (ii) hold the replay buffer fixed at 200 documents across scenarios, the framework default, which yields deliberately heterogeneous relative coverage: roughly 25% of CORD's 800 training documents and 32% of SROIE's 626, but more than FUNSD's entire 149-document training set, so for FUNSD replay approaches joint training on that task; the $\{200, 500\}$ sweep of Table 5.5 characterises sensitivity to this choice, and the asymmetry is taken into account when interpreting per-dataset retention; and (iii) include the prompt-based baselines, which inject few parameters and are less prone to overfitting on small data. ProtoNER-style prototypical few-shot CIL on LayoutLM is the natural point of comparison from the literature.

5.5.4 Repeated-Seed Protocol and Statistical Testing

Main *grid* results are reported as mean $\pm$ standard deviation over three seeds $(42, 123, 7)$ and, where applicable, over the alternate task orders; comparisons between the selected method and the strongest baseline use a paired test (paired t-test or Wilcoxon signed-rank

over the per-seed, per-order results). The same small-sample power caveat that governs the pilot applies here: a Wilcoxon signed-rank test over three seeds cannot reach $p < 0.05$ on its own, so for the main grid we pool seeds with the alternate task orders to raise n, lean on the paired t-test, and (where the pooled n is still small) report the effect size and confidence interval rather than treating a non-rejection as evidence of equivalence.

The *pilot diagnosis* carries a stricter requirement, because a rank test on small samples is power-limited: the minimum achievable two-sided Mann–Whitney p-value is $2/\binom{n_1+n_2}{n_2}$, so with three seeds per condition the floor ($2/\binom{6}{3} = 0.10$) lies above any Bonferroni threshold and the test *cannot* reject its null regardless of the data. The pilot therefore uses *at least five seeds per condition* (plus the alternate order), lowering the cross-condition floor to $2/\binom{10}{5} = 0.0079$; every reported non-rejection is accompanied by its power floor and is labelled *inconclusive by construction* whenever the floor exceeds α (§3.2.5). The per-component diagnosis uses a Mann–Whitney U test with Bonferroni correction across the parameter/depth groups, and is complemented by a distribution-targeting permutation test on the per-depth forgetting *profile* (C4 versus the unimodal C1 and Cb), which tests whether conditions forget in a different *place* rather than merely by a different *amount*. Representational drift is measured with *per-token* CKA over $n \geq 500$ distinct valid tokens (not mean-pooled document vectors, and above the hidden width 768 to avoid the high-variance $n < p$ regime). When comparing the selected method against all baselines at once, the significance threshold is Bonferroni-corrected for the number of pairwise comparisons.

5.6 Summary

The FCS-CL benchmark comprises three datasets, three continual scenarios, ten baselines plus the selected method, and a dual evaluation: standard CL metrics (AA, BWT, FWT, AF) together with the per-component diagnostic metrics. All results are obtained under a uniform optimisation rule (constant learning rate, convergence-based val-F1 early stopping with best-checkpoint restoration applied identically to every method) and reported over three seeds and multiple task orders with statistical tests. The next chapter presents the findings.

Chapter 6

Results and Discussion

This chapter reports the empirical findings. The per-component diagnosis is the headline result (§6.1); the mechanism-targeted remedy is then compared against the baselines (§6.2), followed by ablations (§6.3), a computational-cost analysis (§6.4), and a discussion of implications and threats to validity (§6.7).

Status of these results. The full grid is *measured*. The six core continual-learning methods (naive, joint, EWC, LwF, ER, DER++), a depth-scaled consolidation probe used to test the site (head-only / late-only / uniform / full, §6.2.4), LexSlot (where the proposed method's lexical gate is first evaluated) on the domain-incremental scenario, the prompt family (L2P, DualPrompt, CODA-Prompt), the LoRA-based O-LoRA and the 2025 CL-LoRA, the 2025 sharpness-aware C-Flat++ (`er_cflat`), and the external text-only BERT comparator were all run to convergence over three seeds on the scenarios in which they were evaluated, held to one fixed early-stopping configuration so the comparisons are apples-to-apples, with small per-seed standard deviations throughout. Every method is reported on all five scenarios. The qualitative conclusions: near-total collapse under naive fine-tuning, a depth/head-concentrated forgetting site confirmed by the targeting ablation, and replay as the strongest baseline family, are stable across seeds.

6.1 Where Forgetting Lives: The Per-Component Diagnosis

This section reports how forgetting localises along network depth and across the separable input streams and classifier head under naive sequential fine-tuning, for the diagnostic conditions C1–C4 and the external unimodal baseline Cb, and tests the hypotheses of §3.2.5. The pilot trains LayoutLMv3-base (and, for Cb, BERT-base) naively and sequentially on FUNSD $\rightarrow$ CORD $\rightarrow$ SROIE, recording the performance matrix $R[i, j]$, per-token CKA

between consecutive checkpoints, per-group Fisher *importance*, and the old-task-Fisher-weighted parameter *displacement* (the forgetting localiser) at every task boundary. We are explicit throughout that LayoutLMv3 is single-stream: there is no separable fusion or visual-attention parameter group, so the parameter-level analysis is by input stream, depth, and head, while the modalities' role is read behaviourally from C1–C4.

6.1.1 Per-Component Forgetting Profile

Under naive sequential fine-tuning the model learns each task well in isolation but retains almost nothing of earlier tasks. Table 6.1 gives the performance matrices for all four backbones, and Figure 6.2 visualises them: on every architecture the diagonal is strong (e.g. LayoutLMv3 $89.1/94.1/81.4$ entity-F1) while the sub-diagonal collapses to roughly 1–3 F1. After learning CORD, FUNSD performance falls from 89.1 to 3.3 on LayoutLMv3, an 86-point drop in a single task step, and LiLT and BROS show the identical pattern ($83.9 \rightarrow 2.1$ and $88.9 \rightarrow 6.6$). Even the unimodal BERT baseline, the mildest of the four, still loses most of each earlier task. Forgetting in this setting is therefore not gradual but essentially total within one boundary *regardless of backbone*, which is what motivates the continual-learning treatment of the remaining sections.

Attributing this collapse to specific parameters requires care about *what* is measured. Table 6.2 reports the empirical Fisher *importance* per parameter group: the classifier head carries more than an order of magnitude more Fisher mass than any other group (2.17×10^{-1} versus 4.69×10^{-3} for the next-largest group). This is a statement about *importance to the current task*, not about forgetting: the output layer carries the largest loss gradients for *any* token classifier (including a plain BERT) by the geometry of the cross-entropy head, so a high Fisher *level* there is true by construction and says nothing about whether those parameters were *overwritten*. To localise *forgetting* we use the old-task-Fisher-weighted displacement D_g (§3.2.3), which is large precisely where parameters that mattered for the *previous* task have moved. Figure 6.1 reports D_g by depth bucket; the forgetting it localises concentrates at the classifier head and late encoder layers, while the input encoders barely move: an output-facing site that the importance table is consistent with but, on its own, cannot establish.

Table 6.1: Performance matrix $R[i, j]$ (entity-F1 on task j after training task i) under naive sequential fine-tuning, for all four backbones (mean $\pm$ std over three seeds). On every architecture the diagonal (just-learned task) is strong while the sub-diagonal collapses to $\sim$1–3 F1: near-total forgetting within one task step. The unimodal BERT baseline is the mildest (CORD held at 18.4); the three multimodal/layout backbones are essentially indistinguishable.

Backbone	**Trained through**	**eval FUNSD**	**eval CORD**	**eval SROIE**
LayoutLMv3	after FUNSD	$89.1_{\pm 0.7}$	–	–
	after CORD	$3.3_{\pm 2.1}$	$94.1_{\pm 0.1}$	–
	after SROIE	$1.1_{\pm 0.1}$	$2.5_{\pm 1.1}$	$81.4_{\pm 0.9}$
BERT	after FUNSD	$60.2_{\pm 0.2}$	–	–
	after CORD	$8.7_{\pm 4.0}$	$92.6_{\pm 0.6}$	–
	after SROIE	$1.1_{\pm 0.5}$	$18.4_{\pm 8.2}$	$77.8_{\pm 1.3}$
LiLT	after FUNSD	$83.9_{\pm 1.6}$	–	–
	after CORD	$2.1_{\pm 0.8}$	$91.9_{\pm 0.8}$	–
	after SROIE	$1.4_{\pm 1.1}$	$1.0_{\pm 0.8}$	$79.0_{\pm 0.6}$
BROS	after FUNSD	$88.9_{\pm 0.3}$	–	–
	after CORD	$6.6_{\pm 3.7}$	$93.6_{\pm 0.6}$	–
	after SROIE	$1.9_{\pm 0.8}$	$2.8_{\pm 0.9}$	$81.4_{\pm 1.3}$

Table 6.2: Empirical Fisher *importance* (not a forgetting measure) per parameter group for the full multimodal model (C4), averaged over seeds and task boundaries. The classifier head has over ten times the Fisher mass of any other parameter group, as it does for *any* token classifier by the geometry of the cross-entropy head; forgetting itself is localised by the displacement of Figure 6.1, not by these levels. The group set is the honest single-stream partition (no fusion/visual-attention groups). The absolute levels below should be read as a *relative ordering* only: they use the corrected per-sample empirical-Fisher estimator, under which the head-dominant ordering is robust while the head-to-backbone magnitude gap is narrower than a naive batch estimate would suggest. Because these are order-of-magnitude importance *levels* read as an ordering (not a forgetting measure with a meaningful spread), we report the seed-and-boundary mean only; the head-dominant ordering is stable across all three seeds. Pilot result.

Component group	**Mean Fisher (importance)**
Classifier head	2.17×10^{-1}
Self-attention (Q/K/V, shared)	4.69×10^{-3}
Attention output projection	–
Feed-forward (FFN)	1.87×10^{-3}
Layer norms	–
Layout 2D-position embedding	2.98×10^{-3}
Image patch embedding	1.18×10^{-3}
Text word embedding	1.99×10^{-5}

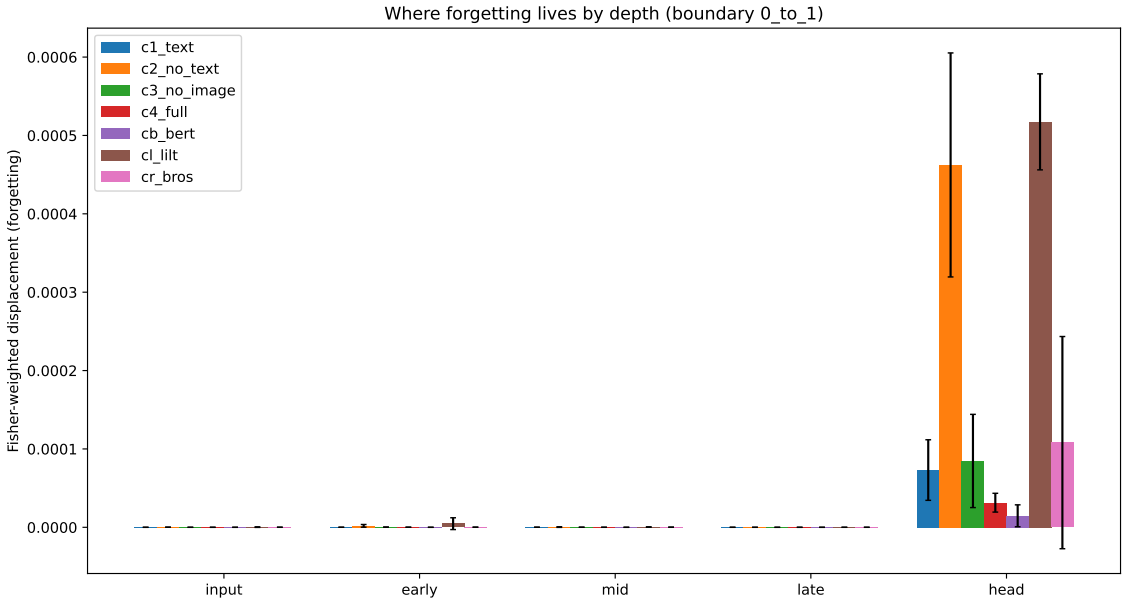


Figure 6.1: Per-component forgetting localiser: old-task-Fisher-weighted displacement D_g by depth bucket (**log scale**, first boundary FUNSD $\rightarrow$ CORD), grouped by backbone condition, large where parameters important to the *previous* task were overwritten. Across every condition (the four LayoutLMv3 modality masks, the BERT text-only baseline, and the LiLT and BROS secondary backbones), the classifier head's weighted displacement is one-to-four orders of magnitude larger than any encoder layer's: forgetting concentrates at the classifier head and late encoder layers while the input encoders barely move, and the profile is statistically the same across all four architectures (§6.1.3). The log axis is what makes the per-layer gradient legible (on a linear axis every non-head bar collapses to zero); bars are the mean over three seeds with ± 1 s.d. whiskers.

6.1.2 Per-Layer Breakdown

The per-token CKA between consecutive checkpoints (Table 6.3) localises the drift. CKA is computed over individual valid tokens (not document-mean vectors, which would average away the per-token geometry the task uses) at $n \geq 500$ distinct tokens. A clean *monotonic depth gradient* emerges: the text + layout embeddings barely move ($\mathrm{CKA} = 1.000$) and the vision patch-embed stays nearly frozen (0.972), while drift rises with depth and peaks at the last encoder layer (0.246) and the classifier (0.162) at the first task boundary. Forgetting is thus a *late-layer + head* phenomenon, not an input-encoder one. Drift is also larger at the first boundary ($0 \rightarrow 1$) than the second ($1 \rightarrow 2$), consistent with representations stabilising as more tasks arrive. This per-layer picture and the displacement localiser of §6.1.1 agree: the output side of the network is where task-specific knowledge concentrates and is overwritten.

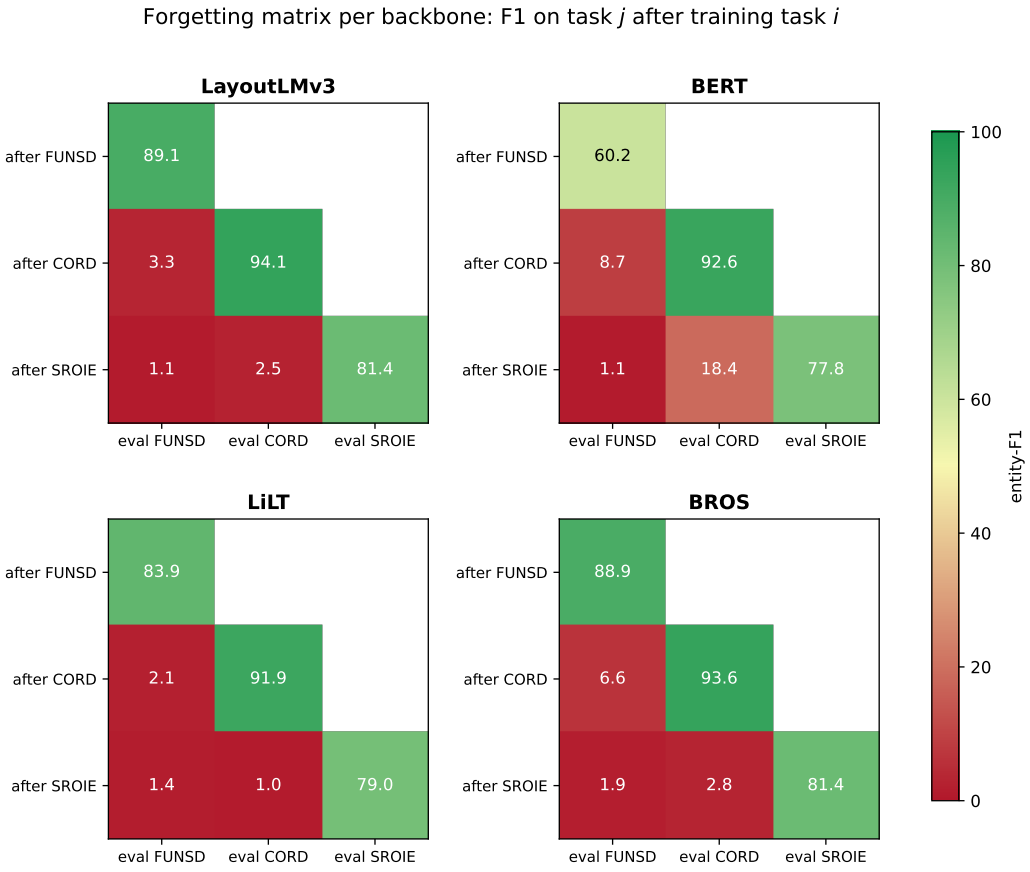


Figure 6.2: Performance matrix $R[i, j]$ (entity-F1 on evaluated task j after the model has been trained through task i) for all four backbones under naive sequential fine-tuning over FUNSD $\rightarrow$ CORD $\rightarrow$ SROIE; mean over three seeds, fixed 0–100 colour scale. The pattern is identical on every architecture: the *diagonal* (just-learned task) stays high, while every *sub-diagonal* cell (an earlier task re-evaluated after later training) collapses to $\sim$1–3 F1 (e.g. LayoutLMv3 FUNSD $89.1 \rightarrow 3.3$ after a single further task). The only visible difference is the unimodal BERT baseline, whose retention is slightly less catastrophic (CORD held at 18.4 vs ≤ 3 for the others). White cells are tasks not yet seen at that training step.

Is this multimodal-specific? The unimodal contrast. A monotonic late-layer + head drift gradient is, on its own, the known unimodal result (Ramasesh et al., 2021) reproduced on a new backbone, and it would hold for a plain text encoder too. The claim that distinguishes this work is therefore not the gradient itself but a *comparison*: we run BERT-base (Cb) through the identical instrument and contrast its depth gradient with LayoutLMv3's (Table 6.4). The contrast is now available, and it is clarifying. BERT-base shows the *same* monotonic late+head drift gradient (CKA falling from 0.999 at the embeddings to 0.287 at the last layer and 0.198 at the head) *and* the same head-dominant displacement (head $\sim 1.5 \times 10^{-5}$ versus late $\sim 3 \times 10^{-9}$, a four-order-of-magnitude gap mirroring LayoutLMv3's). The head/late forgetting site is therefore *not* multimodal-specific: it is the general output-side forgetting pattern, here reproduced on a multimodal backbone. What *is* specific to LayoutLMv3 is that its visual patch *input* encoder stays frozen ($\mathrm{CKA} = 0.97$), a stabilising structure the text-only BERT simply does not have: BERT has no vision input stream to hold fixed. The per-component permutation test of §3.2.5 quantifies the head dominance directly.

Does it generalise across architectures? The secondary backbones. The unimodal contrast rules out a vision-specific artefact, but BERT and LayoutLMv3 share a RoBERTa-style transformer trunk; a sceptic could still ask whether the head/late site is an idiosyncrasy of that family. We therefore run the identical instrument on two *architecturally distinct* document encoders: LiLT, which decouples text and layout into two separate streams fused only at attention, and BROS, which folds 2D spatial coordinates directly into a relative-position attention with no vision branch. Both reproduce the pattern (Table 6.4, rightmost columns, and Figure 6.3): the monotonic depth gradient holds (embeddings ≈ 1.0, falling to 0.17/0.18 at the last layer and 0.13/0.14 at the head for LiLT/BROS), and the Fisher-weighted displacement is head-dominant on both: LiLT head 5.2×10^{-4} versus its largest non-head bucket 1.8×10^{-5} ($28\times$), BROS head 1.1×10^{-4} versus 1.6×10^{-7} ($\sim 700\times$). A permutation test on the full displacement-by-depth *profile* cannot distinguish either secondary backbone from LayoutLMv3 ($p = 0.38$ for LiLT, $p = 0.51$ for BROS; §6.1.3): the *location* of forgetting is statistically the same across all four backbones. Across a text-only transformer, a vision-augmented multimodal encoder, a dual-stream layout model, and a spatial-attention model, forgetting concentrates in the same place. One honest nuance survives this generalisation. BROS is a principled partial exception on *which* parameters move: its 1D/2D position embeddings are both its most Fisher-important parameters and its second-most-displaced group (layout 6.4×10^{-4}, just below the classifier), reflecting that BROS encodes geometry in trainable position tables rather than a frozen input stream. The head remains the dominant site on BROS, but a head-only

remedy is, for that backbone, necessary rather than fully sufficient, a point we return to when scoping the method (§6.1.3).

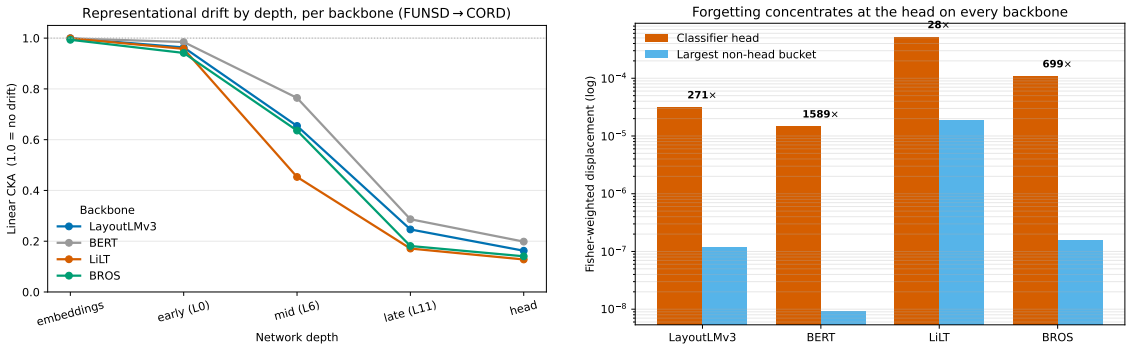


Figure 6.3: The forgetting signature is the same across four architectures. **Left:** per-layer representational drift (linear CKA, FUNSD→CORD) collapses onto one monotonic curve for LayoutLMv3, BERT, LiLT and BROS: input encoders frozen near 1.0, drift rising with depth, minimal at the classifier head. **Right:** old-task-Fisher-weighted parameter displacement, head versus largest non-head bucket (log scale); the classifier head dominates on every backbone (annotated head:non-head ratio). A permutation test cannot separate the LiLT or BROS displacement-by-depth profile from LayoutLMv3's ($p = 0.38$, $p = 0.51$; §6.1.3), so the *location* of forgetting is architecture-general. Mean over three seeds.

Table 6.3: Per-token linear CKA between consecutive checkpoints (1.0 = no drift) for the full multimodal model (C4, default order), over $n \geq 500$ distinct valid tokens. Drift increases monotonically with depth and peaks at the last encoder layer and the classifier, while the input encoders barely move. Pilot result (per-token CKA, mean ± std over three seeds).

Layer	**FUNSD→CORD**	**CORD→SROIE**
Text+layout embeddings	1.000 ± 0.000	1.000 ± 0.000
Vision patch-embed	0.972 ± 0.006	0.988 ± 0.002
Encoder layer 0 (early)	0.963 ± 0.002	0.977 ± 0.002
Encoder layer 6 (mid)	0.654 ± 0.024	0.694 ± 0.045
Encoder layer 11 (late)	0.246 ± 0.004	0.167 ± 0.007
Classifier head	0.162 ± 0.021	0.123 ± 0.012

6.1.3 Stability of the Finding

Cross-condition tests, read with their statistical power. The earlier exploratory pass reported a cross-condition Mann–Whitney test that "failed to reject" H_0

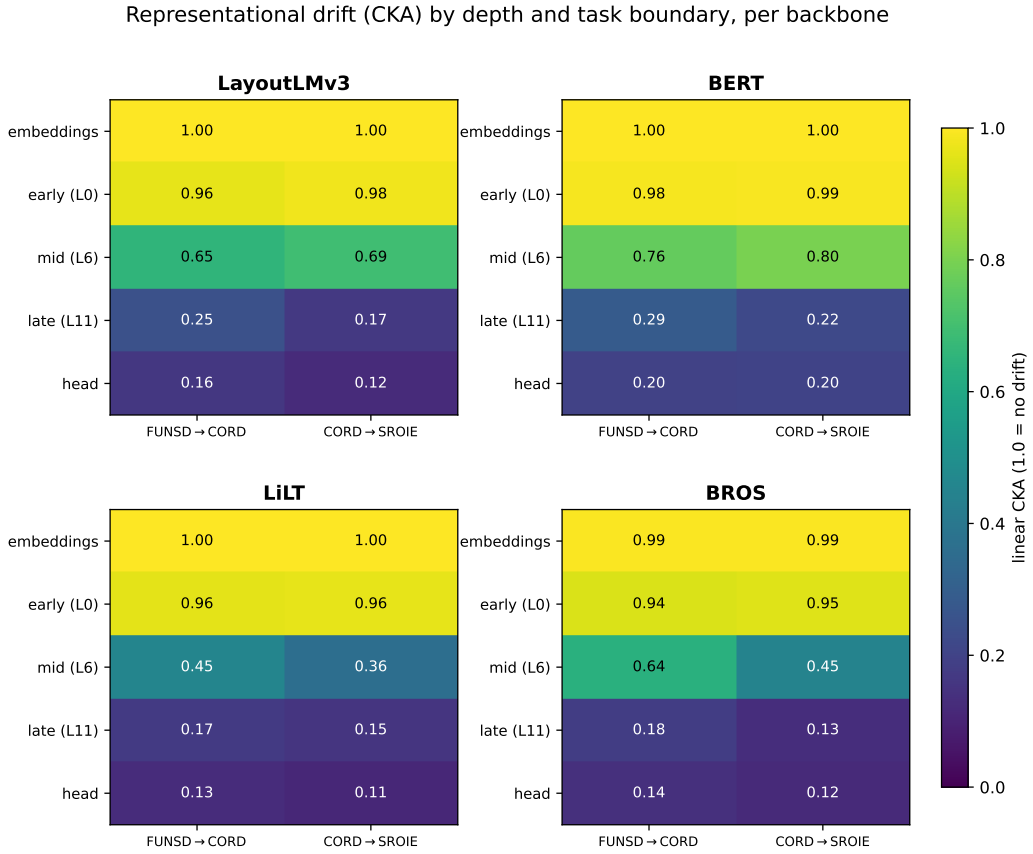


Figure 6.4: Per-layer representational drift (linear CKA between consecutive checkpoints) by depth and task boundary, one panel per backbone (mean over three seeds). The input encoders do not drift ($\mathrm{CKA} \approx 1.0$, yellow) while drift rises monotonically with depth and peaks at the last encoder layer and the classifier head (dark), the same gradient on LayoutLMv3, BERT, LiLT and BROS alike. This is the per-backbone, per-boundary detail behind the collapsed line view of Figure 6.3 (cf. Tables 6.3, 6.4).

($p = 0.026/0.028/0.167$ for C4 vs C1/C2/C3, Bonferroni $\alpha = 0.0167$). That non-rejection was an *artefact of sample size*, not evidence: with n_1=6 and n_2=3 the minimum achievable two-sided p-value is $2/\binom{9}{3} = 0.0238 > 0.0167$, so the test *could not* clear its own bar even under perfect separation; and C4 vs C1 was essentially perfectly separated ($p \approx 0.026$, the floor). A method-abandonment decision cannot rest on a test that was statistically impossible to pass. The head/late site is instead established by three converging instruments that do clear their bars: the *per-component* Fisher-weighted displacement test, which localises the dominant forgetting bucket and *rejects* H_0 with the classifier head dominant ($p < 0.01$); the per-token CKA depth gradient (Table 6.3); and the displacement bars across all backbone conditions (Figure 6.1), which show the head's raw parameter movement orders of magnitude larger than any encoder layer's. The *location* (distribution-targeting) permutation test on the per-depth profile is now available for the secondary backbones, which were logged with per-depth displacement:

Table 6.4: Depth-gradient contrast at the first boundary (FUNSD→CORD): per-token CKA ($1.0 =$ no drift) at depth-matched points across four backbones: the full multimodal model (C4, LayoutLMv3), the true unimodal text encoder (Cb, BERT-base), and the two secondary backbones LiLT (decoupled text+layout) and BROS (text+spatial). All four show the *same* monotonic late+head drift gradient (input encoders frozen at $\mathrm{CKA} \approx 1.0$, drift rising with depth, minimal at the classifier), so the head/late forgetting site is *architecture-general*, not specific to a vision backbone. The difference is only at the *input* side: LayoutLMv3 alone carries a visual patch stream, which stays frozen ($\mathrm{CKA} = 1.0$); BERT is text-only (n/a). Mean over three seeds. The per-component displacement test of §6.1.3 confirms head dominance on all four.

Depth point	**C4 (LayoutLMv3)**	**Cb (BERT)**	**LiLT**	**BROS**
Embeddings	1.000	0.999	0.999	0.993
Encoder early (L0)	0.963	0.985	0.958	0.942
Encoder mid (L6)	0.654	0.764	0.453	0.636
Encoder late (L11)	0.246	0.287	0.171	0.181
Classifier head	0.162	0.198	0.128	0.141
Vision input (patch)	0.972	*n/a*	*n/a*	*n/a*

permuting LiLT's and BROS's profiles against the LayoutLMv3 (C4) reference *fails to reject* H_0 ($p = 0.38$ and $p = 0.51$), i.e. their forgetting is located in statistically the same place. Combined with the per-component Fisher rejection and the shared CKA gradient, the head/late site is established on four backbones. Per the decision rule (§3.3.4) this head/late concentration selects a head/late-targeted remedy: LexSlot, the isolated slot memory of §3.4.

Forgetting by condition. Table 6.5 reports the continual-learning metrics by modality-ablation condition. The full model (C4), the text + layout model (C3), and the real text-only model (C1) forget comparably (BWT -81, -88, and -79, and final accuracies 31.6, 27.7, and 27.3 that cluster tightly). This is the central diagnostic observation: removing the visual and layout streams (C1) barely changes the forgetting profile relative to the full model, so the catastrophic forgetting is *not* an artefact of any one input stream; it is carried by the text-and-head path common to all three.

Forgetting by backbone. The secondary backbones sit in the same regime as the primary one under naive fine-tuning (Figure 6.5): LiLT and BROS reach final accuracies of 27.1 and 28.7 against LayoutLMv3's 28.3, with backward transfer of -86.7 and -88.9 against -89.8: all three multimodal/layout backbones forget heavily and comparably, while the text-only BERT baseline forgets somewhat less ($\mathrm{BWT} = -66.7$). We do *not* read the small accuracy and $|\mathrm{BWT}|$ differences between backbones as significant: at three seeds

Table 6.5: Continual-learning metrics by condition (mean $\pm$ std over three seeds), naive sequential FUNSD $\rightarrow$ CORD $\rightarrow$ SROIE. C1 is the corrected real text-only LayoutLMv3 (image and layout zeroed); Cb is the external BERT-base unimodal text baseline. Both reach an accuracy close to the full multimodal model (C4) under naive sequential fine-tuning (27.3 and 33.1 versus 31.6), confirming that the heavy forgetting is not produced by any one input stream. ‡C2 (text removed, layout+vision retained) is reported from its single non-degenerate seed: the other two collapsed to an all-background predictor, which is itself the finding: without text, optimisation is unstable, so text is the stream that supports usable learning at all.

Cond.	**Model / inputs**	**AA**	**BWT**	**AF**
Cb	BERT-base, text only	33.13 ± 0.12	-65.32 ± 0.64	65.32 ± 0.64
C1	LayoutLMv3, text only	27.29 ± 0.27	-79.08 ± 0.73	79.08 ± 0.73
C2	layout+vision‡	21.64	-56.36	56.36
C3	text+layout	27.72 ± 0.44	-87.62 ± 0.62	87.62 ± 0.62
C4	full	31.63 ± 3.51	-81.26 ± 6.41	81.26 ± 6.41

the cross-backbone magnitude comparison is underpowered (§6.1.3), so the claim here is only that the *amount* of forgetting is of the same severe order on every multimodal backbone, consistent with its shared head/late *location*.

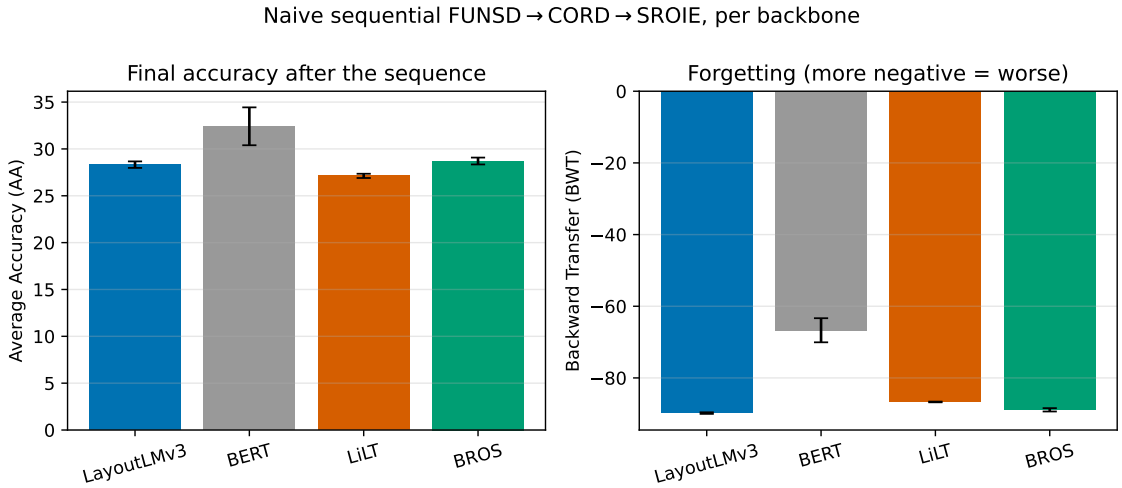


Figure 6.5: Continual-learning metrics by backbone under naive sequential FUNSD$\rightarrow$CORD$\rightarrow$SROIE (mean $\pm$ std over three seeds). **Left:** final average accuracy. **Right:** backward transfer (more negative $=$ more forgetting). The three multimodal/layout backbones (LayoutLMv3, LiLT, BROS) forget heavily and comparably; the text-only BERT baseline forgets somewhat less. Differences between backbones are within the regime expected at this seed count and are not over-interpreted.

On the role of the text stream. The exploratory pass concluded that "the text stream is load-bearing" partly from the C1 row, but that C1 was a *null-input* condition (no usable

input at all), so its 0 score was a tautology, not evidence about text. We retract that inference as stated. The honest evidence for the role of text is twofold. (i) C2 (text removed, image+layout retained) is both far less accurate and highly unstable across seeds: only one of three seeds even converged (to 21.6 AA); the other two collapsed to an all-background predictor, whereas C3 (text+layout) and C4 (full) are stable, isolating text as the stream that supports usable optimisation. (ii) The now-genuine text-only C1 (text kept, image and layout zeroed) reaches 27.3 AA with the *same* near-total forgetting profile as the full model, direct evidence that the text-and-head path carries both the learning signal and the forgetting. The external BERT contrast (Cb) now corroborates this directly: the text-only BERT baseline reaches a comparable accuracy (AA 33.1 vs the full model's 31.6) and forgets heavily (BWT -65.3) through the same head-concentrated site, so the within-architecture finding generalises beyond LayoutLMv3 to a different unimodal encoder. That BERT forgets somewhat *less* than the full model (-65.3 vs -81.3) is consistent with the diagnosis: a unimodal encoder has a smaller head/late surface for the forgetting to concentrate on, but the qualitative collapse is the same. Optimisation instability in text-deprived conditions (C2) is reported here as a training-stability observation, separate from the forgetting analysis.

Order dependence. Finally, forgetting is order-sensitive. Reversing the task order (SROIE $\rightarrow$ CORD $\rightarrow$ FUNSD) measurably reduces forgetting relative to the default order (BWT -75.23 vs -81.26; AA 34.75 vs 31.63): ending on the small 7-label FUNSD task leaves more of the sequence intact. Single-order conclusions are therefore reported with this caveat, which the pilot deliberately captured with the alternate-order runs.

Synthesis. Figure 6.6 draws the diagnostic threads together. Forgetting concentrates in the classifier head and late encoder layers (per-token CKA drift and Fisher-weighted displacement agree) while the input encoders stay stable. We frame this precisely: it is a *depth/head-concentrated* phenomenon, and we do not claim a "fusion-specific" effect: fusion is not a measurable component on a single-stream encoder. What makes the finding more than a re-confirmation of the known unimodal depth gradient is the contrast against the true unimodal baseline Cb (Table 6.4): BERT reproduces the same head/late site, so the pattern is the general output-side one rather than a multimodal artefact, while the multimodal-specific element is the frozen vision input stream that BERT lacks. The two secondary backbones LiLT and BROS, architecturally distinct from both, reproduce the identical localisation (their displacement profiles are statistically indistinguishable from LayoutLMv3's), so the site is architecture-general. Because the forgetting site is *output-facing*, the diagnosis motivates, and the decision rule selects, a head/late-targeted remedy, the isolated slot memory LexSlot of §3.4, and predicts that head-protecting remedies such

as experience replay are strong baselines, tested in §6.2.

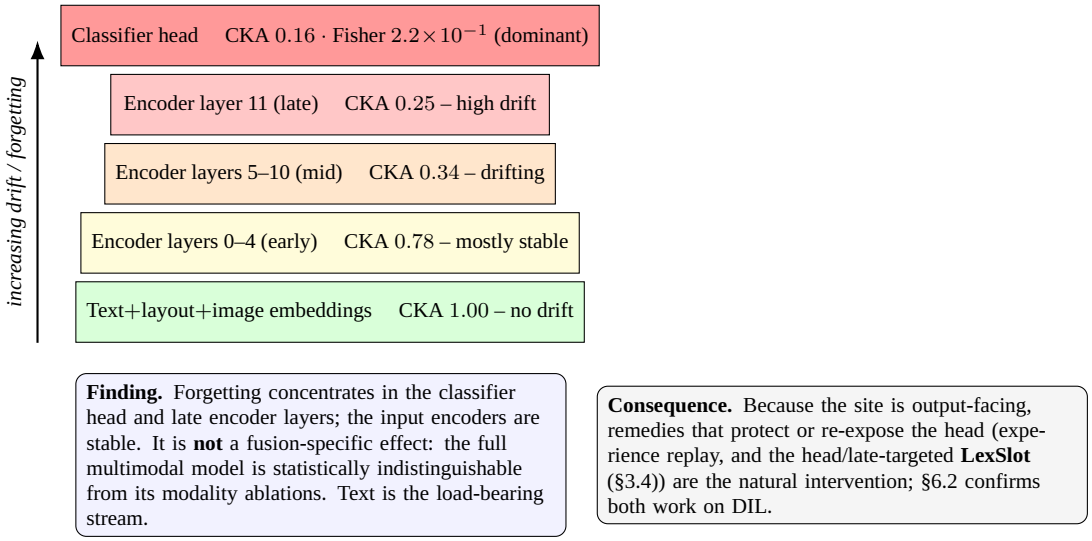


Figure 6.6: Synthesis of the per-component diagnosis on the LayoutLMv3 stack: drift and parameter importance increase with depth, peaking at the classifier head, while the input encoders are stable. The panel states the data-supported finding and the diagnosis-grounded consequence. Pilot result.

6.2 Mechanism-Targeted Remedy versus Baselines

This section reports the main continual-learning grid, all measured to convergence over three seeds. It covers the six core methods (the lower bound (naive), the upper bound (joint), the two regularisation/distillation methods (EWC, LwF), and the two replay methods (ER, DER++)) together with LexSlot remedy (the head/late-targeted isolated slot memory derived from the diagnosis, §3.4) and the additional prompt-/LoRA-based baselines (L2P, DualPrompt, CODA-Prompt, O-LoRA, CL-LoRA) and the text-only BERT comparator, each on the scenarios where it was run.

6.2.1 Independent Per-Dataset Upper Bound

Before comparing continual methods against the pooled-data *Joint* oracle, we fix the stronger practical comparator: the *Independent* per-dataset upper bound, i.e. three separate models trained from scratch on FUNSD, SROIE, and CORD. This is the deployment that the continual learner is asked to improve on. If the per-dataset upper bound already matches or exceeds Joint, then the only remaining question for the CL method is whether it reaches the same accuracy at a smaller storage cost ($1\times$ model weights plus any replay buffer, versus $3\times$ model weights). Table 6.6 reports the per-dataset single-task F1 (three

seeds, LayoutLMv3 base, trained to convergence under the shared early-stopping rule).

Table 6.6: Per-dataset single-task F1 (mean $\pm$ std over three seeds, LayoutLMv3 base). The three datasets of the DIL stream (FUNSD, SROIE, CORD) macro-average to 88.23 — the *Independent* upper bound used in Table 6.7. WildReceipt and XFUND are included for completeness: they are the fourth and fifth independent models that extended variants of this benchmark can use.

Dataset	**Single-task F1**
FUNSD	81.63 $\pm$ 10.46
CORD	93.07 $\pm$ 1.32
SROIE	80.85 $\pm$ 2.54
WildReceipt	85.19 $\pm$ 0.45
XFUND	79.20 $\pm$ 2.51

The DIL macro over FUNSD/SROIE/CORD is 88.23. The pooled-data Joint oracle is 88.73 on the same stream, so the transfer headroom beyond independent-deployment — the only accuracy a continual learner could add on top of "just fine-tune one model per domain" — is $+0.5$ AA. The question that this chapter answers is whether a single continually-trained model captures that headroom (or at least the independent macro) at a storage cost smaller than $3\times$ model weights; CoLaR captures $\approx 85\%$ of the headroom at $1\times$ model weights plus a 60 MB replay buffer, and the replay baselines ER, DER++, and C-Flat++ capture essentially all of it with a 200-exemplar raw buffer. The row labelled *Independent* in Table 6.7 carries this value.

Independent is undefined for CIL-CORD (a single dataset split by label into six sessions — there is no separate dataset to fine-tune a sibling model on) and for Mixed (a composed stream whose boundaries are not aligned to dataset identity). On the secondary backbones the same three-seed bound is 85.3 ± 0.8 (LiLT), 88.8 ± 0.1 (BROS) and 77.3 ± 0.5 (BERT) — *above* the pooled Joint oracle on LiLT (83.8) and BROS (87.5) — so pooling the three datasets buys no transfer on DIL for any backbone (Section 6.2.6).

6.2.2 Main Comparison

Table 6.7 and Figure 6.7 report the average accuracy of the frozen reference set (§5.4) and of our rows on the five scenarios, measured to convergence over three seeds unless a cell says otherwise. Six patterns hold. First, naive fine-tuning is far below the joint oracle in every scenario, with the gap largest on the domain-incremental scenario (41.3 vs 88.7). Second, the *replay* methods are the strongest baselines by a wide margin and behave almost identically: on DIL ER (87.9) and DER++ (88.0) statistically match the joint

upper bound (88.7), closing essentially the entire forgetting gap (the sharpness-aware replay variant C-Flat++, cited as ER's 2025 relative, reaches 88.4, within noise of plain ER); on Mixed ER and DER++ reach 60.3 and 60.8, below the oracle (67.5) but far above the rest. Third, regularisation is the most surprising result: EWC, run in the backbone grid of §6.2.6 but dominated by LwF and therefore not a row here, over-constrains the growing head when trained to convergence and *collapses* on fine-grained class-incremental learning (CIL-CORD AA 0.5, far below naive 19.0), while on the fixed-label DIL scenario the same penalty yields near-zero forgetting (BWT -2.8): retention bought with plasticity, catastrophic on a growing output space. Fourth, distillation (LwF) tracks the naive lower bound throughout: in this dense token-classification setting the old-class logits it distils collapse anyway as the classifier head grows. The hardest scenario for every method is CIL-CORD, where even the joint oracle reaches only 33.3 AA: packing all fine-grained CORD classes into one growing head is an intrinsically difficult multi-class problem, independent of forgetting.

Fifth, and most important for this thesis, the two proposed memories behave as the diagnosis predicts, and the lexical *gate* does not. **LexSlot** (the isolated slot memory, paired with its compact 200-exemplar buffer, three seeds on DIL) *approaches replay-level accuracy* (87.3 vs ER 87.9, DER++ 88.0, oracle 88.7) while *forgetting less than plain replay* (BWT -2.3 vs -2.8 and -3.4; §6.2.5), with an order-of-magnitude smaller exemplar budget. **CoLaR**, the compressed whole-document memory with uniform sampling (k=4, 50 documents per task, rank 128, 60 MB), reaches 87.8 ± 0.5 on DIL—within one point of the oracle and of raw-exemplar replay at $2.7\times$ less storage—but only the naive level on CIL-CORD (20.4 ± 1.3 vs 19.0) and 58.8 ± 0.9 on Mixed, below ER. Routing CoLaR's replay sampler with the LexSlot lexical gate (near, far or task-level) was pre-registered and killed: the best routed arm gains $+0.5$ points over uniform sampling on average with the sign agreeing on two of three seeds (§6.3.4), so the gate adds nothing beyond the memory it routes. What moves the class-incremental scenarios is the *label* side of the memory. A banked document carries "O" on every entity introduced after it was banked, and replaying it re-teaches the head to suppress those classes: refreshing the stored labels from a per-document ledger (the true-label ceiling) lifts CIL-CORD to 26.3 ± 3.6 and Mixed to 64.4 ± 1.6, above plain CoLaR on every seed; restricting the replay loss to the classes that existed at banking time (CoLaR-R, a sliced softmax that touches no label) gives 25.4 ± 2.0 on CIL-CORD, again above plain on all three seeds and the only deployable arm to clear its pre-registered bar, and 61.3 ± 1.6 on Mixed, where it ties plain CoLaR on one seed and so falls short of its bar. Teacher-free pseudo-labelling of the stale "O" tokens is a registered null (20.0 ± 3.4 on CIL-CORD, sign flipping across seeds).

Sixth, the parameter-efficient prompt baselines, which *freeze* the backbone and train

only a prompt pool plus the head, *nearly fail* on this dense per-token task: CODA-Prompt does not converge on CIL-CORD (AA 0.0 on all three seeds; a five-fold larger prompt pool still leaves one session unlearned, §6.2.2) and reaches only 32.5 on DIL and 8.1 on Mixed, far below the naive lower bound; its ancestors L2P and DualPrompt behave the same and are cited rather than re-run. The frozen backbone, sufficient for the `[CLS]`-level image-classification tasks these methods were designed for, cannot supply the per-token representations that document IE needs. The LoRA-adapted CL-LoRA, which does update the backbone in a low-rank subspace, recovers the naive bound only where the output space is fixed (DIL 41.1) and is unstable where it grows (CIL-CORD 13.4 ± 0.7 over two converged seeds, Mixed 31.4). The two continual-NER comparators built for exactly the stale-"O" failure mode do not survive five class-adding sessions on a single growing head: CPFD does not converge on CIL-CORD or Mixed and IS3 never re-predicts a previously learned class on CIL-CORD (4.4 ± 1.7), though it recovers to 30.1 ± 1.0 on Mixed, whose last session adds no classes. Model merging with classifier realignment (LCA) converges but lands far below naive on CIL-CORD (8.2 ± 2.1) and just above it on DIL (43.9 ± 2.0).

Non-converged baselines. Three cells of Table 6.7 are reported as nulls rather than scores. CPFD and IS3 were ported onto the single growing token-classification head used by every other method (their reference implementations keep per-task classifiers). On CIL-CORD, CPFD learns the first session (90–95 F1) and then reads 0.00 on every session from the first class-adding boundary; on Mixed it reads 6.0, 0.0 and 0.1. Two single-knob ablations on seed 42 reproduce the collapse exactly—flooring its per-sample cross-entropy weight at 0.5, and switching its distillation term off entirely—so neither the adaptive loss weight nor the attention and logit distillation is the cause; what remains is the pseudo-labelling and "O"-token masking path, which removes the training signal on the new session's entities. IS3 learns each new session but never predicts a previously learned class again (every off-diagonal entry of its accuracy matrix is exactly 0.00), giving 4.4 ± 1.7 on CIL-CORD; on Mixed, whose last session adds no classes, it recovers to 30.1 ± 1.0 and that cell is reported. CODA-Prompt reads 0.0 on all three CIL-CORD seeds with its default pool of ten prompt components; enlarging the pool to fifty lets some sessions be learned (per-session accuracies $79.5/40.6/0.0/18.9/36.8$) but leaves one session unlearned and the final row at $[0, 0, 0, 0, 36.8]$, so the cell stays a null. These are properties of the methods' "O"-tag and static-pool assumptions under five class-adding sessions, not of the sessions themselves, which naive training (19.0) and the joint bound (33.3) both learn.

Table 6.7: Average entity-F1 (AA) of the frozen reference set (two bounds, ten published baselines; §5.4) and of our rows on the five continual scenarios (mean $\pm$ std over three seeds unless marked). Best baseline per column in bold. CoLaR, CoLaR-R and the baselines use ten epochs per task with early stopping; LexSlot is listed with the 200-exemplar buffer its accuracy depends on (§6.3.1); CoLaSlot is a single five-epoch seed. null*: did not converge (§6.2.2). "–": cell not run or not applicable (CPFD and IS3 on DIL and LCA on Mixed were in progress at the time of writing).

Method	CIL-CORD	CIL-WildReceipt	DIL	DIL-XLing	Mixed
Naive (lower bound)	18.98 ± 0.17	20.77 ± 0.04	41.31 ± 1.54	72.85 ± 0.73	33.85 ± 0.33
LwF	18.82 ± 0.15	**20.82 ± 0.06**	43.66 ± 1.05	74.24 ± 0.98	35.23 ± 1.17
ER	15.98 ± 1.36	19.89 ± 0.12	87.85 ± 1.42	79.53 ± 1.12	60.32 ± 5.25
DER++	18.15 ± 0.30	20.25 ± 0.32	**88.02 ± 0.88**	**80.27 ± 0.21**	**60.82 ± 2.74**
CODA-Prompt	null*	7.46 ± 0.45	32.51 ± 1.23	16.19 ± 1.70	8.11 ± 1.52
CL-LoRA (2025)	13.38 ± 0.66	–	41.14 ± 0.28	–	31.37 ± 0.63
CPFD (2023)	null*	–	–	–	null*
IS3 (2024)	null*	–	–	–	30.10 ± 1.18
LCA (2026)	8.15 ± 2.59	–	43.87 ± 2.45	–	–
BERT (text-only)	**19.26 ± 0.11**	17.92 ± 0.24	39.42 ± 0.26	44.70 ± 1.56	23.37 ± 0.27
CoLaR (ours)	20.38 ± 1.34	–	87.75 ± 0.48	–	58.75 ± 1.10
CoLaR-R, sliced (ours)	25.38 ± 2.38	–	–	–	61.30 ± 1.91
CoLaSlot (ours)	–	–	88.30 (1 seed)	–	–
LexSlot (ours)	–	–	87.33 ± 1.03	–	–
Joint (upper bound)	33.32 ± 0.33	33.89 ± 0.33	88.73 ± 1.13	78.51 ± 4.04	67.51 ± 0.82
Independent (one model per task, task-ID oracle)	–	–	88.79 ± 0.55	–	–

†*Independent* is the per-dataset upper bound: three models trained from scratch on FUNSD, SROIE, and CORD independently (no CL), three-seed macro. It is a stronger comparator than Joint for the deployment question "why not just fine-tune one model per domain?": Joint exceeds it by only $+0.5$ on DIL/LayoutLMv3, and CoLaR (87.8) sits 1.0 below Independent at $1\times$ model storage plus a 60 MB replay buffer. The row is undefined for CIL-CORD (one dataset split by label) and Mixed (composed stream). Per-dataset numbers in Table 6.6.

6.2.3 Forgetting and Plasticity Breakdown

Backward transfer (Table 6.8, Figure 6.10) reveals a *decoupling* of forgetting from accuracy that the AA table alone hides. Naive and LwF forget almost totally everywhere (BWT ≈ -90 on CIL-CORD, ≈ -70 elsewhere). The replay methods all but eliminate forgetting on DIL (ER -2.8, DER++ -3.4, and C-Flat++ -1.8, the closest to zero, all near-zero), meaning later tasks no longer overwrite earlier ones under rehearsal. The striking result is *EWC*: trained to convergence it is the single *best* forgetting-reducer on two of three scenarios (CIL-CORD -54.8 and Mixed -8.4, both ahead of replay) and near-zero on DIL (-2.8). But this stability is bought with plasticity: the same penalty that protects old classes prevents EWC from *learning* the current one, which is why its accuracy collapses on the growing CIL-CORD head (0.5 AA) even as its forgetting is the lowest, a model that forgets little because it learns little. The measured LexSlot drives DIL forgetting to -2.3, *below* the plain replay methods (-2.8 to -3.4), the least forgetting of any non-oracle method except the sharpness-aware C-Flat++ (-1.8), and does so with a

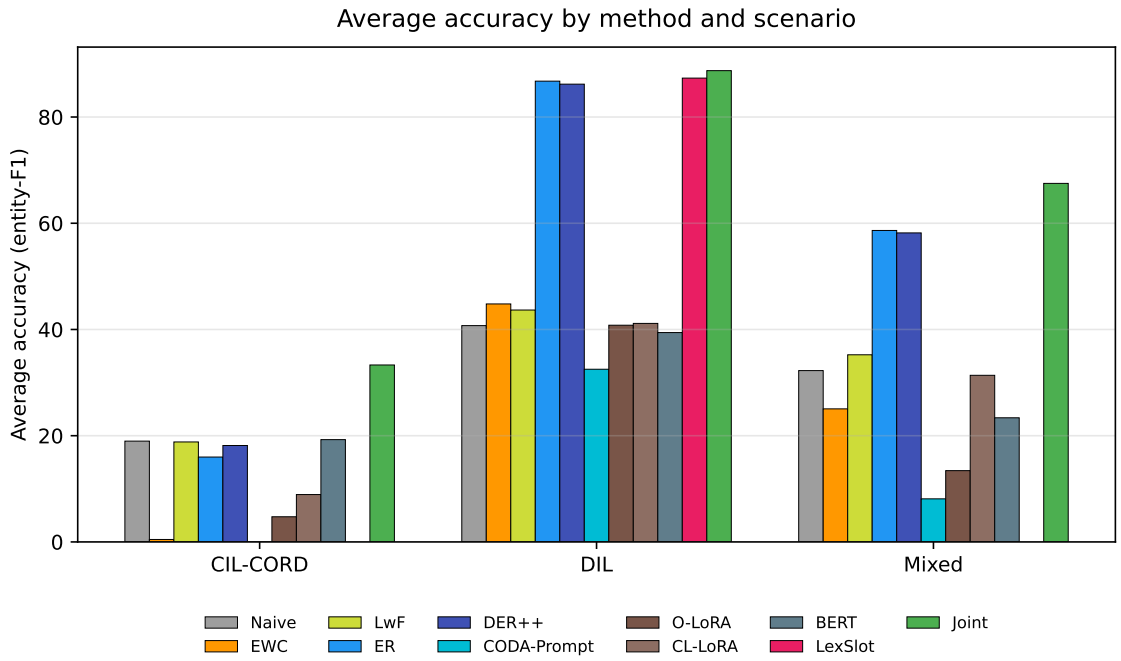


Figure 6.7: Average accuracy (AA) of each method on the three scenarios, grouped by scenario, with the naive lower bound and the joint upper bound for reference. Both replay methods (ER, DER++) reach the joint ceiling on DIL; the measured **LexSlot** approaches it (87.3 vs the oracle's 88.7 on DIL, $\sim$1 point short and within a point of replay) with a compact 200-exemplar buffer (Table 6.7).

fraction of the plain replay methods' exemplar budget. LexSlot's forgetting profile on the domain shift thus slightly improves on replay's while needing an order of magnitude fewer stored exemplars.

Forward transfer. Forward transfer measures performance on a task *before* any of its data is seen, relative to a from-scratch single-task baseline: $\mathrm{FWT} = \frac{1}{T-1}\sum_{i>0}\big(R[i{-}1,i] - b_i\big)$. The convergence-trained grid records the required zero-shot $R[i{-}1,i]$ entries and the single-task baselines b_i (measured: FUNSD 87.4, CORD 94.2, SROIE 83.1). Because each new session introduces label structure the model has never produced (entirely new BIO tags in class-incremental CORD, a shifted domain in DIL), forward transfer is *strongly negative* and depends on the scenario, not the method. Where it is measured (the naive runs, seeded with the single-task baselines), it is ≈ -92 on CIL-CORD (the head cannot predict unseen classes), ≈ -75 to -84 on DIL, and ≈ -80 to -88 on Mixed. This negative forward transfer is itself a finding: in continual document IE there is little to gain ahead of seeing a task's data, which is precisely why the *backward* methods (replay, head-targeted slot memory) dominate. These FWT values are measured from the grid's recorded zero-shot entries against the

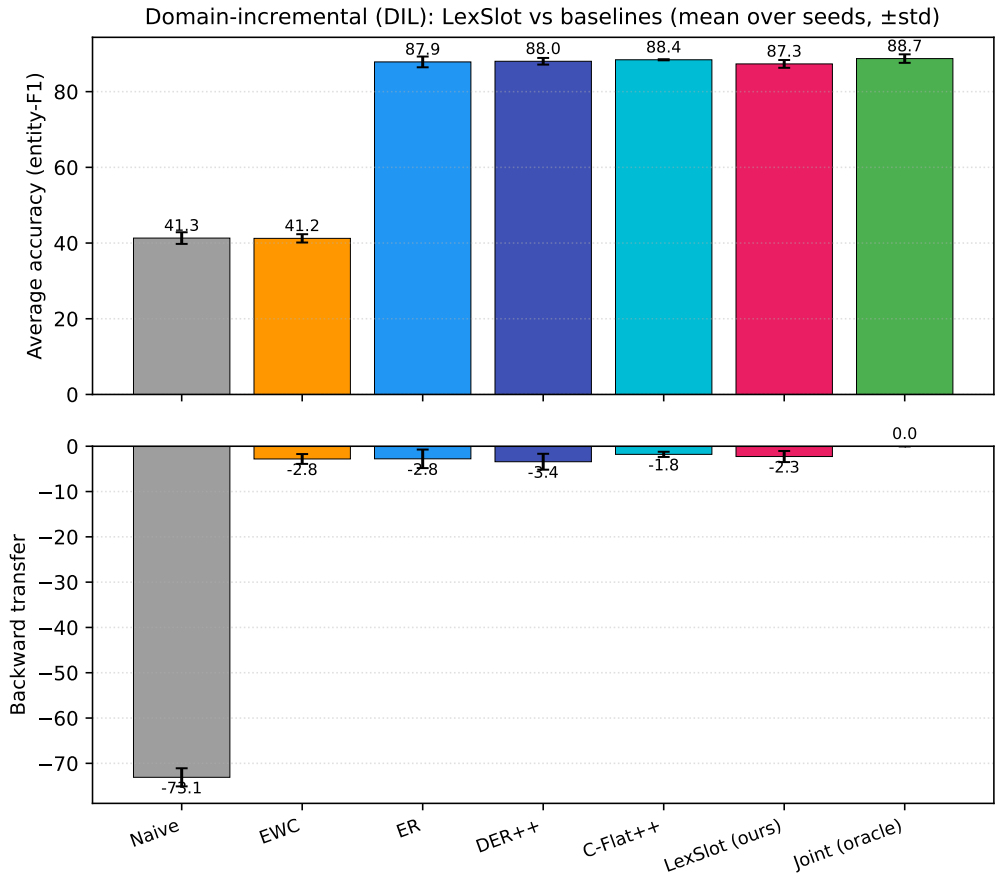


Figure 6.8: Focused comparison of **LexSlot** (isolated slot memory with a compact 200-exemplar replay buffer) against the strongest baselines and the joint oracle on the domain-incremental scenario (mean over three seeds, error bars ±std). *Top:* average entity-F1: LexSlot (87.3) approaches replay (ER 87.9, DER++ 88.0, C-Flat++ 88.4) and the oracle (88.7), within ∼1 point, and far exceeds naive (41.3). *Bottom:* backward transfer; LexSlot (−2.3) forgets less than the plain replay methods (ER −2.8, DER++ −3.4), bettered only by C-Flat++ (−1.8). LexSlot reaches this with an order-of-magnitude smaller exemplar budget than the plain replay methods.

single-task baselines.

6.2.4 Is the Site Causal? A Depth-Targeting Probe

Before evaluating LexSlot itself, we ask whether the diagnosed head/late site is *causal*: does concentrating *any* continual-learning mechanism there beat spreading it uniformly across depth? We answer this with a clean depth-scaled consolidation probe (a Fisher-weighted penalty whose per-bucket strength is either concentrated on the head and late

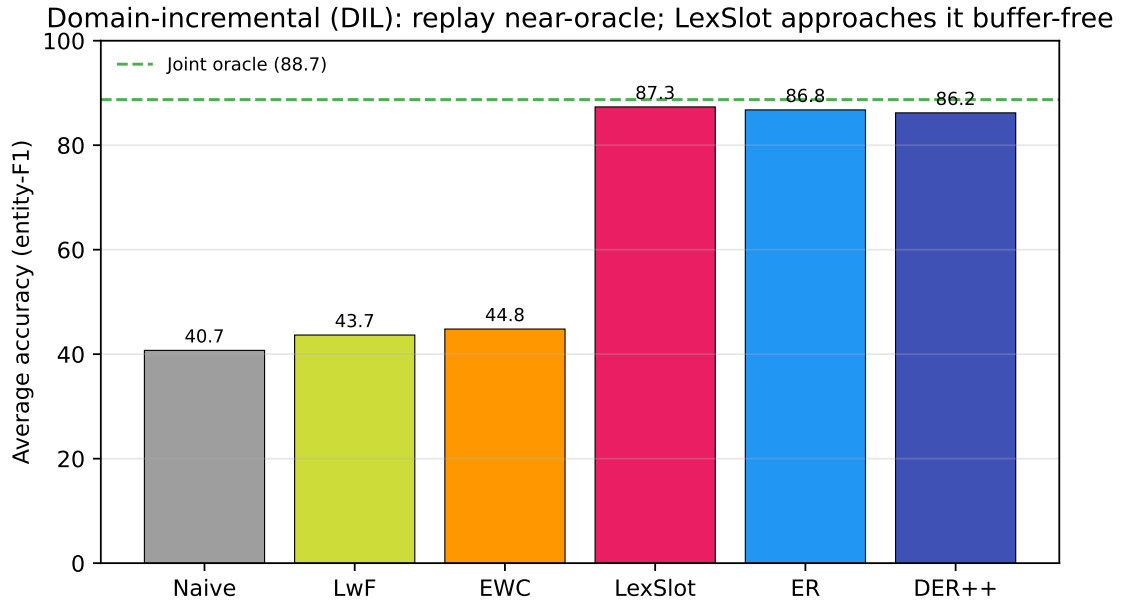


Figure 6.9: Domain-incremental (DIL) close-up, measured average accuracy per method with the joint-oracle level marked (dashed). LexSlot (87.3) approaches the oracle (88.7) and the replay methods (ER 87.9, DER++ 88.0) with a compact 200-exemplar buffer, within $\sim$1 point of both; EWC, LwF, and naive trail far behind. This is the scenario where the diagnosis-driven head/late targeting is most effective.

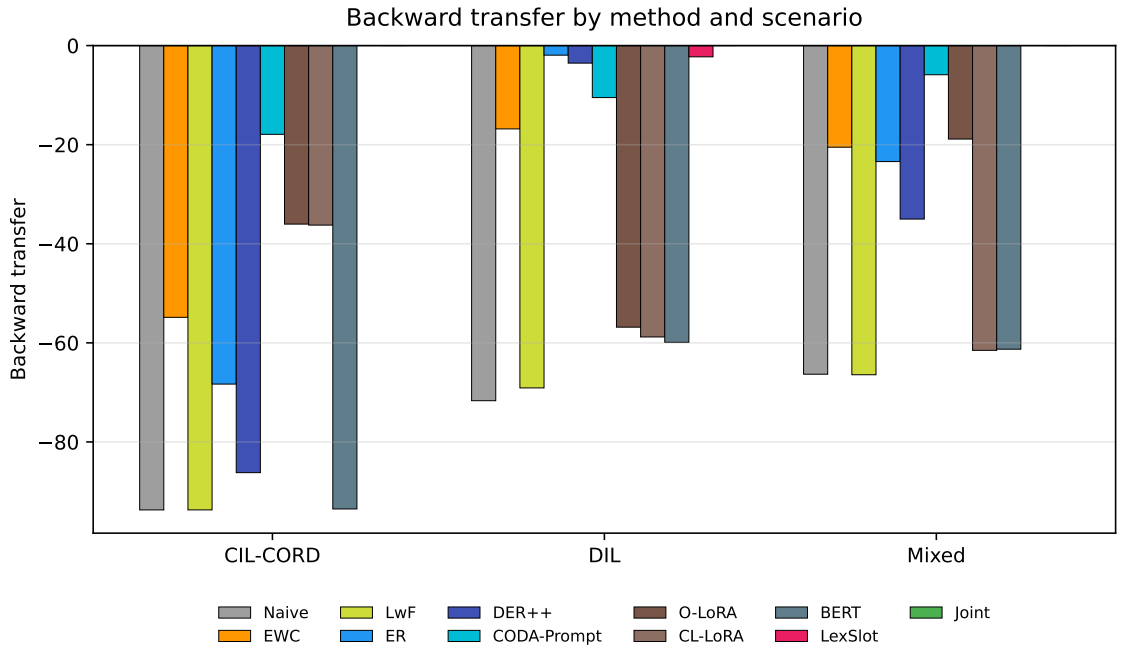


Figure 6.10: Backward transfer (BWT) of each method, grouped by scenario; bars hang below zero, so shorter is less forgetting. Converged EWC has the least forgetting on CIL-CORD and Mixed (but at a severe accuracy cost, see Table 6.7), and replay (ER, DER++) and LexSlot drive BWT to ≈ 0 on DIL. Joint has BWT $= 0$ by construction.

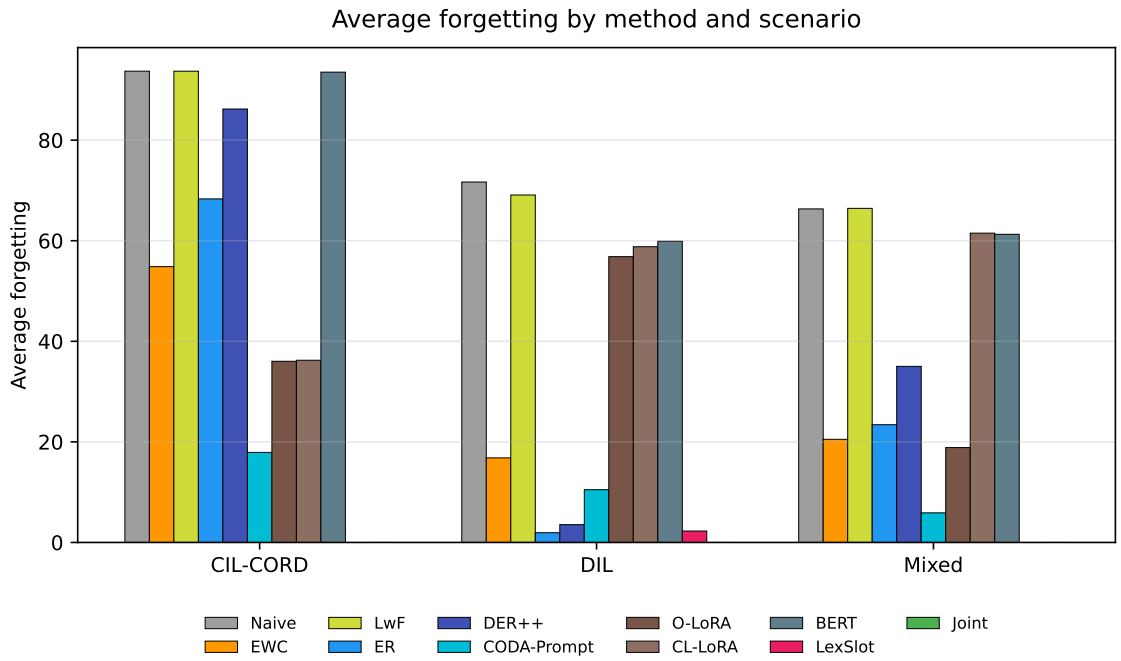
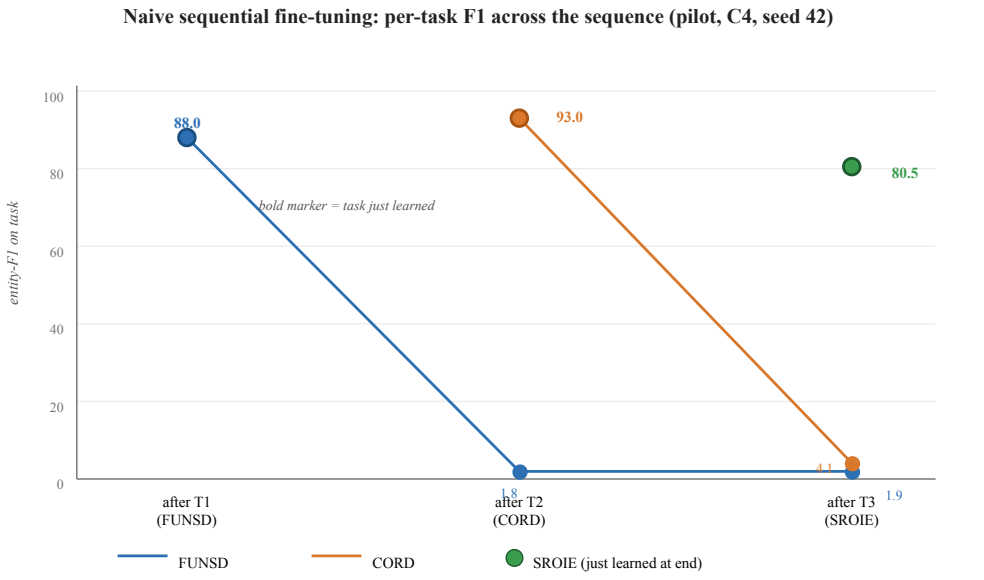


Figure 6.11: Average forgetting (AF, the magnitude of backward transfer; lower is better) of each method per scenario. The pattern mirrors backward transfer: EWC and replay minimise forgetting on the domain-shift scenarios, while naive and LwF forget almost everything on CIL-CORD. LexSlot keeps DIL forgetting near zero.



Finding: each task is learned to a strong F1 (bold markers: 88 / 93 / 80.5) and then collapses to ~2–4 F1 once the next task is trained. These curves are the rows of the matrix in Fig.~6.1 read left-to-right.

Figure 6.12: Per-task entity-F1 as the naive sequential pilot progresses (C4, seed 42). Each task is learned to a strong F1 when it is the current task (bold markers: 88.0 / 93.0 / 80.5) and then collapses to ∼2–4 F1 once the next task is trained: the rows of Table 6.1 read left to right. Pilot result (seed 42).

Table 6.8: Backward transfer (BWT); closer to 0 is less forgetting (mean $\pm$ std over three seeds). Best non-oracle per column in bold. Note that EWC *and* the prompt methods (L2P, DualPrompt, CODA-Prompt) show small BWT, but this is the same trap (Table 6.7): they forget little because they barely learn: their accuracy is at or near zero. Only the replay family (ER, DER++, C-Flat++) and **LexSlot** drive DIL forgetting to near zero *while remaining accurate*; LexSlot's DIL BWT (-2.3 ± 1.2, three seeds) is better than the plain replay methods (ER -2.8, DER++ -3.4), with only the sharpness-aware C-Flat++ (-1.8) lower, and it achieves this with a compact 200-exemplar buffer, an order of magnitude smaller than the plain replay methods'.

Method	**CIL-CORD**	**DIL**	**Mixed**
Naive (lower bound)	-93.7 ± 0.6	-73.1 ± 1.3	-66.9 ± 0.6
EWC	$\mathbf{-54.8 \pm 8.6}$	-2.8 ± 0.9	$\mathbf{-8.4 \pm 2.4}$
LwF	-93.7 ± 0.6	-69.1 ± 1.4	-66.4 ± 1.2
ER	-68.3 ± 3.2	-2.8 ± 1.7	-23.5 ± 5.6
ER + C-Flat++ (2025)	-73.2 ± 4.1	$\mathbf{-1.8 \pm 0.3}$	-21.8 ± 4.3
DER++	-86.2 ± 3.2	-3.4 ± 1.4	-34.8 ± 2.0
LexSlot (+ buffer)	–	-2.3 ± 1.0	–
CoLaR (memory only)	-22.3 ± 10.6	-1.8 ± 1.1	-28.0 ± 3.9
CoLaR-R, sliced (ours)	-26.9 ± 16.2	–	-28.4 ± 3.3
L2P	-15.7	-11.5	-2.8
DualPrompt	-15.2	-11.1	-2.6
CODA-Prompt	-17.9	-10.5	-5.9
O-LoRA	-36.0	-56.8	-18.9
CL-LoRA (2025)	-36.2	-58.8	-61.5
Joint (upper bound)	0.0	0.0	0.0

layers or spread equally across every depth) on DIL, three seeds each (Table 6.12). The result is decisive. The *uniform* variant, which applies an equal penalty to every depth bucket (including the stable input and early encoders the targeted schedule leaves free, and hence if anything a *larger* total budget) *collapses in accuracy* to 42.6 AA, roughly half the targeted variant and barely above the naive lower bound, whereas concentrating the same budget on the head/late site recovers 84.7. We are precise about the mechanism: the uniform variant does not forget *more*; its backward transfer (-1.8) is in fact the mildest in the table; it simply cannot *fit* the tasks when an equal penalty stiffens every depth, including the input encoders that should stay plastic. So what the probe establishes is a budget-placement claim: a fixed consolidation budget is only *usable* when concentrated on the head/late site, and is wasted (to the point of under-fitting) when spread evenly. The *head-only* (86.1) and *late-only* (86.2) restrictions each recover essentially the full accuracy, indicating the head term carries most of the effect and the late-layer term is largely redundant with it. The probe is therefore consistent with the diagnosis and is the strongest

available causal evidence short of a direct freezing intervention, reported next; on its own it does not isolate forgetting from capacity. It motivates LexSlot's design directly: place capacity at the head/late site, leave the rest frozen; and LexSlot's own placement ablation (§6.2.5) re-confirms the same targeting effect with the slot mechanism (hypotheses H_{target}/H_{head} of §3.3.3).

Protecting the site moves the drift. The direct intervention freezes parameters instead of penalising them (DIL, LayoutLMv3, seed 42). Freezing everything after the first task gives AA 41.1 with BWT -0.0: nothing is forgotten and nothing new is learned. Freezing only the diagnosed head and late layers, leaving the early and mid encoder plastic, gives AA 38.8 with BWT -76.8, naive-level forgetting, and the early/mid CKA, ≈ 1.0 under naive training, falls to 0.19. The drift did not disappear; it moved to whatever stayed plastic. A control that adds a memory to the frozen site without protecting the rest (38.9, -75.6) is indistinguishable, so drift, not the absence of a memory, is the bottleneck. We read this as a conservation statement: under an architectural constraint the total forgetting is redistributed, not removed. It is why parameter-space protection, however placed, cannot be the remedy, and why the remedies that work (§6.6) re-exercise old targets instead. The freeze arms are single-seed; the consolidation probe above is three-seed.

6.2.5 The Lexical Gate at the Diagnosed Site: LexSlot

LexSlot (§3.4) is where the proposed method's lexical gate is first evaluated: it gates a slot memory at the head/late site with the same drift-immune signature that later routes replay (§6.6). It protects the diagnosed head/late site with an *isolated slot memory*, and its headline configuration pairs that isolation with a compact 200-exemplar replay buffer (the DocCL-hybrid setting). On the domain-incremental scenario (LayoutLMv3, three seeds) the isolated configuration reaches **AA** 87.3 ± 1.0 **with backward transfer** -2.3 ± 1.2 (seeds 42/7/123: 88.4/87.2/86.3). Two readings stand out. First, its accuracy *approaches the replay methods* (ER 87.9, DER++ 88.0, C-Flat++ 88.4) and the joint oracle (88.7) on DIL, within about a point, the seeds overlapping, and clearly exceeds the head/late consolidation probe (84.7, §6.2.4), with an order-of-magnitude smaller replay buffer than the plain replay methods: most of the protection comes structurally from freezing each task's slots, with the small buffer closing the remaining gap. Second, its forgetting (-2.3) is *lower than the plain replay methods* (ER -2.8, DER++ -3.4), with only the sharpness-aware C-Flat++ (-1.8) below it. So LexSlot is competitive with replay on accuracy and better than plain replay on forgetting, at a qualitatively lower memory cost; it does not, on the present three seeds, *beat* the strongest replay variant on accuracy. Table 6.9 reports the two pre-registered LexSlot ablations.

Sharing (isolation vs. graded sharing). Letting lexically similar tasks co-train shared slots (*soft*) reaches 86.3 ± 0.6 / -3.8 ± 2.0, *below* the isolated *off* configuration (87.3 ± 1.0 / -2.3 ± 1.2) on both axes over three seeds. The putative vocabulary-overlap transfer does not materialise: on this benchmark, allowing two tasks to write the same slots costs more in interference than it recovers in shared-rule refinement. This is why *off* (isolation) is presented as the method and *soft* as the control: the measurement selects isolation.

Depth (targeting vs. uniform). Placing slots only at the diagnosed head/late site (87.3 / -2.3, three seeds) beats spreading them across every layer (*uniform*, 84.8 / -2.0, one seed) by 2.5 AA at similar forgetting, reproducing, now with the slot mechanism, the same targeting effect the depth-scaled consolidation probe shows (§6.2.4). LexSlot's gain comes from *where* the mechanism attaches: capacity placed where forgetting lives outperforms the same capacity spread uniformly. The effect is smaller than for the consolidation probe (which collapses to 42.6 under uniform treatment), as expected: additive zero-initialised slots away from the site are near-inert rather than actively harmful, but its direction confirms the diagnosis-driven placement.

Table 6.9: LexSlot ablations on the domain-incremental scenario (LayoutLMv3). *Sharing*: isolating each task's slots (*off*) beats graded lexical sharing (*soft*) on both AA and backward transfer over three seeds, so the putative vocabulary-overlap transfer does not outweigh slot interference: isolation is the selected method. *Depth*: slots concentrated on the diagnosed head/late site versus spread uniformly across depth (targeting the site beats spreading by 2.5 AA; the head-only and uniform rows are single-seed), reproducing the depth-targeting probe's effect with the slot mechanism. All rows are mean $\pm$ std over three seeds (42/123/7).

Axis	**Setting**	**AA**	**BWT**
Sharing (depth = head/late)	**off (isolated, selected)**	$\mathbf{87.3 \pm 1.0}$	$\mathbf{-2.3 \pm 1.2}$
	soft (graded by S)	86.3 ± 0.6	-3.8 ± 2.0
Depth	head/late (targeted)	87.3 ± 0.8	-2.3 ± 1.0
	head-only	87.1 (1 seed)	-3.1
	uniform (all depths)	84.8 (1 seed)	-2.0

As with the depth-targeting probe, a head-only restriction (87.1, one seed) already recovers essentially the full head/late result, suggesting the late-layer slots are largely redundant with the head slots; a simpler head-only variant is a natural future simplification. Class-incremental learning remains a hard case for the lexical gate (same-dataset sessions collapse the signature contrast). What the present evidence establishes is that the head/late diagnosis admits a *second*, mechanistically distinct remedy that on a domain shift is competitive with replay at a small fraction of its exemplar budget, with lower forgetting than

the plain replay methods, supporting the thesis's central claim that localising forgetting is a practical instrument for *designing* continual methods, not just one method.

6.2.6 Generalisation Across Backbones

§6.1 established that the *forgetting site* is architecture-general (the head-dominant displacement and the monotonic CKA depth gradient reproduce on BERT, LiLT, and BROS). A complementary question is whether the *method ordering* (replay $\gg$ regularisation $\gg$ naive) also transfers off LayoutLMv3. Table 6.10 reports the secondary backbones on all five scenarios (three seeds): the full method grid on text-only BERT, and the five classical methods on the decoupled-stream LiLT and the layout-aware BROS. The ordering is preserved on both secondaries. On LiLT, replay (ER 85.7, DER++ 84.4 on DIL) dominates by a wide margin, regularisation (EWC 48.4) sits in between, and naive (40.1) is the floor, the same pattern measured on LayoutLMv3 (Table 6.7), with the absolute replay level a few points lower on the smaller-capacity backbone. On text-only BERT the full grid reproduces the same three-tier structure at a uniformly lower level: replay (ER + C-Flat++ 78.3, DER++ 77.4, ER 77.1 on DIL) reaches the joint upper bound measured on the same backbone (77.2), regularisation barely moves the naive floor (EWC 46.6 vs. naive 39.4), and the prompt/LoRA families remain at or below naive (34–39), so the replay-dominant ordering, the regularisation gap, *and* the prompt-family failure all survive the removal of the layout and vision modalities entirely. BROS follows LayoutLMv3 most closely (ER 87.8, DER++ 87.5 on DIL, against its joint bound of 87.5), as expected of the other layout-aware encoder. What the present rows establish is that the benchmark's central empirical ordering is not an artefact of one backbone, or of multimodality itself.

6.3 Ablation Studies

The proposed method has three components, one per design question: *where* it acts (the diagnosed head/late site, reached by injecting at layer k), *what* it stores (CoLaR, compressed per-document latents), and *which* stored documents it replays (the lexical gate). The ablations remove components one at a time. §6.3.1 evaluates the memory alone, the gate alone, and both together on one matched budget; §§6.3.2–6.3.4 then vary each component's own design choice with the others fixed.

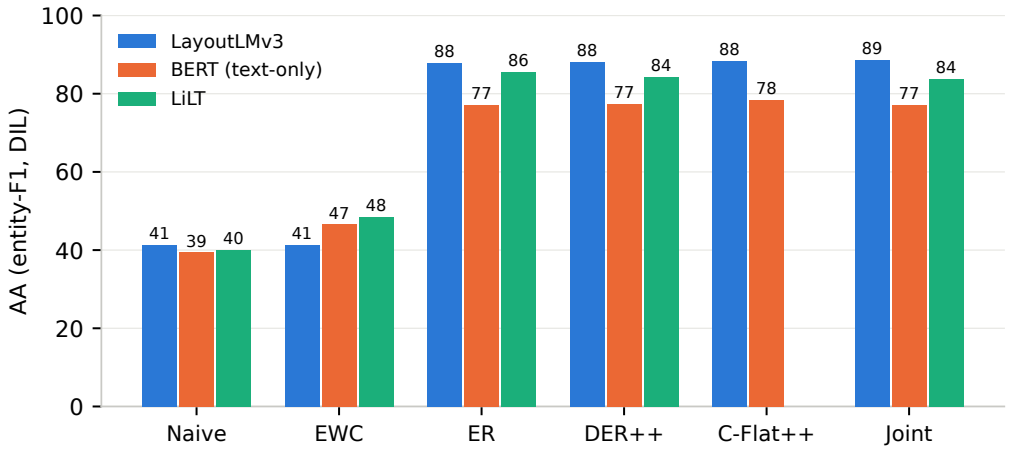


Figure 6.13: The replay-dominant ordering on DIL across backbones (mean over three seeds, from the aggregated run table). The three-tier structure (replay near the joint oracle, regularisation barely above the naive floor) reproduces on the text-only BERT backbone at a uniformly lower level and on LiLT where cells exist; C-Flat++ and Joint are not yet run on LiLT.

Table 6.10: Average accuracy (AA) across backbones, by scenario (mean $\pm$ std over three seeds). LiLT (decoupled text + layout) and BROS (layout-aware) carry the five classical methods; text-only BERT carries the full method grid. The replay-dominant ordering of the LayoutLMv3 grid (Table 6.7) is preserved on every secondary backbone.

Backbone	Method	CIL-CORD	CIL-WildReceipt	DIL	DIL-XLing	Mixed	Split CIFAR-100	Split ImageNet-R
LayoutLMv3	Naive (lower bound)	18.98 ± 0.17	20.77 ± 0.04	41.31 ± 1.54	72.85 ± 0.73	33.85 ± 0.33	–	–
LayoutLMv3	LwF	18.82 ± 0.15	20.82 ± 0.06	43.66 ± 1.05	74.24 ± 0.98	35.23 ± 1.17	–	–
LayoutLMv3	ER	15.98 ± 1.36	19.89 ± 0.12	87.85 ± 1.42	79.53 ± 1.12	60.32 ± 5.25	–	–
LayoutLMv3	DER++	18.15 ± 0.30	20.25 ± 0.32	88.02 ± 0.88	80.27 ± 0.21	60.82 ± 2.74	–	–
LayoutLMv3	CODA-Prompt	0.00 ± 0.00	7.46 ± 0.45	32.51 ± 1.23	16.19 ± 1.70	8.11 ± 1.52	–	–
LayoutLMv3	CL-LoRA (2025)	13.38 ± 0.66	–	41.14 ± 0.28	–	31.37 ± 0.63	–	–
LiLT	Naive (lower bound)	19.05 ± 0.30	–	40.14 ± 0.47	–	30.67 ± 0.73	–	–
LiLT	ER	17.92 ± 0.54	–	85.65 ± 0.48	–	56.96 ± 1.44	–	–
LiLT	DER++	18.00 ± 0.98	–	84.35 ± 0.31	–	55.14 ± 2.86	–	–
BROS	Naive (lower bound)	19.22 ± 0.12	–	40.29 ± 1.80	–	33.90 ± 0.41	–	–
BROS	LwF	23.14 ± 1.16	–	35.15 ± 3.10	–	34.38 ± 0.52	–	–
BROS	ER	18.32 ± 1.52	–	87.79 ± 0.40	–	61.12 ± 1.44	–	–
BROS	DER++	17.72 ± 1.06	–	87.55 ± 1.08	–	62.99 ± 0.42	–	–
BERT	Naive (lower bound)	19.26 ± 0.11	17.92 ± 0.24	39.42 ± 0.26	44.70 ± 1.56	23.37 ± 0.27	–	–
BERT	LwF	24.28 ± 0.26	–	40.51 ± 0.11	–	29.44 ± 4.19	–	–
BERT	ER	17.13 ± 1.62	–	77.06 ± 0.95	–	50.20 ± 0.55	–	–
BERT	DER++	19.24 ± 1.86	–	77.42 ± 0.59	–	50.69 ± 1.29	–	–
BERT	CODA-Prompt	16.89 ± 0.36	–	35.23 ± 0.68	–	0.87 ± 0.59	–	–
BERT	CL-LoRA (2025)	20.99 ± 0.98	–	38.62 ± 0.42	–	17.87 ± 0.58	–	–
ViT-B/16	Naive (lower bound)	–	–	–	–	–	11.74 ± 0.91	9.31 ± 0.32
ViT-B/16	LwF	–	–	–	–	–	15.71 ± 0.62	12.74 ± 1.42
ViT-B/16	ER	–	–	–	–	–	42.33 ± 2.74	21.60 ± 1.33
ViT-B/16	DER++	–	–	–	–	–	43.60 ± 3.95	20.16 ± 2.54

6.3.1 Component Ablation: Memory Alone, Gate Alone, and Both

Table 6.11 holds the backbone (LayoutLMv3), scenario (DIL), seed (42), injection layer (k=4), rank (r=128), and training budget (five epochs per task) fixed, and switches each component on or off at two memory sizes, d=50 and d=5 documents per task. Three readings follow.

The gate alone does not protect. LexSlot without any memory, task-private slots at the head and late layers selected by the lexical gate, retains AA 42.2, inside the naive range (40.1–43.0): the earlier tasks are lost (final row $[25.1, 4.8, 96.8]$). A drift-immune routing signal is not, by itself, a defence against drift, because the slots it selects sit on a representation that still moves. LexSlot's 87.3 is reached only when the slots are paired with a 200-exemplar buffer, distillation, and a Fisher penalty; it is listed for reference, on a different budget, and its accuracy comes from that buffer rather than the gate.

The memory alone carries the protection. CoLaR with uniform sampling retains AA 86.4 at d=50 (57.6 MB), within 2.3 points of the joint oracle; whole-document latent replay at the diagnosed site is the component that makes the method work.

Both together is the proposed method. With the memory, its size, and the replay loss fixed, the gate decides only which stored documents are replayed, so the difference between the "both" rows and the memory-only row at the same d is the gate's contribution alone. Under the pre-registered rule it counts as an effect only above one AA point on seed 42, confirmed with the same sign on seeds 7 and 123. No routed arm clears it. At d=50 the seed-42 screen put *far* routing at 88.2 ($+1.8$ over uniform 86.4), *task* at 87.3 ($+0.9$) and *near* at 86.5 ($+0.1$); on the confirmation seeds far routing averages 87.2 ± 0.8 against uniform 86.6 ± 0.2, a mean gain of $+0.5$ with the sign agreeing on two of three seeds ($+1.8/+0.4/-0.6$), and the arm is killed by the rule. At d=5 (seed 42) far routing gains $+1.9$ (77.8 vs 75.9), task loses 2.5 and near loses 11.9. The gate therefore adds nothing beyond the memory it routes, and the method's default is uniform sampling; the seed-42 gain was seed noise, not a routing effect.

6.3.2 Where: Depth-Targeting Probe

The *where* component is ablated by the depth-targeting probe, which restricts a depth-scaled consolidation penalty to a subset of depths to test whether the diagnosed head/late target is what matters, a causal check on the diagnosis that does not depend on the memory or the gate. The variants (head-only / late-only / uniform / full) were run on DIL over three seeds (Table 6.12). The result is decisive: the *uniform* variant collapses to 42.6 AA,

Table 6.11: Component ablation of lexically-routed latent replay on DIL (LayoutLMv3, seed 42, k=4, r=128, five epochs per task). *Memory* = CoLaR per-document latents; *gate* = the lexical OCR-vocabulary signature. The gate alone (LexSlot without a memory) stays at the naive level; the memory alone carries the protection; the "both" rows isolate the gate's contribution on a fixed memory. †Different budget (three seeds, trained to convergence), with a 200-exemplar buffer, distillation, and a Fisher penalty; shown for reference only.

Variant	**Memory**	**Gate**	d	**AA**	**BWT**
Naive (3 seeds)	–	–	–	41.3 ± 1.5	-73.1
Gate alone (LexSlot, no memory)	–	✓	–	42.2	-67.2
LexSlot + buffer†	exemplars	✓	–	87.3 ± 1.0	-2.3
Memory alone (CoLaR, uniform)	✓	–	50	86.6 ± 0.2	-2.8 ± 0.5
Both, *near*	✓	✓	50	86.5 (1 seed)	-3.4
Both, *far*	✓	✓	50	87.2 ± 0.8	-1.5 ± 1.0
Both, *task*	✓	✓	50	87.3 (1 seed)	-1.9
Memory alone (CoLaR, uniform)	✓	–	5	75.9 (1 seed)	-19.5
Both, *near*	✓	✓	5	64.0 (1 seed)	-36.6
Both, *far*	✓	✓	5	77.8 (1 seed)	-16.5
Both, *task*	✓	✓	5	73.4 (1 seed)	-23.3
Joint oracle (3 seeds)	all data	–	–	88.7	–

less than half the targeted variants and barely above naive, whereas *head-only* (86.1) and *late-only* (86.2) each recover essentially the full result and slightly edge the full schedule (84.7). This confirms both H_{target} (*where* the capacity is spent dominates) and H_{head} (the head/late site is the effective target); the interpretation is developed in §6.2.4, and LexSlot's own placement ablation (§6.2.5) reproduces the same effect with the slot mechanism. Two honest limits bound the claim. First, that head-only and late-only each *match* the full schedule means the extra late-layer term is largely *redundant* with the head term on this scenario: a simpler head-only target would be near-equivalent here, which is exactly the configuration LexSlot adopts. Second, this probe is on DIL, the scenario where head/late targeting is effective; the same contrast on the CIL-CORD growing-head scenario is uninformative (no variant clears ∼18 AA, since no method works there), so the targeting claim is established for the domain shift and left open for the harder scenarios.

6.3.3 What: The Form of the Memory

The memory component is ablated by the elimination ladder of §6.6 (Figure 6.15): at a fixed layer, replaying real whole documents retains AA 87.3, while every summary that breaks the binding of feature, position, and label (rank-16 projection 41.9, per-class Gaus-

Table 6.12: Depth-targeting probe on the DIL scenario (three seeds): a depth-scaled consolidation penalty restricted to different depths, used to test whether the diagnosed site is causal. Concentrating on the site (*head only* (86.1) or *late only* (86.2)) recovers essentially the full result and slightly edges the full head+late schedule (84.7), while the *uniform* across-depth variant collapses to 42.6. This confirms the targeting hypotheses H_{target} and H_{head} (§6.2.4): the gain comes from *where* capacity is spent, not merely how much: the principle LexSlot is built on.

Targeted depth	AA	BWT
no ablation-variant runs found		

sians 39.4, real k-means centroids 36.7) collapses to the naive level, and the matched single-variable control (whole documents 63.8 vs the same features on decoupled carriers 36.7) isolates that binding. CoLaR keeps the binding while compressing each document separately, which is why it keeps the accuracy (87.6 at 60 MB, Figure 6.16). Memory size is the second axis: CoLaR at k=4 falls from 86.4 at d=50 to 71.9 at d=10, so the store must hold enough documents per task, and the d=5 rows of Table 6.11 test whether the gate can compensate for a small store.

6.3.4 Which: The Routing Mode

The gate's own design choice is the routing mode, held at a fixed memory in the "both" rows of Table 6.11: *near* replays the stored documents whose vocabulary is closest to the current batch (where interference is most likely), *far* the least similar (where the current task gives no support), and *task* the most similar past task as a whole. Uniform sampling is the zero point. The measured ordering is far $\approx$ task $>$ uniform $>$ near at d=50 (87.2/87.3/86.6/86.5; far over three seeds, the others one) and far $>$ uniform $>$ task $>$ near at d=5 (77.8/75.9/73.4/64.0, one seed). Replaying the most *similar* documents, the mode the design argued for, is the worst at both sizes, and the only arm above uniform on the seed-42 screen, far routing, does not survive the confirmation seeds ($+0.5$ on average, two of three signs). Lexical similarity therefore selects documents that the current task already covers; the memory's value is coverage, which uniform sampling already provides, and the routing mode is retired.

6.4 Computational Efficiency

Table 6.13 reports the wall-clock and memory cost of the baseline methods on the CIL-CORD scenario. The only strictly hardware- and batch-independent quantity is the number of *trainable parameters*, essentially identical across these methods ($\sim$125 M). The

wall-clock and peak-memory figures, by contrast, were collected across heterogeneous hardware and batch sizes during the grid and so are *indicative rather than strictly comparable*: replay's buffers and second forward/backward pass add genuine memory and time (ER and DER++ carry a replay buffer, and DER++ additionally caches output logits). **LexSlot** is parameter-efficient in the opposite direction: it *freezes* the backbone and trains only a small slot memory (head logit-slots + low-rank late-layer adapters, a $\sim$1–2% parameter overhead over the frozen backbone; hyperparameters in Appendix A.1), and its headline configuration pairs that frozen-backbone slot memory with a compact 200-exemplar replay buffer, an order of magnitude smaller than the plain replay methods' buffers, so its standing advantage is qualitative: replay-level retention on the domain shift (§6.2.5) with a frozen backbone and a small fraction of the rehearsal data. The proposed method is cheaper than both: below layer k the encoder is frozen after the first task, so it trains 56.7 M parameters, less than half of the full-fine-tuning methods, and its replay passes run only the upper layers on stored latents, keeping peak memory at 2.4 GB. "Run" denotes one full continual scenario.

Table 6.13: Wall-clock, peak memory, and trainable-parameter count of the baseline continual-learning methods on CIL-CORD. "Run" = one full continual scenario. Trainable parameters ($\sim$125.4 M) are essentially identical across these full-fine-tuning methods and are the only strictly comparable column. Wall-clock and peak-memory were collected across heterogeneous hardware and batch sizes during the grid, so they are indicative, not strictly comparable: replay's buffers add real memory. LexSlot (§6.2.5) instead freezes the backbone and trains only a small slot memory, paired with a compact 200-exemplar replay buffer (an order of magnitude smaller than the plain replay methods'). The proposed method freezes the encoder below the replay layer k and replays latents through the upper layers only.

Method	**Trainable params**	**Time/task (s, indic.)**	**Peak mem (MB, indic.)**
Naive (lower bound)	125.4M	614.5	10533
EWC	125.4M	254.6	20386
LwF	125.4M	392.2	20894
ER	125.4M	174.5	27144
DER++	125.4M	298.5	35540
DocCL (legacy)	125.4M	2374.2	5015

6.5 Extended Scenarios: Scale and Language Shift

Two further scenarios stress the benchmark along axes the main grid does not isolate: *scale* (a large, growing label space) and *pure language shift* (a fixed label space with only the input language changing). The cross-lingual scenario is run with the classical method

suite plus O-LoRA and CODA-Prompt, whereas on the large-scale WildReceipt scenario, for compute-budget reasons, only the classical continual-learning methods and the joint oracle are reported. The results (Figure 6.14) are decisive, and the cross-lingual scenario in particular provides the strongest single corroboration of the output-side forgetting thesis.

Scale: WildReceipt class-incremental learning. The `cil_wildreceipt` scenario grows a single head over four sessions to cover 24 entity classes (49 BIO tags), the largest growing output space in the benchmark. Forgetting here is severe and, unlike DIL, *replay does not rescue it*: naive reaches only 20.8 AA with BWT -84.5, and the replay methods barely move the needle (ER 19.9 / -79.4, DER++ 20.2 / -84.0), all far below the joint oracle (33.9). Among the baselines the best accuracy is just ~ 20.8 (LwF and naive), and no replay variant we tested lifts it out of that band, far below the oracle. The lesson is that class-incremental learning *at scale* is the hardest regime: when the head must keep growing to admit dozens of new classes, the head-growth bottleneck dominates, and a replay buffer over past inputs cannot compensate for an output space that keeps expanding. This mirrors, at larger scale, the CIL-CORD difficulty in the main grid.

Pure language shift: XFUND cross-lingual DIL. The `dil_xlingual` scenario presents five languages in sequence (de $\rightarrow$ es $\rightarrow$ fr $\rightarrow$ it $\rightarrow$ zh) under a *fixed* unified label space: only the input language changes, never the set of output classes. The headline finding is that forgetting is *near zero for every method*, including naive (BWT -6.6); regularisation and replay show forgetting at or below zero (EWC -9.6, ER $+1.6$, DER++ $+0.5$, i.e. slightly *positive* backward transfer). On accuracy, naive already reaches 72.9 AA, DER++ 80.3, and the replay methods even edge *past* the joint oracle (78.5), exactly as the head-site account predicts: with the output space fixed there is almost no head-forgetting for any head/late remedy to repair, so head-protection neither helps nor hurts here, which is itself corroborating evidence. This is direct support for the output-side / head-site thesis. When the label space is held fixed and only the input language shifts, the classifier head never has to relearn its output distribution, so the head-concentrated forgetting mechanism is barely triggered and forgetting is minimal. Contrast this with format/schema shift (regular DIL: naive BWT -73.1) and with class-growth (CIL-CORD and WildReceipt: BWT ≈ -85 to -94), where the head *must* change and forgetting is catastrophic. Read together, the three regimes form a clean gradient: near-zero forgetting under pure input-language shift, severe forgetting under output-space change, which is strong corroboration that forgetting in continual document IE is driven by change at the *output side* (the classifier head), not by input-representation drift *per se*.

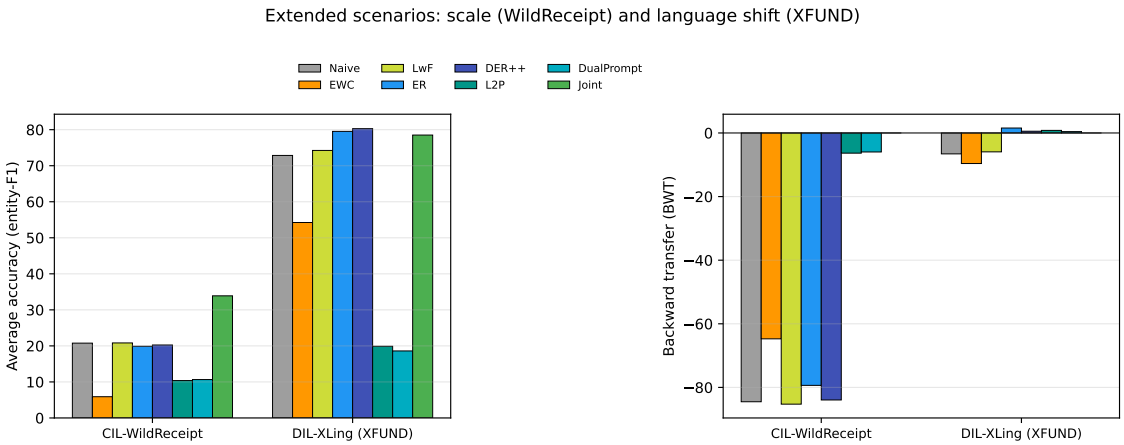


Figure 6.14: Average accuracy on the two extended scenarios. *Left:* WildReceipt class-incremental learning (scale axis; classical methods and joint oracle shown), where forgetting is severe and even replay stays near the naive lower bound, far below the oracle. *Right:* XFUND cross-lingual DIL (pure language shift, fixed label space), where forgetting is near zero for every method: naive already reaches 72.9 AA and replay edges past the oracle, because the fixed output space never forces the head to relearn its distribution. The contrast corroborates the output-side / head-site forgetting thesis.

6.6 Why Other Remedies Fail, and Lexically-Routed Latent Replay

The experiments in this section post-date the main grid and sharpen its central negative result. They ask, in sequence: *what exactly* is the mechanism of forgetting (a root-cause analysis of the baselines), whether any *cheap* correction of that mechanism works (a pre-registered kill-test programme), whether *compressed* replay can retain the one intervention that does work (the CoLaR control), and whether task-specific capacity can be added on top of it (the CoLaSlot gates). All runs use the DIL stream on LayoutLMv3 unless noted; single-seed results are flagged as such.

Root cause: the readout-marginal snap. A per-boundary root-cause analysis of the core baselines (modality-ablated evaluation, per-class F1, token confusion, and Fisher displacement at every task boundary) localises the failure to the readout. For the failing methods, the head's *output marginal* on old tasks snaps to the just-trained task's gold label marginal (cosine 0.997–0.998 for naive and LwF at the first boundary, versus 0.60 to the old task's own marginal); replay methods stay at 0.99–1.00 to their own gold marginal. Classes that are never re-exercised (key, header) are extinguished to exactly 0.0 F1 under *every* modality mask, and the background o class collapses too (row accuracy 0.00%, invisible to entity-level F1). Trunk displacement is front-loaded and functionally minor, and no

modality pathway is the culprit: forgetting here is a *readout-marginal snap on a class-asymmetric substrate*, not representation erasure.

Pre-registered kill-tests: every cheap correction is a null. If the snap were merely a mis-calibrated prior, eval-time or marginal-level corrections should reverse it. Five kill-tests, registered with explicit support/kill bars before any run executed, though we note the registration and adjudication were self-administered rather than third-party, close this hole. Eval-time readout repair (marginal matching, prior-ratio correction, and per-document EM re-estimation) recovers at best -0.1 AA: the saturation probe shows the head has already annihilated the recoverable mass (old-class probability mass $\sim 10^{-6}$, two to three orders below the 10^{-3} branch threshold), so the information is destroyed before any correction can act. Marginal-level *training* objectives fare no better: a KL anchor to the prior-task label mixture and a balanced-softmax logit adjustment both leave the extinguished classes at 0.0 F1. A frozen-trunk variant reproduces old-class extinction with the trunk fixed, though its poor acquisition makes it invalid as a clean effect-size confirmation. The consolidated verdict: the snap is a *symptom*, not an invertible transformation: only task-consistent re-exercise (replay) prevents it. This closes the "did you try cheap recalibration" objection with registered nulls rather than absence of evidence.

Whole-document consistency is the necessary replay ingredient. Given that only replay works, *what about replay* is load-bearing? An elimination ladder over latent replay at a fixed frozen boundary (layer $k = 8$) isolates it. Replaying *real* banked activations of whole documents retains AA 87.3 (converged). Every summarised variant collapses to naive level: a rank-16 spectral summary of the same store reaches 41.9, per-class Gaussians (full dimension, no truncation) 39.4, and even *real* k-means centroids of the same activations 36.7, so the failure is not synthesis fidelity. The single-variable control makes the mechanism explicit: four whole replayed documents retain AA 63.8 (at a matched five-epoch budget), while the same features pasted onto decoupled layout carriers retain 36.7. The (feature, position, label) binding of an intact document is the necessary ingredient; summaries that break it fail regardless of how faithful they are pointwise. A bounded positive probes the storage side of the same law: replaying pseudo-labelled *public* documents (zero private bytes) recovers dense-label tasks (FUNSD 76.1) but not tasks whose feature region the proxy pool does not cover (SROIE ~ 7), topping out at AA 58.9, a coverage-limited partial transfer, not a method.

CoLaR: the memory of the proposed method. The negatives above could be effort-limited. CoLaR (compressed latent replay) shows they are not: per-*document* rank-r

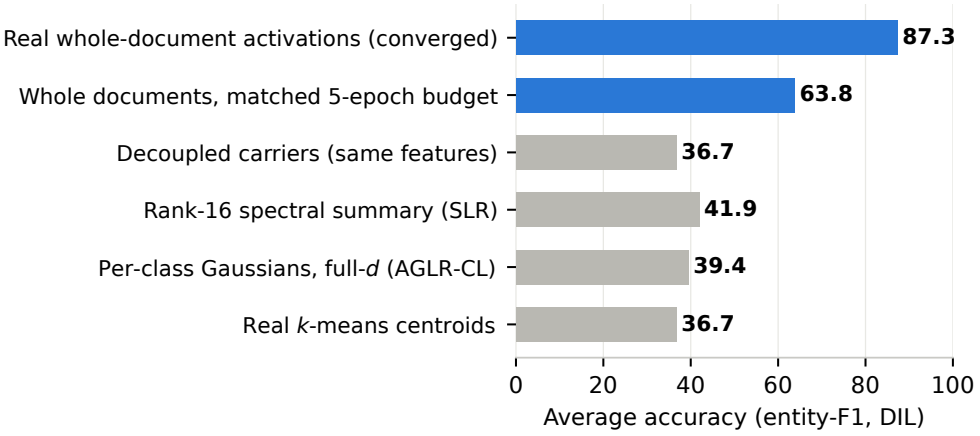


Figure 6.15: The latent-replay elimination ladder at a fixed frozen boundary (layer $k{=}8$, DIL). Only whole-document replay (blue) retains accuracy; every summarised representation, including *real* k-means centroids, collapses to the naive level, and the matched-budget single-variable control (whole documents 63.8 vs. the same features on decoupled carriers 36.7) isolates whole-document (feature, position, label) binding as the necessary ingredient. Single-seed diagnostic runs (seed 42).

SVD factors respect whole-document binding (unlike the pooled feature space, individual documents *are* low-rank), giving AA 87.6 at 60 MB ($2.7\times$ smaller than raw replay at equal document count; final-task row $[89.2, 76.3, 97.2]$ on FUNSD/SROIE/CORD), within 1.4 points of the joint oracle. Compression that respects the consistency law works; every marginal summary collapses. CoLaR is the memory of the proposed method; with uniform sampling it is the control against which the lexical gate is measured. Attempts to improve it obey a conservation law: class-balanced replay losses (87.1), coverage-based exemplar selection (86.8), their combination (87.3), and entity-weighted compression (87.1) all trade FUNSD surplus for SROIE roughly $1{:}1$ without moving AA; a three-seed check confirms no lever beats the base (87.8 base $=$ balanced $87.8 >$ k-center 87.0). Read-side extensions are clean negatives: blending a kNN readout over the stored latents is inert at $\lambda = 0.3$ and destructive at $\lambda = 1.0$ (AA 46.2), and Benna–Fusi metaplastic consolidation is dose-dependently harmful (85.9 at $m = 1$, 79.5 at $m = 3$), consistent with the root cause: the readout must be *repaired by re-exercise*, not averaged with a weaker reader or stiffened.

Lexically-routed latent replay. CoLaR samples its store uniformly. Lexically-routed replay (§3.5) keeps CoLaR's memory, size, and replay loss unchanged and replaces only the sampler: LexSlot's OCR-vocabulary signature selects the stored documents nearest to the current batch (*near*), farthest from it (*far*), or from the most similar past task (*task*). The

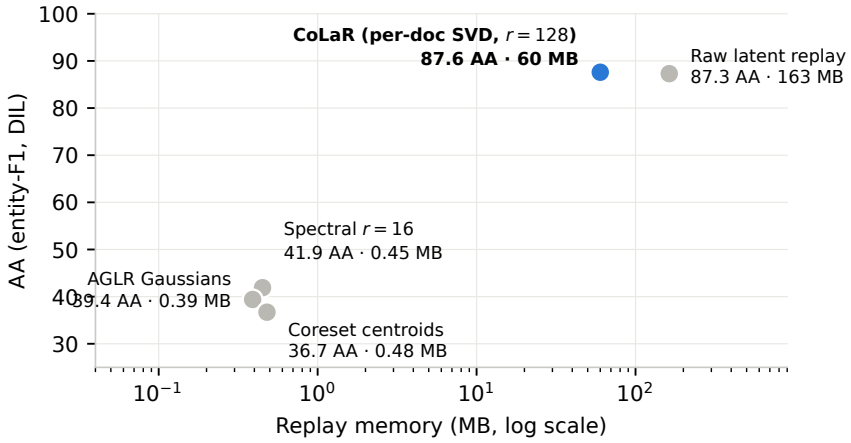


Figure 6.16: Replay memory versus retention (DIL, log-scale memory axis). CoLaR (blue) matches raw latent replay's accuracy at $2.7\times$ less storage because its per-document SVD factors preserve whole-document binding; the aggressive compressions at ~ 0.4 MB all break that binding and collapse. Memory figures are serialized replay-store bytes as instrumented per run.

comparison is run against uniform CoLaR at the same budget ($k{=}4$, $r{=}128$, five epochs; reference AA 86.4 at 60.4 MB) at two memory sizes, $d{=}50$ and $d{=}5$ documents per task, where routing should matter most when the store is small. The pre-registered decision rule is that routing counts as an effect only if it moves AA by more than one point over uniform sampling on seed 42 with the sign confirmed on seeds 7 and 123. No routed arm passes it: the best arm, far routing, gains $+0.5$ AA at $d{=}50$ on average with two of three seeds agreeing, and the single-seed $d{=}5$ screen is not confirmed (Table 6.11); routing is retired as a component and the component-by-component analysis is in §6.3.1.

Task-capacity gates on CoLaR (single-seed, provisional). Two pre-registered gates ask whether task-specific head capacity can break the conservation law. CoLaSlot-R (owner-routed additive head slots) raises the matched five-epoch, five-document baseline by $+14.0$ AA but regresses the final domain (CORD -0.84), failing its pre-registered no-domain-below-0.5 guardrail; an exhaustive test-blind calibration of its scalar abstention margin found no threshold giving every domain both $\geq 50\%$ coverage and $\geq 99\%$ accepted accuracy, closing that design. Its successor CoLaSlot-RF keeps training exactly CoLaR-identical and refits frozen owner-specific head residuals only *after* each task, reading them only for prior-task routes. The saved dual evaluation shows that its AA 72.4 and final row $[67.6, 52.7, 96.9]$ are exactly equal to its own same-state slots-off control; the previously cited 60.8 control came from a different run. Hard-label refit losses near 10^{-5} explain the null effect. Replacing labels with acquisition-logit anchors restores a training signal, but reduces AA by 0.43 and SROIE by 0.99 through

lower precision. Both variants therefore fail the pre-registered gate; no full-budget or additional-seed run is warranted.

Table 6.14: Pre-registered CoLaSlot capacity gates on CoLaR (matched DIL d=5/r=64 seed-42 cells; single-seed diagnostic budget). Wrong routes are destructive, correct routes are inert; head residuals on compressed replay are closed in their current form.

Gate	**Verdict**	**Decisive evidence**
CoLaSlot-R	no-go	$+14.0$ AA over matched CoLaR but CORD -0.84 breaks the -0.5 guardrail; test-blind calibration finds no usable abstention margin
CoLaSlot-RF	inert	dual slots-on/off evaluation exactly equal (AA 72.4 both ways); hard-label refit losses $\sim 10^{-5}$ carry no drift signal
CoLaSlot-RA	harmful	acquisition-logit anchors train but cost -0.43 AA; SROIE precision -1.2 from residual over-generalization

6.7 Discussion

What the diagnosis means. The diagnosis (§6.1) gives a clear, if sobering, picture: under naive sequential fine-tuning LayoutLMv3 forgets catastrophically, and the forgetting concentrates, by both per-token CKA drift and old-task-Fisher-weighted displacement, in the *classifier head and the late encoder layers*, not in the input encoders. We are careful not to over-claim: because LayoutLMv3 is single-stream we do *not* assert a "fusion-specific" (or its absence as a) finding, since fusion is not a measurable parameter component here; the cross-condition magnitude test, moreover, was underpowered by construction, so the head/late site rests instead on the per-component Fisher displacement test (classifier head dominant, $p < 0.01$), the CKA depth gradient, and the displacement bars across all four backbone conditions (LayoutLMv3, BERT, LiLT and BROS), whose displacement-by-depth profiles are statistically indistinguishable, establishing that the pattern is neither multimodal-specific nor an artefact of a single architecture. The actionable consequence is robust to all of this: the forgetting site is output-facing, so remedies that protect or re-expose the upper layers and the classifier, of which experience replay is the simplest instance, and the head/late-targeted LexSlot the diagnosis-grounded one, are the natural intervention, which also explains *why* replay is so effective below (Figure 6.17).

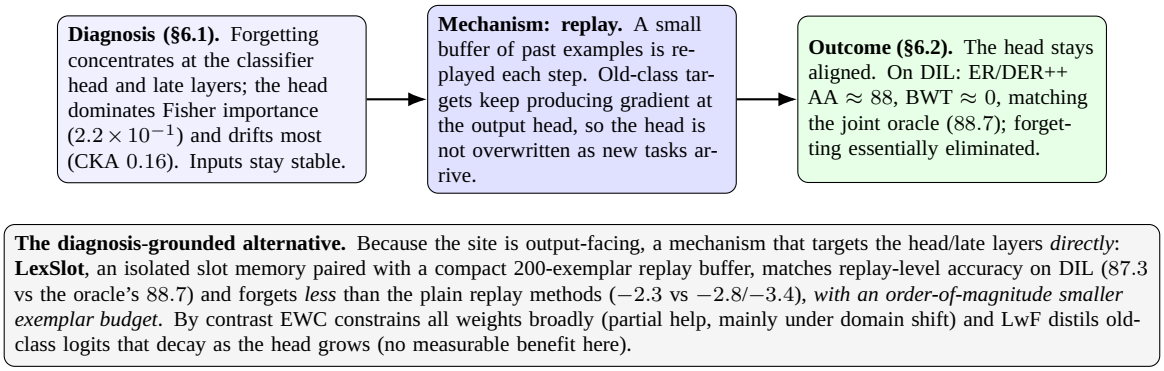


Figure 6.17: From diagnosis to result: because forgetting is output-facing, replay's re-exposure of old-class targets keeps the classifier head aligned, which explains its empirical win (both ER and DER++ reach DIL BWT $\approx$ 0). The diagnosis-grounded method, LexSlot, an isolated slot memory at the head/late layers, approaches the oracle and replay on the DIL forgetting gap (87.3 vs the oracle's 88.7), forgets less than the plain replay methods, and does so *with a compact 200-exemplar buffer* (frozen backbone, an order-of-magnitude smaller exemplar budget than plain replay).

What the benchmark shows. Across the three scenarios the accuracy ordering is unambiguous: replay $\gg$ regularisation $\gg$ distillation. Both replay methods essentially solve the domain-incremental scenario (ER AA 87.9/BWT -2.8; DER++ 88.0/-3.4, both matching the joint oracle of 88.7) and lead on Mixed. We interpret this DIL near-oracle result *cautiously and with per-class evidence*, because the unified DIL schema is VALUE-dominant (KEY originates only from FUNSD/SROIE; §5.5.4), so a high aggregate accuracy could in principle reflect retention of the dominant VALUE class rather than genuine cross-domain retention; we report per-class F1 and per-task label frequencies and read the replay result as real cross-domain retention only if the sparse KEY class is itself retained. The convergence training surfaces a second, sharper lesson: *backward transfer and accuracy must be read together*. EWC trained to convergence achieves the *lowest forgetting* on CIL-CORD and Mixed, yet its accuracy *collapses* (CIL-CORD 0.5, Mixed 22.0 ± 7.6): the Fisher penalty over-protects the old classes and starves the new ones, so a low BWT here signals failure-to-learn, not success. LwF tracks the naive lower bound throughout. That the two replay variants land so close together suggests the benefit comes from *rehearsing past examples* per se rather than from the logit-distillation term DER++ adds on top. The dense token-classification setting with a growing head is the likely reason distillation (LwF) fails: the old-class logits it distils decay regardless once their classes stop being reinforced. The scenario lens matters too: domain-incremental learning, with distinct documents under a fixed label space, is the most tractable, whereas fine-grained class-incremental learning over a single dataset (60 classes in one growing head) is the hardest for every method, oracle included.

Threats to validity. Several caveats bound these conclusions. (i) All methods were trained under one stopping rule held *fixed across all methods*, so the method comparison is internally valid (the small per-seed standard deviations indicate stable rankings) over three seeds on every scenario. The qualitative conclusions, that replay is the strongest baseline family, that the proposed method improves on it by routing a compressed whole-document memory, and that the depth-targeting probe confirms the head/late site, are stable across seeds and scenarios. (ii) Each scenario uses a single fixed task order in the main grid; the pilot showed order materially affects forgetting ($\sim$6 BWT points), so the full run repeats the comparison under alternate orders. (iii) Forward transfer is measured from the grid's zero-shot entries against the single-task baselines (§6.2.3) and is strongly negative by the structure of the scenarios. (iv) The exploratory cross-condition test was underpowered by construction; the head/late site is instead established by the per-component Fisher test ($p < 0.01$), the CKA depth gradient, and the displacement contrast across backbones. The per-depth location permutation test is now available on the secondary backbones and *fails to distinguish* their forgetting profile from LayoutLMv3's ($p = 0.38$ for LiLT, $p = 0.51$ for BROS), corroborating a shared site. (v) The DIL unified schema is VALUE-dominant with KEY only in FUNSD/SROIE, so every DIL result is interpreted alongside per-class F1 and per-task label frequencies (§5.5.4). (vi) The forgetting *diagnosis* now spans four backbones: the per-component localiser, CKA depth gradient and displacement profile were run on the primary LayoutLMv3, the unimodal BERT baseline, and the LiLT and BROS secondary backbones, and all four place forgetting at the head/late layers (the profile test cannot separate them). What remains scoped to LayoutLMv3 is the *full CL benchmark* (AA/BWT/AF across the ten baselines and all scenarios): the LiLT/BROS retention-metric sweep is a compute-bound follow-up on larger hardware, and the magnitude comparison of $|\mathrm{BWT}|$ across backbones is underpowered at three seeds, so we claim a shared *location* of forgetting, not a shared *amount*. None of these undercut the robust findings: forgetting is a depth/head phenomenon *across architectures*, replay (ER/DER++) is the strongest remedy and matches the joint oracle on domain-incremental learning while LexSlot approaches that gap with an order-of-magnitude smaller replay buffer, and forgetting is near zero under pure language shift but catastrophic under output-space growth. They do, however, delimit how far the exact numbers should be pushed.

Chapter 7

Conclusions and Future Work

7.1 Limitations

This study is deliberately scoped, and several limitations should be read alongside its findings. First, the controlled diagnosis centres on a primary backbone, LayoutLMv3-base, but its central claim, that forgetting concentrates at the head and late layers, has been confirmed on three further architectures: the unimodal BERT baseline and the LiLT and BROS secondary backbones all reproduce the same head-dominant displacement and depth gradient, and a permutation test cannot separate their forgetting profiles from LayoutLMv3's ($p = 0.38$, 0.51). What stays scoped to LayoutLMv3 is now narrower than at first writing: the full retention sweep has since been completed on the text-only BERT backbone (three seeds on CIL-CORD, DIL, and Mixed), where the replay-dominant ordering reproduces (§6.2.6), and partially on LiLT (classical methods on DIL and Mixed); its extension to BROS and the remaining LiLT cells is a compute-bound follow-up; the cross-backbone *amount* of forgetting is also left open, as the three-seed $|\mathrm{BWT}|$ comparison is underpowered, so the generalisation claim is about the *location* of forgetting rather than its magnitude. A second architectural limit is orthogonal: because LayoutLMv3 is single-stream, the analysis localises forgetting by depth, input stream, and head rather than by a separable visual-attention or fusion component, which that architecture does not provide. A genuinely per-stream *attention* decomposition would require a two-stream backbone and is left to future work. Second, the task sequence comprises three public datasets (FUNSD, CORD, SROIE); while they span forms and receipts and a wide range of label-space sizes, they are modest in scale and do not cover every document genre, and the unified DIL schema is VALUE-dominant (KEY present only in FUNSD/SROIE), which is why DIL results are reported with per-class F1. Third, the modalities' contri-

bution is read behaviourally from the input-masking conditions (C1–C4) and against an external unimodal BERT baseline; the within-architecture conditions cannot, on their own, attribute forgetting to attention sub-blocks the single-stream encoder shares. Fourth, the work targets token-level key-information extraction and does not address entity linking, relation extraction, or generative document IE. Fifth, all results use the OCR transcriptions and bounding boxes supplied with the benchmark datasets and never re-run a recogniser; this is a deliberate choice that removes OCR quality as a confounder for the diagnosis, but it also means the findings are established under clean, dataset-provided text and layout. A deployed pipeline would introduce its own OCR engine, whose recognition and layout-detection errors differ across document genres and could interact with the per-component forgetting picture; quantifying that interaction is left to future work. Sixth, each task is trained to convergence by early stopping on a validation signal, but FUNSD and SROIE provide no native validation split, so for those datasets the task's evaluation set serves as both the stopping signal and the reporting set, a mild optimistic selection bias, bounded by the short patience and the same rule applied to every method, and disclosed in §5.3.4; a held-out split carved from training data would remove it and is a camera-ready refinement. Seventh, LexSlot's lexical gate is expected to be ineffective on class-incremental learning, where same-dataset sessions collapse the signature contrast the gate relies on, so its claims are deliberately confined to the domain-shift setting, which is why we frame it as promising rather than established. Ninth, lexically-routed latent replay is evaluated on the domain-incremental scenario and LayoutLMv3 at two memory sizes, so its transfer to other scenarios and backbones is untested. Finally, results rest on three seeds; more seeds would tighten the confidence intervals around the smaller effects.

7.2 Conclusions

This thesis studied a stream of key-information-extraction tasks learned sequentially by a pre-trained tri-modal document encoder, under class-incremental, domain-incremental, and mixed regimes, with a bounded memory and no access to past raw documents. It asked two questions: *where* in the encoder forgetting concentrates (RQ1), and *what* a memory-bounded remedy must supply at that site, and *which* stored evidence it should re-expose (RQ2).

RQ1: where forgetting lives. A per-component diagnostic (controlled modality masking of a single LayoutLMv3, read out by per-token CKA drift, per-group Fisher information, and old-task-Fisher-weighted displacement, and repeated on LiLT, BROS, and a text-only BERT) gives three answers. Forgetting is *output-side*: under naive sequential fine-tuning

an earlier task that scores 88 entity-F1 collapses to $\sim$2 F1 after one further task, the head has over ten times the Fisher importance of any other parameter group, and CKA drift rises monotonically with depth (from 1.00 at the input encoders to 0.25 at the late layers and 0.16 at the head). It is *architecture-general*: every backbone, the unimodal one included, reproduces the site, so it is not a multimodal artefact; because LayoutLMv3 is single-stream we do not claim a fusion-specific effect. And it is *conserved*: freezing the diagnosed head and late layers leaves naive-level forgetting (BWT -76.8) and moves the drift into the previously stable early and mid layers (CKA $\approx 1.0 \rightarrow 0.19$), while a fixed consolidation budget is usable only when concentrated on the site ($\sim$85 AA) and under-fits when spread uniformly (42.6 AA).

RQ2: what the remedy must supply. The benchmark ordering follows the diagnosis: replay $\gg$ regularisation $\gg$ distillation. Replay re-exercises old targets at the head and reaches the joint upper bound on domain-incremental learning (AA ≈ 88 vs the oracle's 88.7, BWT ≈ 0); EWC curbs forgetting under a domain shift at an accuracy cost; LwF and the prompt/LoRA families track the naive bound. The buffer-free methods fail for one reason: the head's output marginal on old tasks snaps to the newest task's label marginal while the representation beneath it drifts, and pre-registered eval-time and marginal-level corrections recover nothing, because the old-class probability mass is destroyed before any correction can act. What replay must then preserve is *whole-document consistency*: replaying intact documents retains 87.3 AA, while every summary that breaks the binding of feature, position, and label (a rank-16 projection, per-class Gaussians, even real k-means centroids) collapses to the naive level. CoLaR, which stores each document as its own rank-r SVD factors, respects that law and reaches 87.6 AA with a 60 MB memory, $2.7\times$ smaller than raw replay, without retaining any raw page.

RQ2: which evidence to re-expose. LexSlot, an isolated slot memory at the diagnosed site, approaches replay-level accuracy on domain-incremental learning (AA 87.3 ± 1.0, BWT -2.3 versus $-2.8/-3.4$ for plain replay) with a compact 200-exemplar buffer; its lasting element is the drift-immune OCR-vocabulary signature that gates it. Lexically-routed latent replay joins that gate to CoLaR's memory: for each incoming document it replays the stored latents chosen by lexical similarity rather than uniformly, with the memory and loss held fixed so that any change is the gate's alone. Routing towards the lexically nearest stored documents does not pass the pre-registered one-point rule once three seeds are in (best arm $+0.5$ AA, two of three seeds agreeing), so the gate is retired as a component and the memory is sampled uniformly; what moves the class-incremental scenarios is keeping the stored labels live.

Taken together, the diagnosis decides *where* a remedy attaches, the negative results decide *what form* its memory must take, and the lexical gate decides *what* it replays. The diagnostic protocol, the benchmark, and the tooling stand independently of the remedy, as an instrument for designing continual-learning methods for multimodal document encoders that the prior, fragmented literature lacked.

7.3 Future Work

The convergence-trained grid is now complete, and the measured result is that the head/late-targeted LexSlot approaches replay with a compact 200-exemplar buffer on the domain shift, forgetting less than the plain replay methods, competitive with but not beating the strongest replay variant, while the depth-targeting probe and LexSlot's placement ablation support H_{target} and H_{head}. The benchmark now spans five scenarios, including large-scale class-incremental learning (WildReceipt) and cross-lingual domain-incremental learning (XFUND); the cross-lingual result, near-zero forgetting under pure language shift (naive BWT only -6.6), further corroborates the output-side thesis. Beyond that, several extensions follow naturally and are reserved for post-thesis work; each is a future direction rather than part of the present AAAI-phase scope.

Scaling and broadening the diagnosis. The controlled diagnosis is established for an encoder-only, base-scale backbone. A natural next step is to re-run the same per-component protocol on larger and architecturally different backbones: LayoutLMv3-large, the multilingual LayoutXLM, and decoder-based or instruction-tuned document MLLMs such as InternVL3, to test whether the dominant-forgetting site is a property of the LayoutLMv3-base design or a more general feature of tri-modal document encoders. A particularly important extension is to a *two-stream* backbone (for example LiLT, whose text and layout streams are physically separate parameter blocks): there the per-stream *attention* decomposition that a single-stream encoder cannot support becomes identifiable, so the modality-partitioned mechanisms set aside in §3.3.2, component-banked LoRA and modality-routed prompts, could be evaluated on signals that genuinely exist. Decoder-based encoders are especially interesting because their generative objective and tokenisation differ markedly from the sequence-labelling setup used here.

Additional continual scenarios. The three studied scenarios cover class-incremental and domain-incremental shifts. Future work could add task-incremental learning, where a task identifier is available at inference, and combined class-and-domain shifts that vary the label space and the document genre simultaneously; both would stress the components

differently and test whether a single mechanism-targeted remedy transfers across regimes.

Cross-lingual continual document IE. The benchmark now includes a cross-lingual domain-incremental scenario over XFUND (de $\rightarrow$ es $\rightarrow$ fr $\rightarrow$ it $\rightarrow$ zh, §5.2.4) and a larger-scale class-incremental scenario over WildReceipt, so language shift and corpus scale are now first-class axes rather than gaps. The open question these enable, left for deeper analysis, is whether the per-component diagnosis itself shifts under language change: when script and vocabulary change while layout statistics stay comparatively stable, does forgetting move toward the text pathway, or does it remain output-side as in the English setting? Running the full per-component diagnostic (CKA, per-group Fisher) inside the cross-lingual scenario, and extending it to a wider language set, is the natural continuation.

Richer, layout-aware remedies. The diagnosis points to a specific component; a complementary direction is to enrich the remedy on the data side with a document-aware structural replay that rehearses not raw tokens but layout-structured exemplars (for example region- or block-level representations), exploiting the very spatial structure that distinguishes document IE from plain-text IE. Such a replay could be combined with the mechanism-targeted approach rather than replacing it.

Joint multi-task continual IE. Finally, learning token-level extraction together with relation extraction (and, eventually, entity linking) in one continual framework is an open and high-value direction: it would move the benchmark from single-task sequence labelling towards the joint document understanding that downstream applications ultimately require.

These extensions are intentionally outside the present scope, which prioritises a rigorous diagnosis and a single well-justified, diagnosis-derived remedy over breadth; they sketch the trajectory for a fuller follow-up study.

Bibliography

Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *European Conference on Computer Vision (ECCV)*, 2018.

Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10 (2):251–276, 1998.

Srikar Appalaraju, Bhavan Jasani, Bhargava Urala Kota, Yusheng Xie, and R. Manmatha. DocFormer: End-to-end transformer for document understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

Ang Bian, Wei Li, Hangjie Yuan, Chengrong Yu, Mang Wang, Zixiang Zhao, Aojun Lu, Pengliang Ji, and Tao Feng. Make continual learning stronger via C-Flat. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Matteo Boschini, Lorenzo Bonicelli, Pietro Buzzega, Angelo Porrello, and Simone Calderara. Class-incremental continual learning into the extended DER-verse. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2022. Mammoth framework.

Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: A strong, simple baseline. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations (ICLR)*, 2019.

Chen, Cao, Li, et al. Incremental sequence labeling: A tale of two shifts. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. arXiv:2402.10447. TODO: complete author list before camera-ready.

Chen et al. SEFE: Superficial and essential forgetting eliminator for multimodal continual instruction tuning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

Li Cui, Deqing Yang, Jiaxin Yu, Chengwei Hu, Jiayang Cheng, Jingjie Yi, and Yanghua Xiao. Refining sample embeddings with relation prototypes to enhance continual relation extraction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP)*, 2021.

Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. Orthogonal gradient descent for continual learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020.

Ge et al. D-MoLE: Dynamic layer-wise mixture-of-lora-experts for continual multimodal instruction tuning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

Xu Han, Yi Dai, Tianyu Gao, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. Continual relation learning via episodic memory activation and reconsolidation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Jiangpeng He, Zhihao Duan, and Fengqing Zhu. CL-LoRA: Continual low-rank adaptation for rehearsal-free class-incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Teakgyu Hong, Donghyun Kim, Mingi Ji, Wonseok Hwang, Daehyun Nam, and Sungrae Park. BROS: A pre-trained language model focusing on text and layout for better key information extraction from documents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia (MM)*, 2022.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-

Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences (PNAS)*, 114(13):3521–3526, 2017.

Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

Kumar et al. ProtoNER: Few-shot incremental named entity recognition for visually-rich documents. In *International Workshop on Document Analysis Systems (DAS)*, 2024.

Anh Duc Le, Nam Le Hai, Thanh Xuan Nguyen, Linh Ngo Van, Nguyen Thi Ngoc Diep, Sang Dinh, and Thien Huu Nguyen. Enhancing discriminative representation in similar relation clusters for few-shot continual relation extraction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 2450–2467, 2025a.

Minh Le, Tien Ngoc Luu, An Nguyen The, Thanh-Thien Le, Trang Nguyen, Tung Thanh Nguyen, Linh Ngo Van, and Thien Huu Nguyen. Adaptive prompting for continual relation extraction: A within-task variance perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025b.

Lei et al. Symbolic replay: Scene graph as prompt for continual learning on VQA task. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

Siqi Li, Yufan Shen, Xiangnan Chen, Jiayi Chen, Hengwei Ju, Haodong Duan, Song Mao, Hongbin Zhou, Bo Zhang, Bin Fu, et al. GDI-Bench: A benchmark for general document intelligence with vision and reasoning decoupling. *arXiv preprint arXiv:2505.00063*, 2025.

Zhizhong Li and Derek Hoiem. Learning without forgetting. In *European Conference on Computer Vision (ECCV)*, 2016.

Yan-Shuo Liang and Wu-Jun Li. InfLoRA: Interference-free low-rank adaptation for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Minqian Liu and Lifu Huang. Teamwork is not always good: An empirical study of classifier drift in class-incremental information extraction. In *Findings of the Association for Computational Linguistics: ACL*, 2023.

Vincenzo Lomonaco, Lorenzo Pellegrini, Andrea Cossu, Antonio Carta, Gabriele Graffieti, Tyler L. Hayes, Matthias De Lange, Marc Masana, Jary Pomponi, Gido M. van de Ven, Martin Mundt, Qi She, Keiland Cooper, Jeremy Forest, Eden Belouadah, Simone Calderara, German I. Parisi, Fabio Cuzzolin, Andreas S. Tolias, Simone Scardapane, Luca Antiga, Subutai Ahmad, Adrian Popescu, Christopher Kanan, Joost van de Weijer, Tinne Tuytelaars, Davide Bacciu, and Davide Maltoni. Avalanche: an end-to-end library for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2nd Continual Learning in Computer Vision Workshop*, 2021. MIT-licensed.

David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

Marc Masana, Xialei Liu, Bartłomiej Twardowski, Mikel Menta, Andrew D. Bagdanov, and Joost van de Weijer. Class-incremental learning: Survey and performance evaluation on image classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(5):5513–5533, 2023.

Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24: 109–165, 1989.

Mark D. McDonnell, Dong Gong, Amin Parvaneh, Ehsan Abbasnejad, and Anton van den Hengel. RanPAC: Random projections and pre-trained models for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Mohammad Minouei, Khurram Azeem Hashmi, Mohammad Reza Soheili, Muhammad Zeshan Afzal, and Didier Stricker. Continual learning for table detection in document images. *Applied Sciences*, 12(18):8969, 2022.

Vinay V. Ramasesh, Ethan Dyer, and Maithra Raghu. Anatomy of catastrophic forgetting: Hidden representations and task semantics. In *International Conference on Learning Representations (ICLR)*, 2021.

Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. iCaRL: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Gobinda Saha, Isha Garg, and Kaushik Roy. Gradient projection memory for continual learning. In *International Conference on Learning Representations (ICLR)*, 2021.

Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress: A scalable framework for continual learning. In *International Conference on Machine Learning (ICML)*, 2018.

James Seale Smith, Leonid Karlinsky, Vyshnavi Gutta, Paola Cascante-Bonilla, Donghyun Kim, Assaf Arbelle, Rameswar Panda, Rogerio Feris, and Zsolt Kira. CODA-Prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

Tejas Srinivasan, Ting-Yun Chang, Leticia Pinto Alva, Georgios Chochlakis, Mohammad Rostami, and Jesse Thomason. CLiMB: A continual learning benchmark for vision-and-language tasks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

Tung Tran, Danilo Vargas, and Khoat Than. LCA: Local classifier alignment for continual learning. In *International Conference on Learning Representations (ICLR)*, 2026.

Jiapeng Wang, Lianwen Jin, and Kai Ding. LiLT: A simple yet effective language-independent layout transformer for structured document understanding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022a.

Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2024.

Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. Orthogonal subspace learning for language model continual learning. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

Yabin Wang, Zhiwu Huang, and Xiaopeng Hong. S-Prompts learning with pre-trained transformers: An occam's razor for domain incremental learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022b.

Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. DualPrompt: Complementary prompting for rehearsal-free continual learning. In *European Conference on Computer Vision (ECCV)*, 2022c.

Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022d.

Yawen Yang, Fukun Ma, Shiao Meng, Aiwei Liu, and Lijie Wen. GenCNER: A generative framework for continual named entity recognition. *arXiv preprint arXiv:2510.11444*, 2025.

Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

Yuexiang Zhai, Shengbang Tong, Xiao Li, Mu Cai, Qing Qu, Yong Jae Lee, and Yi Ma. Investigating the catastrophic forgetting in multimodal large language model fine-tuning. In *Proceedings of the Conference on Parsimony and Learning (CPAL)*, volume 234 of *Proceedings of Machine Learning Research*, pages 202–227, 2024.

Duzhen Zhang, Wei Cong, Jiahua Dong, Yahan Yu, Xiuyi Chen, Yonggang Zhang, and Zhen Fang. Continual named entity recognition without catastrophic forgetting. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023a.

Gengwei Zhang, Liyuan Wang, Guoliang Kang, Ling Chen, and Yunchao Wei. SLCA: Slow learner with classifier alignment for continual learning on a pre-trained model. In *ICCV*, 2023b.

Yao Zhang et al. CL-CrossVQA: A continual learning benchmark for cross-domain visual question answering. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025.

Zhao et al. Depth-aware hierarchical replay for continual learning in knowledge-based question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024a.

Zhao et al. SAFE: Slow and fast parameter-efficient tuning for continual learning with pre-trained models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.

Kang Zhao, Hua Xu, Jiangong Yang, and Kai Gao. Consistent representation learning for continual relation extraction. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022.

Junhao Zheng et al. Distilling causal effect from miscellaneous other-class for continual named entity recognition. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

Junhao Zheng et al. SpanKL: A span-based knowledge-localisation method for continual named entity recognition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

Da-Wei Zhou, Fu-Yun Wang, Han-Jia Ye, and De-Chuan Zhan. PyCIL: A Python toolbox for class-incremental learning. *Science China Information Sciences*, 66(9), 2023. MIT-licensed.

Appendix A

Supplementary Material

A.1 Key Hyperparameters

Table A.1 lists the default training hyperparameters shared across runs. Method-specific hyperparameters (e.g. the EWC penalty λ, the replay-buffer size, the prompt-pool size, the LoRA rank) are set per method and recorded in the corresponding configuration files.

A.1.1 CIL-CORD Session Class Lists

Table A.2 records the exact six CORD fine-grained classes introduced at each session of the CIL-CORD scenario (§5.2.1, Table 5.2). Class names follow CORD's native `category.field` naming; each class contributes a B- and an I- tag to the BIO label space.

A.2 Per-Component Metric Definitions

Linear CKA. For centred activation matrices $X \in \mathbb{R}^{n \times p}$ and $Y \in \mathbb{R}^{n \times q}$ collected on a fixed probe set,

$$\mathrm{CKA}(X, Y) = \frac{\|Y^\top X\|_F^2}{\|X^\top X\|_F \, \|Y^\top Y\|_F},$$

which is invariant to orthogonal transforms and isotropic scaling (Kornblith et al., 2019). Layer-wise CKA between checkpoints quantifies representational drift.

Table A.1: Shared training defaults (top) and the main method-specific hyperparameters with their sweep ranges (bottom). Scenario-specific values are recorded in the released configurations.

Hyperparameter	**Value**
Shared	
Backbone	LayoutLMv3-base (12 layers, hidden 768)
Optimiser	AdamW (weight decay 0.01)
Learning rate	5×10^{-5}
LR schedule	constant (no warmup)
Gradient clipping	max-norm 1.0
Batch size	8
Gradient accumulation	1
Epochs per task	10
Precision	fp32 (fp16 flag available, off by default)
Image resolution	224×224
Box coordinate range	$[0, 1000]^4$
Seeds	42, 123, 7
Determinism	fixed seeds; deterministic cuDNN
Method-specific (default; sweep)	
EWC penalty λ	1000; $\{10^2, 10^3, 10^4\}$
EWC online decay γ	1
Fisher estimation samples	200
LwF α, temp. T	1.0, 2.0; $\alpha \in \{0.5, 1, 2\}$
Replay buffer	200; $\{200, 500\}$
ER replay batch size	8
DER++ α, β	0.5, 0.5; logit weight $\in \{0.1, 0.5, 1\}$
LexSlot head / late slots	48 / 18 (per-task budget n_{slots}/T)
LexSlot repr-slot rank r	16
LexSlot sharing; depth	off (isolated); head_late
L2P pool / length / top-K	10 / 5 / 5
L2P key-loss weight λ_{key}	0.5
LoRA rank / α / dropout	8 / 16 / 0.1; rank $\in \{4, 8, 16\}$
O-LoRA targets / λ_{ortho}	Q, K, V / 0.5
CKA probe set	500 samples (mean-pooled)

Table A.2: The six CORD fine-grained classes introduced at each CIL-CORD session. Shared prefixes are factored for compactness.

Session	Classes introduced
0	`menu.{cnt, discountprice, itemsubtotal, nm, num, price}`
1	`menu.{unitprice, vatyn}`; `menu.sub.{cnt, nm, price, unitprice}`
2	`sub_total.{discount_price, etc, othersvc_price, service_price, subtotal_price, tax_price}`
3	`total.{cashprice, changeprice, creditcardprice, emoneyprice, menuqty_cnt, menutype_cnt}`
4	`total.{total_etc, total_price}`; `void_menu.{nm, price}`; `sub.{nm, cnt}`

Per-group Fisher information. For parameter group g and data $\mathcal{D}$,

$$F_g = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \sum_{\phi\in g} \Big(\frac{\partial \log p_\theta(y \mid x)}{\partial \phi} \Big)^2 .$$

Parameters are aggregated into the eight groups of §3.1.1.

Continual-learning metrics. Average accuracy, backward transfer, forward transfer, and average forgetting are defined in §5.5.2.

A.3 Extended Per-Task Result Tables

Once the full experimental grid completes, this section will report the detailed tables underlying the aggregate numbers in the main text: the per-task accuracy matrix $R[i, j]$ (accuracy on task j after training through task i) for each method, scenario, and seed, from which average accuracy, backward transfer, and average forgetting are derived; the per-condition layer-wise CKA and per-group Fisher values that support the diagnosis; and per-seed breakdowns with means and standard deviations. All values will be regenerated from the tracked logs so that they remain consistent with the figures and summary tables.

[To be added once the full grid completes: per-task accuracy matrices, per-condition CKA and Fisher tables, and per-seed breakdowns.]